NESTOR

ENTERPRISE ARCHITECTURE HANDBOOK



Tài liệu lưu hành nội bộ

Lời Mở Đầu: Triết Lý Thiết Kế Của Cuốn Sách

Chào mừng bạn đọc đến với cuốn sách **Thiết Kế & Xây Dựng Hệ Thống Backend Doanh Nghiệp Với NestJS** (*Enterprise Backend Engineering with NestJS*). Cuốn sách này được biên soạn với tôn chỉ trở thành một **giáo trình kỹ thuật chuyên sâu** và một **cẩm nang tham chiếu mã nguồn hoàn chỉnh**, dẫn dắt người đọc qua từng ngóc ngách của quá trình phát triển hệ thống backend quy mô lớn.

Trong nền công nghiệp phần mềm hiện đại, Node.js đã chứng minh được sức mạnh vượt trội về xử lý I/O bất đồng bộ và hệ sinh thái gói thư viện khổng lồ. Tuy nhiên, các khung làm việc truyền thống như Express hay Koa thường mang tính phi cấu trúc (*unopinionated*), trao toàn bộ quyền tự do cấu trúc mã nguồn cho lập trình viên. Sự tự do này thường dẫn đến tình trạng mã nguồn phân mảnh, thiếu tính nhất quán và trở thành gánh nặng kỹ thuật khổng lồ khi dự án phình to.

NestJS ra đời như một bước tiến mang tính cách mạng: kết hợp mô hình **Inversion of Control (IoC)** và **Dependency Injection (DI)** chuẩn mực, tận dụng hệ thống kiểu tĩnh mạnh mẽ của **TypeScript**, và thiết lập một kiến trúc phân tầng mạch lạc lấy cảm hứng từ Clean Architecture và Domain-Driven Design.

Phương pháp tiếp cận sư phạm của cuốn sách:

1. **Dạy kỹ lưỡng mọi khía cạnh từ gốc rễ**: Mỗi khái niệm (IoC Scopes, Lifecycle Hooks, Exception Filters, ORM Data Mapper, Argon2 Hashing, Factory Pattern...) đều được phân tích cặn kẽ về nguyên lý hoạt động, động lực kiến trúc, so sánh ưu nhược điểm với các giải pháp khác và minh họa bằng mã nguồn thực tế.
2. **Trực quan hóa với 100% Lược đồ Chuẩn Hóa**: Toàn bộ các lược đồ luồng dữ liệu, chu trình sống của request, kiến trúc phân tầng và quan hệ đối tượng đều được thiết kế tỉ mỉ bằng TikZ với tỷ lệ chuẩn hóa vừa vặn 95% bề rộng trang in, bảo đảm tính mạch lạc và mỹ thuật cao nhất.
3. **Dành riêng Phần Phụ Lục Trang Trọng (Dedicated Appendix)**: Toàn bộ thông tin khảo sát mã nguồn, lược đồ CSDL toàn dự án (ERD), cùng giải pháp thực chiến hoàn chỉnh giải quyết trọn vẹn bài toán Quản lý Công việc cá nhân (**Task2.md**) được đúc kết thành một phần Phụ lục quy chuẩn ở cuối sách.

Hy vọng cuốn sách sẽ là người bạn đồng hành tin cậy, giúp bạn nâng tầm tư duy từ một lập trình viên viết code đơn thuần thành một Kỹ sư Phần mềm thực thụ, tự tin thiết kế và vận hành những hệ thống phần mềm doanh nghiệp bền vững.

MỤC LỤC TỔNG QUAN

Lời Mở Đầu: Triết Lý Thiết Kế Của Cuốn Sách **1**

I Nền Tảng Kiến Trúc Cốt Lõi NestJS & Chu Trình Sống (Core Architecture) **7**

1 Triết Lý Thiết Kế & Hệ Sinh Thái NestJS **8**
1.1 Bối Cảnh Lịch Sử: Từ Callback Hell Đến Khung Làm Việc Doanh Nghiệp . 8
1.2 Ba Trụ Cột Cốt Lõi Của Ứng Dụng NestJS 9
1.3 Module Động (Dynamic Modules) & Các Mẫu Thiết Kế Doanh Nghiệp . . . 9
1.4 Bản Đồ Cây Phụ Thuộc Module & Ranh Giới Đóng Gói 10
1.5 Cơ Chế TypeScript Metadata Reflection 11
1.6 So Sánh Toàn Diện: NestJS vs Express Thuần vs Fastify 12
1.7 Kiến Trúc Phân Tầng Chuyên Sâu Của Dự Án 12
1.8 Giải Phẫu Chu Trình Bootstrap Khởi Động Tại `src/main.ts` 12

2 Inversion of Control (IoC) & Dependency Injection Chuyên Sâu **16**
2.1 Nguyên Lý Đảo Ngược Phụ Thuộc (Dependency Inversion) 16
2.2 Cơ Chế Bên Dưới Của IoC Container (Container Internals) 16
2.3 Bốn Cú Pháp Khai Báo Custom Providers 17
2.4 Phân Tích Chuyên Sâu 3 Injection Scopes 18
2.5 Hiệu Ứng Lan Truyền (Bubble-up Effect) Của `Scope.REQUEST` 19
2.6 Xử Lý Phụ Thuộc Vòng (Circular Dependency) Với `forwardRef()` 19
2.7 Bốn Hooks Vòng Đời Của Ứng Dụng & Cơ Chế Graceful Shutdown 20

3 Chu Trình Sống 12 Bước Của HTTP Request **21**
3.1 Đường Ống Xử Lý Yêu Cầu (The Request Lifecycle Pipeline) 21
3.2 Phân Biệt Rạch Ròi Vai Trò Của Các Thành Phần Pipeline 21
3.3 Cơ Chế Đa Giao Thức: `ExecutionContext` vs `ArgumentsHost` 23

4 Thẩm Định Dữ Liệu Với ValidationPipe & DTO Pattern **24**
4.1 Bảo Vệ Hệ Thống Trước Lỗ Hổng Mass Assignment 24
4.2 Bộ Ba Cờ Cấu Hình Toàn Cục Trong `src/main.ts` 24
4.3 Kế Thừa DTO Linh Hoạt Với `PartialType()` 25
4.4 Phân Tách DTO Quản Trị Viên (Admin) & Người Dùng Thường (Client) . . 26
4.5 Chuyển Đổi Kiểu Tham Số URL Với `class-transformer` 26
4.6 Tích Hợp Giữa `ValidationPipe` & `ErrorService` 27

II Tầng Dữ Liệu & ORM Doanh Nghiệp Với TypeORM (Database Engineering) 28

5 Kiến Trúc Truy Cập Dữ Liệu Với TypeORM 0.3 29
5.1 Mẫu Thiết Kế Data Mapper vs Active Record . . . 29
5.2 Cấu Hình DataSource Tập Trung Trong TypeORM 0.3 . . . 29
5.3 Quản Lý Hồ Kết Nối (Connection Pooling) & Tải Cao . . . 30
5.4 Nguy Cơ Tiềm Ẩn Của Cờ `synchronize: true` . . . 31
5.5 Quản Lý Giao Dịch Dữ Liệu (Transactions) Với `QueryRunner` . . . 31

6 Mô Hình Hóa Thực Thể & Decorators Tùy Biến 33
6.1 Lớp Cơ Sở Trừu Tượng `AbstractEntity` & Mẫu `toDto()` . . . 33
6.2 Khóa Chính Toàn Cầu UUID v4 vs BigInt Auto-Increment . . . 34
6.3 Chiến Lược Đặt Tên Tự Động (`SnakeNamingStrategy`) . . . 34
6.4 Giải Phẫu Bộ Decorators Tùy Biến Chuyên Sâu . . . 34

7 Chiến Lược Di Trú Không Gián Đoạn 36
7.1 Bản Chất Của TypeORM Migrations & Triết Lý Bất Biến . . . 36
7.2 Mô Hình Di Trú Mở Rộng & Thu Hẹp (The Expand and Contract Pattern) 36
7.3 Phân Cấp Khóa Bảng (Table Lock Hierarchy) Trong PostgreSQL . . . 37
7.4 Đánh Chỉ Mục Phi Khóa Với `CREATE INDEX CONCURRENTLY` . . . 38
7.5 Kỹ Thuật Di Chuyển Dữ Liệu Theo Lô (Batch Backfilling) . . . 39
7.6 Vận Hành Tự Động Hóa Migrations Qua NPM Scripts . . . 39

8 Khởi Tạo Dữ Liệu Mẫu Tự Động Với Hygen Seeding 41
8.1 Kiến Trúc Động Cơ Seeding Tại `src/database/typeorm/seeds/` . . . 41
8.2 Giải Phẫu Bộ Điều Phối `run-seed.ts` . . . 41
8.3 Tích Hợp Hygen Code Generator Tạo Mẫu Tự Động . . . 42

III An Ninh, Xác Thực & Phân Quyền Doanh Nghiệp (Security & Auth) 43

9 Cơ Chế Xác Thực Kép Passport Local & JWT Strategy 44
9.1 Mô Hình Xác Thực Không Lưu Trạng Thái (Stateless Authentication) . . . 44
9.2 Giải Phẫu Chiến Lược Xác Thực `JwtStrategy` . . . 44

10 Mã Hóa Mật Khẩu Thế Hệ Mới Với Thuật Toán Argon2 47
10.1 Tại Sao Argon2 Vượt Trội Hơn Bcrypt & PBKDF2? . . . 47
10.2 So Sánh Chuyên Sâu Các Thuật Toán Băm Mật Khẩu . . . 47
10.3 Tích Hợp Tự Động Trong Entity & AuthService . . . 47

11 Đăng Nhập Xã Hội (Google OAuth2 ID Token Integration) 50
11.1 Kiến Trúc Định Danh OpenID Connect (OIDC) So Với OAuth 2.0 . . . 50
11.2 Luồng Xác Thực Google ID Token 4 Tác Nhân . . . 50
11.3 Giải Phẫu Thẩm Định Dữ Liệu Với `AuthGoogleLoginDto` . . . 50
11.4 Xác Minh Chữ Ký Điện Tử Bất Đối Xứng Với `google-auth-library` . . . 51
11.5 Phòng Ngự Tấn Công Chiếm Quyền Tài Khoản (Account Hijacking Defense) 53

12 Phân Quyền Theo Vai Trò (Role-Based Access Control - RBAC) 54
12.1 Sự Kết Hợp Giữa Decorator `@Roles()` & `RolesGuard` 54
12.2 Giải Phẫu Mã Nguồn `RolesGuard` 54

IV Hạ Tầng Dịch Vụ Dùng Chung & Tối Ưu Vận Hành (Cross-Cutting Concerns) 56

13 Hệ Thống Lọc Ngoại Lệ Toàn Cục (Exception Filters Pipeline) 57
13.1 Kiến Trúc Bắt Lỗi Tập Trung Trong Doanh Nghiệp 57
13.2 Giải Phẫu Mã Nguồn `BadRequestExceptionFilter` 57

14 Hệ Thống Nhật Ký Đa Môi Trường Với Winston Logger 60
14.1 Tầm Quan Trọng Của Structured Logging 60
14.2 Thiết Kế Factory Pattern Cho Logger Tại `src/core/loggers/` 60
14.3 Theo Dõi Yêu Cầu Với `RequestLoggerMiddleware` 61

15 Hệ Thống Phân Trang Doanh Nghiệp (Pagination Architecture) 63
15.1 Thiết Kế Bộ Ba Đối Tượng Phân Trang Chuẩn Hóa 63
15.2 Công Thức Toán Học & Câu Lệnh SQL Bên Dưới 64

16 Đa Ngôn Ngữ & Quản Trị Lỗi Với `nestjs-i18n` 66
16.1 Cấu Trúc Thư Mục Từ Điển Đa Ngôn Ngữ 66
16.2 Giải Phẫu Toàn Diện Dịch Vụ `ErrorService` 66

V Các Mẫu Thiết Kế Nghiệp Vụ Thực Tế (Business Patterns in Practice) 69

17 Mẫu Thiết Kế Factory Pattern Trong Thanh Toán Trực Tuyến 70
17.1 Ứng Dụng GoF Factory Method Trong Thanh Toán Đa Kênh 70
17.2 Giải Phẫu `PaymentFactory` & Vòng Đời `Scope.REQUEST` 71
17.3 Tích Hợp Cổng MoMo Với Chữ Ký Điện Tử HMAC-SHA256 72

18 Quản Lý Đơn Hàng Master-Detail & Giao Dịch Dữ Liệu 73
18.1 Kiến Trúc Quan Hệ Thực Thể Master-Detail Trong E-Commerce 73
18.2 Định Nghĩa Thực Thể Với Quan Hệ TypeORM 73
18.3 Chu Trình Xử Lý Đơn Hàng & Quản Lý Giao Dịch Nguyên Tử ACID 74
18.4 Giải Phẫu Mã Nguồn Xử Lý Giao Dịch Tại `OrderService` 74
18.5 Phòng Ngự Xung Đột Đồng Thời (Race Conditions) & Trừ Tồn Kho 77
18.6 Máy Trạng Thái Đơn Hàng (Order State Machine) & Capture Order 78

19 Lưu Trữ Tệp Tin Đám Mây Cloudinary (Media Pipeline) 79
19.1 Kiến Trúc Đường Ống Tải Lên Đa Phương Tiện 79
19.2 Giải Phẫu Mã Nguồn `MediaService` & Dọn Dẹp An Toàn 79

20 Kỹ Thuật Xóa Mềm Trong Cơ Sở Dữ Liệu Doanh Nghiệp 81
20.1 Bản Chất Của Xóa Mềm (Soft Delete) vs Xóa Cứng (Hard Delete) 81

20.2 Giải Phẫu Thực Thể PostEntity Với @DeleteDateColumn 81
20.3 Cơ Chế Tự Động Lọc SQL Phía Dưới Của TypeORM 82
20.4 Khôi Phục Bản Ghi Xóa Mềm & Truy Vấn Thùng Rác (Trash Bin) 83
20.5 Xung Đột Ràng Buộc Duy Nhất (Unique Constraint) & Giải Pháp Partial Index 83
20.6 Phân Định Kiến Trúc: Controller Công Khai vs Controller Quản Trị 84

VI Kiểm Thử Tự Động, DevOps & Tiêu Chuẩn Kỹ Thuật (Engineering Standards) 85

21 Chiến Lược Kiểm Thử Tự Động Toàn Diện Với Jest 86
21.1 Kim Tự Tháp Kiểm Thử Phần Mềm (The Test Pyramid) 86
21.2 Triết Lý Mock Repository Trong Kiểm Thử Đơn Vị 86
21.3 Kiểm Thử Các Kịch Bản Thành Công & Ngoại Lệ (Happy vs Error Paths) . 87

22 Tài Liệu Hóa Tự Động API Với Swagger / OpenAPI 88
22.1 Tiêu Chuẩn OpenAPI Specification 3.0 & Triết Lý API-First 88
22.2 Giải Phẫu Tệp Khởi Tạo Trung Tâm `src/setup-swagger.ts` 88
22.3 Cẩm Nang Tra Cứu Bộ Decorators OpenAPI Trong Controller & DTO . . . 90
22.4 Bảo Vệ Tài Liệu API Trên Môi Trường Sản Xuất 90

23 Container Hóa Hệ Thống Với Docker & Compose 92
23.1 Nguyên Lý Ảo Hóa Container & Multi-Stage Build 92
23.2 Lưu Ý Về Alpine Linux & Native Addons (Argon2) 93
23.3 Điều Phối Cụm Dịch Vụ Với Docker Compose 93

24 Quy Chuẩn Kỹ Thuật Mã Nguồn & Git Hooks 94
24.1 Mô Hình Phân Nhánh Git Flow Doanh Nghiệp 94
24.2 Quy Chuẩn Thông Điệp Commit (Conventional Commits) 94
24.3 Tự Động Hóa Kiểm Soát Chất Lượng Với Husky & Lint-staged 96
24.4 Lợi Ích Kinh Tế & Vận Hành Của Trạm Kiểm Soát Cục Bộ 97

Lời Kết: Con Đường Trở Thành Kỹ Sư Phần Mềm Cấp Cao 98

VII Phụ Lục Kỹ Thuật Chuyên Sâu (Technical Appendix) 99

A Toàn Cảnh Hệ Thống Thực Nghiệm NestJS Boilerplate 100
A.1 Kiến Trúc Tổng Thể & Cấu Trúc Thư Mục Dự Án 100
A.2 Bảng Khảo Sát Chi Tiết 8 Feature Modules Trong Hệ Thống 101
A.3 Lược Đồ Bản Đồ Phụ Thuộc Module Toàn Hệ Thống 102

B Giải Pháp Kỹ Thuật Toàn Diện Cho Module Tasks (Task2.md) 103
B.1 Bối Cảnh Nghiệp Vụ & Đặc Tả Bài Toán 103
B.2 Mã Nguồn Hiện Thực Hóa Chi Tiết 103
B.3 Lược Đồ Luồng Xử Lý 5 Endpoints Chuẩn Hóa (Resize 95%) 107
B.3.1 1. Luồng Tạo Mới Công Việc (POST /api/v1/tasks) 107

B.3.2 2. Luồng Lấy Danh Sách & Lọc Công Việc (GET /api/v1/tasks) . . 107
B.3.3 3. Luồng Xem Chi Tiết Công Việc Theo ID (GET /api/v1/tasks/:id) 107
B.3.4 4. Luồng Cập Nhật Từng Phần (PATCH /api/v1/tasks/:id) 107
B.3.5 5. Luồng Xóa Bản Ghi Tối Ưu (DELETE /api/v1/tasks/:id) 107
B.4 Kết Quả Kiểm Thử Đơn Vị Tự Động (Jest 100% PASS) 107

C Bộ Câu Hỏi Khảo Sát Kỹ Thuật & Đánh Giá Thiết Kế Hệ Thống (FAQ) 111

D Sổ Tay Vận Hành & Khởi Động Môi Trường Thực Địa 115
D.1 Bảng Tra Cứu Biến Môi Trường Cấu Hình (.env) 115
D.2 Hướng Dẫn Khởi Chạy 4 Bước Chuẩn Doanh Nghiệp 115
D.3 Cẩm Nang Xử Lý Sự Cố Thực Địa (Troubleshooting Playbook) 116
D.4 Bảng Kiểm Định Chất Lượng Hoàn Thành Bài Tập (Task2 Checklist) 116

PHẦN I

Nền Tảng Kiến Trúc Cốt Lõi NestJS & Chu Trình Sống (Core Architecture)

CHƯƠNG 1

Triết Lý Thiết Kế & Hệ Sinh Thái NestJS

1.1 Bối Cảnh Lịch Sử: Từ Callback Hell Đến Khung Làm Việc Doanh Nghiệp

Trong những năm đầu phát triển của nền tảng Node.js, cộng đồng kỹ sư phần mềm phải đối mặt với hai thách thức kỹ thuật lớn. Thách thức thứ nhất nằm ở tầng cú pháp: mô hình xử lý bất đồng bộ lồng nhau nhiều tầng (*Callback Hell*), khiến mã nguồn trở nên khó đọc, khó gỡ lỗi và dễ dẫn đến rò rỉ ngoại lệ chưa bắt được (*unhandled rejections*). Vấn đề này sau đó đã được tiêu chuẩn hóa và giải quyết hiệu quả bằng Promises và cú pháp `async/await` trong chuẩn ECMAScript hiện đại.

Tuy nhiên, thách thức thứ hai mang tính sống còn đối với các dự án phần mềm quy mô lớn: **sự thiếu vắng một kiến trúc ứng dụng đồng thuận chuẩn mực**. Các thư viện định tuyến phổ biến lúc bấy giờ như Express hay Koa chỉ hoạt động như một tầng định tuyến mỏng (*thin routing layer*). Chúng trao toàn bộ quyền tự do tổ chức thư mục, định nghĩa dịch vụ, kết nối cơ sở dữ liệu và quản lý vòng đời đối tượng cho từng cá nhân lập trình viên.

Hệ quả thực tế trong các doanh nghiệp là mỗi nhóm phát triển tự sáng tạo ra một kiến trúc riêng: nhóm này áp dụng mô hình MVC cổ điển, nhóm khác viết mã dạng script thủ tục trải dài hàng nghìn dòng, nhóm khác lại cố gắng mô phỏng mô hình hướng đối tượng nhưng thiếu một cơ chế quản trị phụ thuộc tập trung. Mã nguồn nhanh chóng rơi vào trạng thái suy thoái kiến trúc (*Architectural Erosion*), biến thành một "mớ bòng bong" (*Big Ball of Mud*). Trong môi trường đó, việc viết kiểm thử tự động (Unit Test, E2E Test) trở nên vô cùng tốn kém, các module bị dính chặt vào nhau (*tight coupling*), và việc bàn giao dự án giữa các thế hệ kỹ sư thường dẫn đến nguy cơ phải đập đi xây lại từ đầu.

NestJS được kiến trúc sư Kamil Myśliwiec khai sinh nhằm giải quyết dứt điểm cuộc khủng hoảng kiến trúc này. Lấy cảm hứng sâu sắc từ sự thành công rực rỡ của Angular ở phía Frontend và Spring Boot / ASP.NET Core ở phía Backend doanh nghiệp, NestJS mang lại:

- **Cấu trúc đồng thuận (Opinionated Architecture)**: Phân định rạch ròi vai trò và ranh giới của từng thành phần: Modules, Controllers, Services, Guards, Interceptors, Pipes và Exception Filters. Mọi thành viên trong dự án đều tuân thủ cùng một quy ước kiến trúc thống nhất.
- **Sức mạnh của TypeScript**: Ép buộc định nghĩa kiểu dữ liệu chặt chẽ ngay từ thời điểm viết mã, loại bỏ hoàn toàn các lỗi thời gian chạy ngớ ngẩn (như `undefined is not a function`), hỗ trợ tự động sinh tài liệu chuẩn OpenAPI/Swagger trực tiếp từ

các lớp DTO.

- **Tính độc lập với HTTP Engine (Platform Agnostic)**: Tách rời triệt để logic nghiệp vụ khỏi tầng mạng cấp thấp. Mặc định NestJS sử dụng Express Adapter, nhưng khi hệ thống đòi hỏi thông lượng cực cao, kỹ sư có thể hoán đổi sang Fastify Adapter chỉ bằng việc thay đổi một dòng mã khởi tạo.

1.2 Ba Trụ Cột Cốt Lõi Của Ứng Dụng NestJS

Một hệ thống NestJS tiêu chuẩn được cấu thành từ sự phối hợp nhịp nhàng của 3 khối xây dựng cơ bản:

1. **Modules (`@Module`)**: Là đơn vị đóng gói và phân định ranh giới chức năng (*Encapsulation Boundary*). Một Module gom nhóm các Controller và Provider có mối quan hệ nghiệp vụ mật thiết, đồng thời kiểm soát chính xác những gì được phơi bày ra bên ngoài thông qua mảng `exports`. Modules giúp chia tách mã nguồn thành các miền nghiệp vụ độc lập (*Bounded Contexts*) theo chuẩn Domain-Driven Design (DDD).
2. **Controllers (`@Controller`)**: Là tầng tiếp xúc trực tiếp với thế giới mạng Internet. Controller có nhiệm vụ tiếp nhận các yêu cầu HTTP (GET, POST, PATCH, DELETE...), phân giải các tham số định tuyến (*route params*), chuỗi truy vấn (*query strings*), và thân yêu cầu (*request body*). Controller kích hoạt các đường ống thẩm định (Pipes) và ủy thác toàn bộ việc xử lý cho tầng Service. Controller tuyệt đối không chứa logic nghiệp vụ hay truy vấn SQL trực tiếp.
3. **Providers (`@Injectable`)**: Là nơi lưu trữ "trái tim và khối óc" của ứng dụng — bao gồm các Dịch vụ nghiệp vụ (*Domain Services*), Kho lưu trữ dữ liệu (*Repositories*), Nhà máy khởi tạo (*Factories*), và các Trợ lý tiện ích (*Helpers*). Mọi Provider đều được đăng ký với IoC Container để tự động giải quyết phụ thuộc.

1.3 Module Động (Dynamic Modules) & Các Mẫu Thiết Kế Doanh Nghiệp

Trong bất kỳ dự án NestJS quy mô lớn nào, việc cấu hình tĩnh thông qua metadata đóng vai trò nền tảng. Tuy nhiên, khi cần cấu hình các dịch vụ hạ tầng phụ thuộc vào biến môi trường hoặc các tham số bất đồng bộ (như cấu hình CSDL, kết nối Redis, hoặc thông số cổng thanh toán), NestJS cung cấp cơ chế **Dynamic Modules**. Đây là một mẫu thiết kế nâng cao mà mọi Kỹ sư Doanh nghiệp đều phải nắm vững:

1. **Mẫu Thiết Kế `forRoot()`**: Thường được dùng để cấu hình một dịch vụ dạng Singleton duy nhất cho toàn bộ hệ thống ngay tại `AppModule` (ví dụ: `TypeOrmModule.forRoot(options)` hoặc `ConfigModule.forRoot()`). Nó chịu trách nhiệm khởi tạo kết nối cốt lõi và cung cấp các dịch vụ nền tảng.
2. **Mẫu Thiết Kế `forFeature()`**: Được dùng trong các Feature Modules con để định nghĩa và đăng ký các thành phần cụ thể trong phạm vi cục bộ của phân hệ đó mà không khởi tạo lại kết nối hạ tầng. Ví dụ, trong `TasksModule`, ta khai báo:

Ví dụ forFeature trong Feature Module

```
@Module({
  imports: [TypeOrmModule.forFeature([TaskEntity])], // Đăng ký Repository cục bộ
  controllers: [TasksController],
  providers: [TasksService],
})
export class TasksModule {}
```

3. **Mẫu Thiết Kế Bất Đồng Bộ `registerAsync()` / `forRootAsync()`**: Giải quyết bài toán tải cấu hình bất đồng bộ. Khi thông số kết nối CSDL hoặc khóa bí mật JWT cần được nạp động từ `ConfigService` hoặc một máy chủ quản lý khóa bí mật từ xa (như HashiCorp Vault hay AWS Secrets Manager), kỹ sư sử dụng `forRootAsync()` kết hợp với `useFactory` và mảng `inject`:

Ví dụ forRootAsync nạp cấu hình động

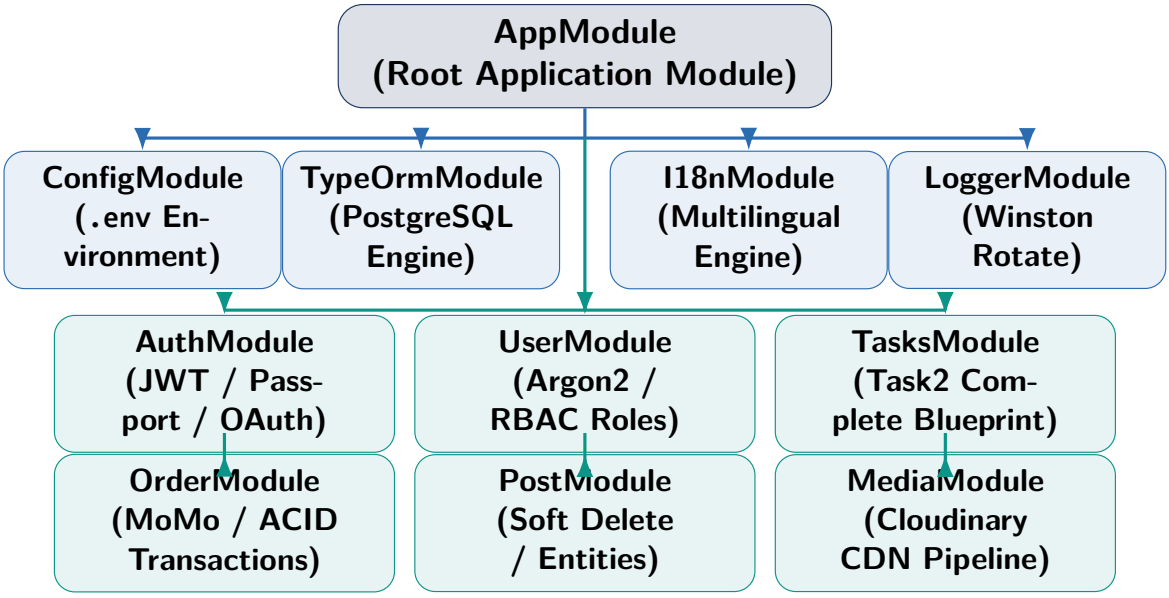
```
JwtModule.registerAsync({
  imports: [ConfigModule],
  inject: [ConfigService],
  useFactory: async (configService: ConfigService) => ({
    secret: configService.get<string>('JWT_SECRET'),
    signOptions: { expiresIn: configService.get<string>('JWT_EXPIRATION') },
  }),
})
```

4. **Decorator `@Global()`**: Khi một Module được đánh dấu là `@Global()`, toàn bộ các Provider trong mảng `exports` của nó sẽ có hiệu lực trên toàn hệ thống mà các module khác không cần phải nhập (*import*) thủ công. Mặc dù tiện lợi, quy tắc thiết kế doanh nghiệp khuyến cáo chỉ sử dụng `@Global()` cho các dịch vụ xuyên suốt tuyệt đối (như Logger, EventBus, hoặc Database Connection), tránh lạm dụng làm mờ ranh giới đóng gói giữa các phân hệ.

1.4 Bản Đồ Cây Phụ Thuộc Module & Ranh Giới Đóng Gói

Trong dự án `c:\Users\LAPTOP\nest`, hệ thống được tổ chức xoay quanh Module gốc `AppModule` (`src/app.module.ts`), đóng vai trò là điểm hội tụ của toàn bộ các module chức năng con:

Một nguyên lý tối quan trọng trong NestJS là **Ranh giới đóng gói**: Nếu Module A muốn sử dụng `UserService` của `UserModule`, thì `UserModule` bắt buộc phải liệt kê `UserService` trong mảng `exports: [UserService]`, và Module A phải đưa `UserModule` vào mảng `imports: [UserModule]`. Cơ chế này ngăn chặn việc truy cập tùy tiện xuyên module, buộc kỹ sư phải suy nghĩ kỹ càng về API công khai của từng phân hệ.



Hình 1.1: Sơ đồ cây phụ thuộc Module phân tầng chuẩn mực trong NestJS Enterprise

1.5 Cơ Chế TypeScript Metadata Reflection

Trọng tâm sức mạnh của NestJS nằm ở cơ chế **Decorators** kết hợp với thư viện `reflect-metadata`. Trong TypeScript, Decorator thực chất là một hàm bậc cao (*Higher-Order Function*) được trình biên dịch thực thi ngay khi định nghĩa lớp, phương thức, thuộc tính hoặc tham số:

$$\text{Decorator}(T) \longrightarrow \text{Reflect.defineMetadata}(\text{key}, \text{value}, T) \tag{1.1}$$

Khi tùy chọn `"emitDecoratorMetadata": true` được kích hoạt trong `tsconfig.json`, trình biên dịch TypeScript (`tsc`) sẽ tự động phân tích cú pháp kiểu dữ liệu và phát sinh 3 khóa metadata tiêu chuẩn vào bytecode JavaScript:

- `design:type`: Kiểu dữ liệu của thuộc tính hoặc trường được chú thích.
- `design:paramtypes`: Mảng các kiểu dữ liệu của tham số đầu vào trong constructor hoặc phương thức.
- `design:returntype`: Kiểu dữ liệu trả về của phương thức.

Nhờ `design:paramtypes`, khi kỹ sư viết:

Ví dụ constructor injection qua metadata

```
@Injectable()
export class TasksService {
  constructor(
    @InjectRepository(TaskEntity)
    private readonly tasksRepository: Repository<TaskEntity>,
  ) {}
}
```

NestJS không cần kỹ sư phải cấu hình thủ công chuỗi token định danh. IoC Container sử dụng `Reflect.getMetadata('design:paramtypes', TasksService)` để đọc trực tiếp các tham số và tự động tiêm (*inject*) đúng instance của `Repository<TaskEntity>` vào thời điểm khởi động.

1.6 So Sánh Toàn Diện: NestJS vs Express Thuần vs Fastify

Bảng 1.1: So sánh chuyên sâu giữa NestJS, Express thuần và Fastify

Tiêu chí	**NestJS (Dự án)**	**Express thuần**	**Fastify thuần**
Kiến trúc tổ chức	Mô-đun hóa, DDD, chuẩn doanh nghiệp.	Tự do hoàn toàn, dễ phân mảnh.	Dựa trên Schema, có hỗ trợ plugin.
Dependency Injection	Tích hợp sẵn IoC Container mạnh mẽ.	Không có (phải cài thư viện ngoài).	Không có tích hợp sẵn.
Type Safety	TypeScript hạng nhất, DTO validation.	JavaScript thuần (phải tự tích hợp).	Hỗ trợ schema qua JSON Schema.
Tự động hóa Swagger	Tự động sinh từ DTO và Decorators.	Viết file YAML/JSON thủ công.	Có plugin fastify-swagger.
Khả năng kiểm thử	Cực kỳ dễ dàng qua TestingModule	Phải monkey-patch hoặc mock thủ công.	Hỗ trợ inject HTTP request test.

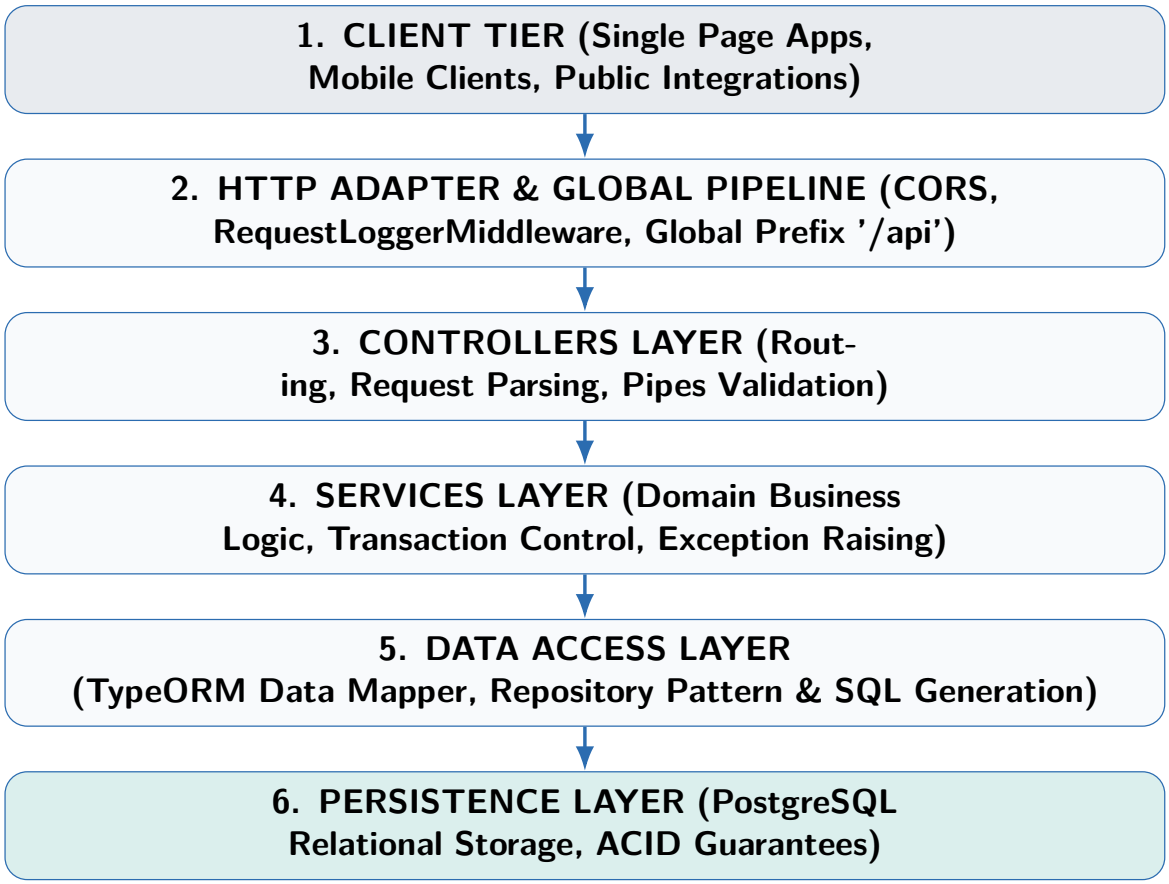
1.7 Kiến Trúc Phân Tầng Chuyên Sâu Của Dự Án

Hệ thống mã nguồn thực nghiệm `c:\Users\LAPTOP\nest` được thiết kế theo mô hình phân tầng chặt chẽ:

Dữ liệu di chuyển theo **Luồng Một Chiều (Unidirectional Data Flow)**: Yêu cầu từ Client đi tuần tự từ trên xuống dưới, và kết quả từ cơ sở dữ liệu được bao bọc ngược trở lại lên trên. Tầng dưới tuyệt đối không được gọi ngược trực tiếp lên tầng trên, bảo đảm tính độc lập và khả năng bảo trì tối đa.

1.8 Giải Phẫu Chu Trình Bootstrap Khởi Động Tại `src/main.ts`

Tệp tin `src/main.ts` là điểm nhập cảnh duy nhất (*Single Entry Point*) của ứng dụng. Hãy giải phẫu chi tiết mã nguồn khởi động thực tế của dự án:



Hình 1.2: Mô hình phân tầng kiến trúc 6 lớp trong NestJS Enterprise

Mã nguồn khởi động lõi trong src/main.ts

```
import { ValidationPipe } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
import { ProjectLogger } from '@core/loggers';
import { ConfigService } from '@nestjs/config';
import { requestLoggerMiddleware } from '@shared/middleware/request-logger.middleware';
import { setupSwagger } from './setup-swagger';
import { ExpressAdapter, NestExpressApplication } from '@nestjs/platform-express';

const logger = new ProjectLogger('bootstrap');

async function bootstrap(): Promise<NestExpressApplication> {
  // 1. Khởi tạo ứng dụng với Express Adapter và bật CORS
  const app = await NestFactory.create<NestExpressApplication>(
    AppModule,
    new ExpressAdapter(),
    { cors: true },
  );

  // 2. Thiết lập tiền tố URL toàn cục: Mọi route đều bắt đầu bằng /api
  app.setGlobalPrefix('/api');

  // 3. Đăng ký Middleware ghi log toàn cục
  app.use(requestLoggerMiddleware);
  app.enableCors();

  // 4. Kích hoạt đường ống thẩm định DTO toàn cục
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      stopAtFirstError: true,
      transform: true,
    }),
  );

  // 5. Khởi tạo hệ thống tài liệu tương tác Swagger OpenAPI
  setupSwagger(app);

  // 6. Đăng ký Middleware xử lý tiêu đề CORS nâng cao
  app.use((req, res, next) => {
    res.header('Access-Control-Allow-Origin', '*');
    res.header('Access-Control-Allow-Methods', 'GET,PUT,POST,DELETE');
    res.header('Access-Control-Allow-Headers', 'Content-Type, Accept');
    next();
  });

  // 7. Lấy cổng lắng nghe từ ConfigService và mở cổng kết nối
```

```
  const config = app.get<ConfigService>(ConfigService);
  await app.listen(config.get('PORT'), async () => {
    logger.info(`Application is running on: ${await app.getUrl()}`);
  });

  return app;
}
bootstrap();
```

Ý Nghĩa Của Từng Bước Trong Chu Trình Khởi Động

1. **CORS Configuration**: Dự án áp dụng cấu hình kép: vừa qua tùy chọn khởi tạo `{ cors: true }`, vừa qua middleware thiết lập tường minh các headers (`Access-Control-Allow-Origin`, `Methods`, `Headers`) nhằm bảo đảm khả năng tích hợp trơn tru với các Single Page Applications (React, Vue, Angular) chạy trên các domain khác nhau.
2. **Global Prefix `'/api'`**: Chuẩn hóa không gian tên cho mọi API backend, giúp phân biệt rõ ràng giữa các route nghiệp vụ và các route hệ thống (như giao diện Swagger tại `/docs` hoặc health check).
3. **ConfigService Discovery**: Thay vì đọc trực tiếp biến môi trường qua `process.env.PORT` vốn dễ gây lỗi thời gian chạy nếu biến chưa nạp, ứng dụng trích xuất `ConfigService` trực tiếp từ IoC Container thông qua `app.get<ConfigService>(ConfigService)` để bảo đảm tính an toàn kiểu dữ liệu.

CHƯƠNG 2

Inversion of Control (IoC) & Dependency Injection Chuyên Sâu

2.1 Nguyên Lý Đảo Ngược Phụ Thuộc (Dependency Inversion)

Trong thiết kế phần mềm hướng đối tượng theo bộ nguyên lý SOLID, chữ **D** đại diện cho **Dependency Inversion Principle (DIP)**:

> *"Các module cấp cao không nên phụ thuộc vào các module cấp thấp. Cả hai nên phụ thuộc vào sự trừu tượng (Abstraction). Sự trừu tượng không nên phụ thuộc vào chi tiết, mà chi tiết phải phụ thuộc vào sự trừu tượng."*

Khi một lớp tự khởi tạo đối tượng phụ thuộc của nó bằng từ khóa `new`:

Mã nguồn liên kết chặt chẽ - Phản mẫu thiết kế (Anti-pattern)

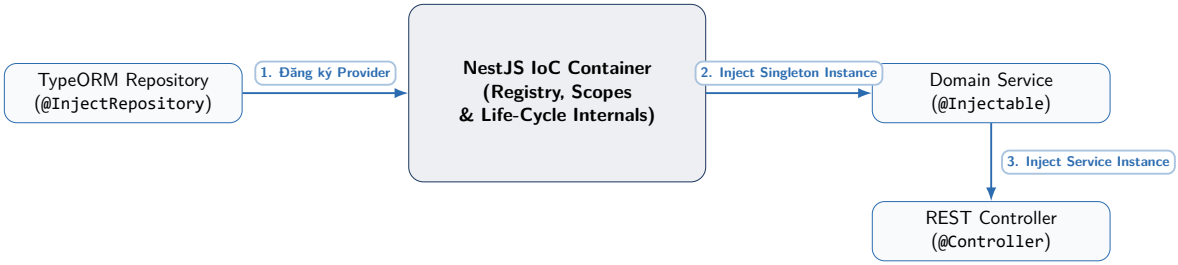
```
export class TasksService {
  private repository: TaskRepository;
  constructor() {
    this.repository = new PostgresTaskRepository(); // LIÊN KẾT CHẶT!
  }
}
```

Lớp `TasksService` bị "cột chặt" vào triển khai cụ thể của PostgreSQL. Kỹ sư không thể nào thay thế kho lưu trữ này bằng một Mock Repository khi viết Unit Test mà không phải sửa mã nguồn gốc!

NestJS hiện thực hóa hoàn chỉnh mô hình **Inversion of Control (IoC)**: Lập trình viên chỉ cần khai báo kiểu dữ liệu cần thiết trong constructor, việc khởi tạo, giải quyết thứ tự phụ thuộc và truyền instance vào sẽ do **IoC Container** phụ trách hoàn toàn:

2.2 Cơ Chế Bên Dưới Của IoC Container (Container Internals)

Khi ứng dụng NestJS boot, IoC Container thực hiện chu trình giải quyết phụ thuộc 3 pha:



Hình 2.1: Sơ đồ luồng giải quyết phụ thuộc qua NestJS IoC Container

1. **Pha Đăng Ký (Registration Phase)**: Quét toàn bộ các Module và nạp định nghĩa của các Provider vào một bảng đăng ký nội bộ (*Internal Registry*).
2. **Pha Xây Dựng Đồ Thị (Graph Resolution Phase)**: Sử dụng giải thuật sắp xếp topo (*Topological Sort*) trên cây phụ thuộc để phát hiện chu trình vòng lặp và xác định thứ tự khởi tạo chính xác từ các lá (leaf nodes) lên gốc.
3. **Pha Cấp Phát Thực Thể (Instantiation Phase)**: Khởi tạo các instance theo đúng thứ tự topo, lưu trữ các Singleton vào bộ nhớ đệm (*Singleton Cache* dạng `Map<Token, Instance>`) và tiêm vào các consumer tương ứng.

2.3 Bốn Cú Pháp Khai Báo Custom Providers

Bên cạnh cú pháp khai báo rút gọn `providers: [TasksService]`, NestJS hỗ trợ 4 dạng khai báo tường minh chuyên sâu:

1. **useClass**: Dùng để hoán đổi triển khai động theo môi trường (ví dụ môi trường test dùng MockService, môi trường production dùng RealService).

Khai báo useClass hoán đổi môi trường

```
{
  provide: TasksService,
  useClass: process.env.NODE_ENV === 'test' ? MockTasksService : TasksService,
}
```

2. **useValue**: Dùng để inject các đối tượng tĩnh, hằng số cấu hình, hoặc mock object vào cây phụ thuộc mà không cần khởi tạo class.

Khai báo useValue cho đối tượng cấu hình

```
{
  provide: 'APP_CONFIG',
  useValue: Object.freeze({ timeout: 5000, maxRetries: 3 }),
}
```

3. **useFactory**: Cho phép khởi tạo Provider một cách bất đồng bộ hoặc phụ thuộc vào kết quả tính toán từ các Provider khác (rất phổ biến khi khởi tạo kết nối database hoặc SDK bên thứ ba như Cloudinary hay Redis).

Khai báo useFactory bất đồng bộ

```
{
  provide: 'REDIS_CLIENT',
  useFactory: async (configService: ConfigService) => {
    const client = new Redis(configService.get('REDIS_URL'));
    await client.connect();
    return client;
  },
  inject: [ConfigService],
}
```

4. **useExisting**: Tạo ra một bí danh (*alias*) trỏ tới một provider đã tồn tại mà không khởi tạo thêm phiên bản mới.

Khai báo useExisting làm bí danh

```
{
  provide: 'ITasksService',
  useExisting: TasksService,
}
```

2.4 Phân Tích Chuyên Sâu 3 Injection Scopes

Mặc định trong NestJS, mọi provider đều hoạt động dưới dạng **Singleton** (`Scope.DEFAULT`). Đối tượng chỉ được khởi tạo một lần duy nhất lúc máy chủ khởi động và dùng chung cho toàn bộ các request:

Bảng 2.1: So sánh 3 Injection Scopes trong NestJS

Scope	Vòng đời thực thể	Phân tích ứng dụng trong dự án
`Scope.DEFAULT`	Khởi tạo duy nhất 1 lần (Singleton). Dùng chung cho toàn bộ requests.	Áp dụng cho 99% các services (`TasksService`, `UserService`). Tối ưu hóa bộ nhớ RAM và CPU tối đa.
`Scope.REQUEST`	Tạo mới instance cho mỗi HTTP request đi vào và tự hủy sau khi phản hồi.	Áp dụng tại `PaymentFactory` (`@Injectable({ scope: Scope.REQUEST })`) khi cần lưu giữ ngữ cảnh phiên thanh toán riêng biệt.
`Scope.TRANSIENT`	Một instance mới được tạo mỗi khi có một class khác inject nó vào.	Sử dụng cho các tiện ích loggers hoặc helper độc lập giữa các consumer.

2.5 Hiệu Ứng Lan Truyền (Bubble-up Effect) Của Scope.REQUEST

Trong dự án, tại `src/modules/order/factories/payment.factory.ts`, lớp `PaymentFactory` được đánh dấu rõ:

PaymentFactory với Scope.REQUEST

```
@Injectable({ scope: Scope.REQUEST })
export class PaymentFactory { ... }
```

Hiệu Ứng Lan Truyền (Bubble-up Effect) của Scope.REQUEST

Nếu Service A có `Scope.REQUEST`, bất kỳ Controller hoặc Service B nào inject Service A cũng sẽ tự động bị nâng lên thành `Scope.REQUEST`!
Điều này dẫn tới hiện tượng **Áp Lực Dọn Rác (Garbage Collection Churn)**: Với mỗi request gửi đến, Node.js V8 Engine phải liên tục cấp phát bộ nhớ heap cho hàng loạt đối tượng mới, sau đó thu hồi khi request kết thúc. Trong các bài kiểm tra tải cao (Load Testing), việc lạm dụng `Scope.REQUEST` có thể khiến thông lượng xử lý sụt giảm từ 20,000 req/sec xuống dưới 4,000 req/sec và làm tăng độ trễ p99! Kỹ sư chỉ nên dùng Scope.REQUEST khi thực sự cần lưu giữ ngữ cảnh phiên (như Transaction Context hoặc Multi-tenancy Isolation).

2.6 Xử Lý Phụ Thuộc Vòng (Circular Dependency) Với forwardRef()

Khi Service A inject Service B, và đồng thời Service B lại cần inject Service A, NestJS sẽ gặp tình trạng "con gà và quả trứng" khi dựng cây phụ thuộc. Để giải quyết, ta sử dụng hàm bao bọc tham chiếu trễ `forwardRef()`:

Giải quyết Circular Dependency với forwardRef

```
@Injectable()
export class UsersService {
  constructor(
    @Inject(forwardRef(() => AuthService))
    private readonly authService: AuthService,
  ) {}
}
```

Cơ chế này cũng áp dụng tương tự ở cấp Module thông qua `imports: [forwardRef(() => AuthModule)]`.

2.7 Bốn Hooks Vòng Đời Của Ứng Dụng & Cơ Chế Graceful Shutdown

NestJS cung cấp bộ 4 hooks cho phép can thiệp vào các giai đoạn khởi tạo và dọn dẹp hệ thống:

- `OnModuleInit`: Được gọi ngay khi các phụ thuộc của module đã được giải quyết xong. Thường dùng để khởi tạo kết nối mạng ngoại vi hoặc nạp cache khởi động.
- `OnApplicationBootstrap`: Được kích hoạt sau khi toàn bộ cây ứng dụng đã boot xong và chuẩn bị mở cổng mạng đón request.
- `OnModuleDestroy`: Được kích hoạt khi module nhận tín hiệu chuẩn bị hủy bỏ.
- `BeforeApplicationShutdown`: Cho phép thực hiện các thao tác đóng kết nối database an toàn (*Graceful Shutdown*), giải phóng socket và hoàn tất các tác vụ nền đang xử lý dở dang trước khi tiến trình Node.js kết thúc:

Hiện thực hóa Graceful Shutdown với BeforeApplicationShutdown

```
@Injectable()
export class DatabaseService implements BeforeApplicationShutdown {
  constructor(private readonly dataSource: DataSource) {}

  async beforeApplicationShutdown(signal?: string) {
    console.log(`[Graceful Shutdown] Nhận tín hiệu ${signal}. Đang đóng hồ kết nối database..
    if (this.dataSource.isInitialized) {
      await this.dataSource.destroy();
      console.log('[Graceful Shutdown] Hồ kết nối PostgreSQL đã đóng an toàn.');
    }
  }
}
```

Để kích hoạt các hooks tắt máy này, trong `main.ts` cần bổ sung lệnh `app.enableShutdownHooks()` để lắng nghe các tín hiệu hệ điều hành POSIX như `SIGTERM` hoặc `SIGINT`.

CHƯƠNG 3

Chu Trình Sống 12 Bước Của HTTP Request Trong Hệ Thống

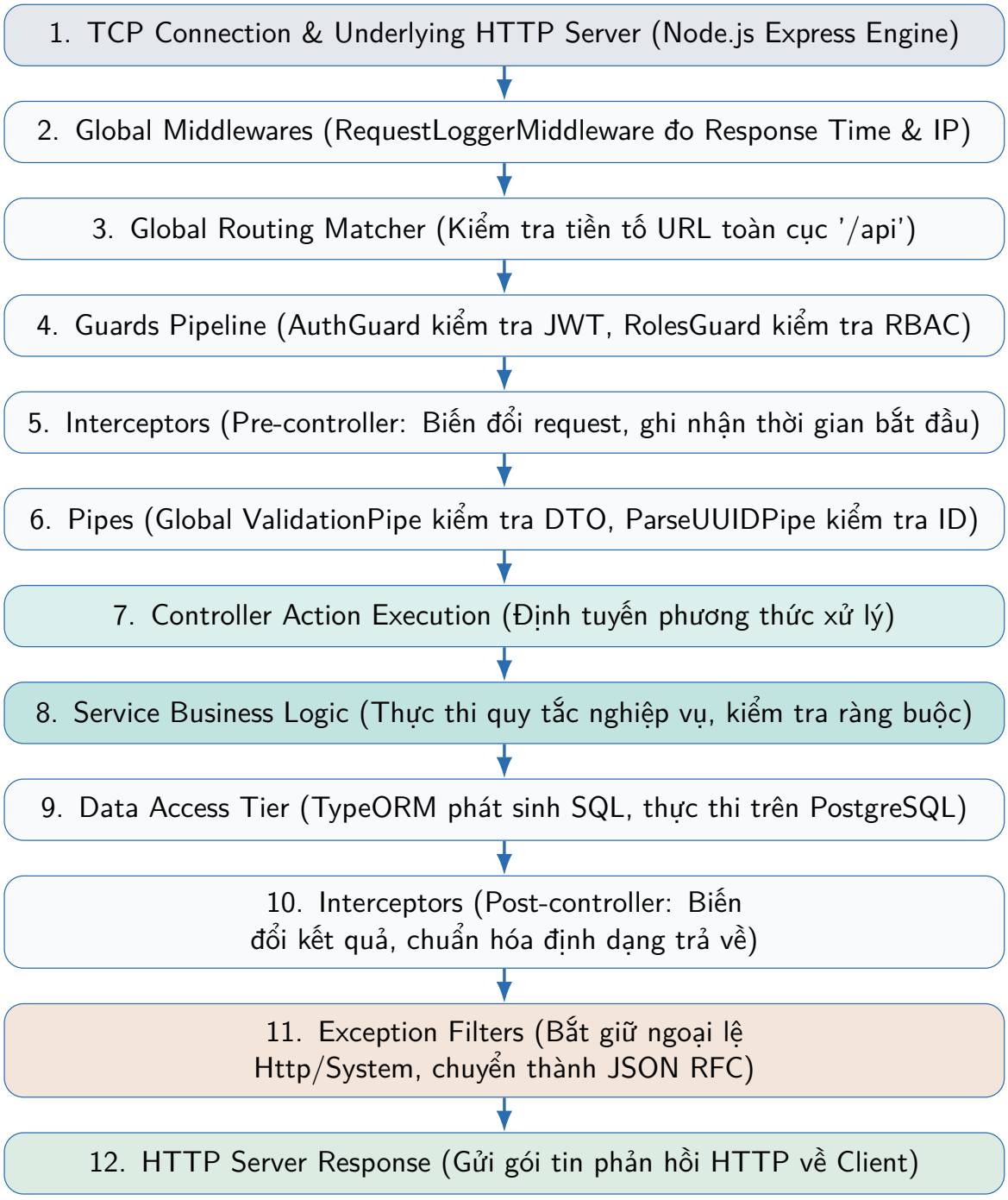
3.1 Đường Ống Xử Lý Yêu Cầu (The Request Lifecycle Pipeline)

Khi một yêu cầu HTTP được gửi từ phía máy khách đến máy chủ NestJS, nó không đi thẳng vào Controller mà phải đi qua một chuỗi các trạm kiểm soát nghiêm ngặt theo thứ tự tuần tự:

3.2 Phân Biệt Rạch Ròi Vai Trò Của Các Thành Phần Pipeline

Nhiều kỹ sư khi mới tiếp cận NestJS thường nhầm lẫn giữa Middleware, Guard, Interceptor và Pipe. Hãy phân định rạch ròi trách nhiệm của từng thành phần:

- **Middlewares**: Có thể truy cập đối tượng thô `req`, `res`, `next`. Hoàn toàn không biết Controller hay Action nào sẽ được gọi phía sau. Thích hợp cho ghi log toàn cục, nén dữ liệu gzip, thiết lập CORS.
- **Guards**: Trả lời câu hỏi duy nhất: **"Request này có được phép đi tiếp không?"**. Có quyền truy cập `ExecutionContext` và đọc metadata từ Decorators. Chuyên xử lý xác thực (Authentication) và phân quyền (Authorization).
- **Interceptors**: Kế thừa tư tưởng Lập trình hướng khía cạnh (*Aspect-Oriented Programming - AOP*). Bao bọc việc thực thi của hàm xử lý bằng RxJS `Observable`. Có thể can thiệp trước và sau khi Controller chạy, dùng để đo lường hiệu năng, ghi cache hoặc biến đổi cấu trúc dữ liệu trả về.
- **Pipes**: Trả lời câu hỏi: **"Dữ liệu truyền vào có đúng kiểu và hợp lệ không?"**. Chuyên trách thẩm định (*Validation*) và biến đổi kiểu (*Transformation*).
- **Exception Filters**: Trả lời câu hỏi: **"Khi có lỗi xảy ra, cấu trúc phản hồi lỗi gửi về Client sẽ như thế nào?"**. Bắt mọi ngoại lệ chưa được xử lý và định dạng



Hình 3.1: Toàn cảnh chu trình sống 12 bước của một HTTP Request trong NestJS

thành JSON chuẩn.

3.3 Cơ Chế Đa Giao Thức: `ExecutionContext` vs `ArgumentsHost`

NestJS được thiết kế để có thể chạy trên nhiều nền tảng truyền thông khác nhau: HTTP REST, WebSocket, Microservices (gRPC, RabbitMQ, Kafka). Để các Guards, Interceptors và Exception Filters có thể tái sử dụng xuyên suốt các giao thức này mà không phải viết lại, NestJS cung cấp hai lớp trừu tượng:

1. **`ArgumentsHost`**: Đóng vai trò là một wrapper bao bọc mảng tham số được truyền tới handler của nền tảng bên dưới:

Trích xuất ngữ cảnh từ ArgumentsHost trong Exception Filter

```
@Catch(NotFoundException)
export class NotFoundExceptionFilter implements ExceptionFilter {
  catch(exception: NotFoundException, host: ArgumentsHost) {
    const ctx = host.switchToHttp(); // Chuyển đổi sang ngữ cảnh HTTP
    const res = ctx.getResponse<Response>();
    const req = ctx.getRequest<Request>();
    res.status(404).json({ statusCode: 404, path: req.url });
  }
}
```

   Nếu ứng dụng chuyển sang chạy gRPC, kỹ sư chỉ cần gọi `host.switchToRpc().getData()` mà không làm thay đổi chữ ký của Exception Filter!

2. **`ExecutionContext`**: Kế thừa trực tiếp từ `ArgumentsHost`, bổ sung thêm hai phương thức phản chiếu quan trọng:

   - `context.getClass()`: Trả về tham chiếu tới lớp Controller đang thực thi.
   - `context.getHandler()`: Trả về tham chiếu tới phương thức xử lý cụ thể.

   Đây chính là chìa khóa để `RolesGuard` sử dụng `Reflector` nhằm trích xuất metadata do decorator `@Roles()` gắn vào, như chúng ta sẽ tìm hiểu chi tiết ở Chương 12.

CHƯƠNG 4

Thẩm Định Dữ Liệu Với ValidationPipe & DTO Pattern

4.1 Bảo Vệ Hệ Thống Trước Lỗ Hổng Mass Assignment

Lỗ hổng bảo mật **Mass Assignment** (hay *Over-posting Attack*) xảy ra khi ứng dụng tự động gán toàn bộ các thuộc tính người dùng gửi lên vào đối tượng lưu trữ cơ sở dữ liệu mà không qua tầng sàng lọc.

Hãy tưởng tượng một người dùng độc hại gửi yêu cầu tạo bài viết hoặc đăng ký:

Payload tấn công gửi kèm trường đặc quyền nguy hiểm

```
{
  "title": "Hướng dẫn NestJS",
  "category": "Technology",
  "role": "ADMIN",            // TRƯỜNG TẤN CÔNG ĐẶC QUYỀN!
  "is_completed": true,
  "isSuperUser": true         // TRƯỜNG TẤN CÔNG ĐẶC QUYỀN!
}
```

Nếu Service gán thẳng `repository.save(req.body)`, đối tượng có thể bị ghi đè các cột đặc quyền hệ thống, biến người dùng thường thành Quản trị viên cấp cao!

NestJS dập tắt triệt để nguy cơ này nhờ mô hình kết hợp giữa **Data Transfer Object (DTO)** và **ValidationPipe**.

4.2 Bộ Ba Cờ Cấu Hình Toàn Cục Trong `src/main.ts`

Tại tệp khởi động `src/main.ts`, hệ thống kích hoạt ValidationPipe với bộ ba cấu hình chuẩn mực:

Cấu hình ValidationPipe toàn cục chuẩn mực trong src/main.ts

```
app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,
    stopAtFirstError: true,
```

```
    transform: true,
  }),
);
```

- **whitelist: true**: Quét qua toàn bộ các thuộc tính của đối tượng gửi lên. Bất kỳ thuộc tính nào không có decorator thẩm định khai báo trong class DTO sẽ bị **loại bỏ hoàn toàn** trước khi đến tay Controller. Các trường độc hại như `role` hay `isSuperUser` ở trên sẽ bị gọt sạch ngay tại cửa ngõ!
- **transform: true**: Chuyển đổi payload từ dạng plain JavaScript object thành thể hiện thực sự (*class instance*) của lớp DTO, cho phép các hàm phương thức, getter, và các decorator biến đổi của `class-transformer` hoạt động chuẩn xác.
- **stopAtFirstError: true**: Ngay khi phát hiện trường đầu tiên vi phạm ràng buộc, đường ống dừng thẩm định ngay lập tức và ném lỗi 400 Bad Request về Client, giúp tiết kiệm tối đa chu kỳ CPU của máy chủ dưới áp lực tải cao.

4.3 Kế Thừa DTO Linh Hoạt Với `PartialType()`

Để tuân thủ nguyên lý DRY (*Don't Repeat Yourself*), NestJS cung cấp hàm tiện ích `PartialType()` từ thư viện `@nestjs/swagger` (hoặc `@nestjs/mapped-types`). Hãy khảo sát cách thức áp dụng thực tế trong hai module `post` và `tasks`:

Kế thừa DTO trong src/modules/post/dto/ và tasks/dto/

```
// 1. DTO tạo mới: Bắt buộc các trường
export class CreatePostDto {
  @IsString()
  @IsNotEmpty()
  title: string;

  @IsString()
  @IsNotEmpty()
  category: string;

  @IsString()
  @IsOptional()
  description?: string;
}

// 2. DTO cập nhật PATCH: Mọi trường tự động trở thành Optional
export class UpdatePostDto extends PartialType(CreatePostDto) {}
```

Cơ Chế Phía Sau Của PartialType

Khi `UpdatePostDto` kế thừa `PartialType(CreatePostDto)`, thư viện sẽ tự động sao chép toàn bộ các quy tắc thẩm định (`@IsString()`, `@MaxLength()`...) nhưng gắn thêm decorator `@IsOptional()` cho từng trường, đồng thời cập nhật đúng metadata hiển thị

cho Swagger UI. Kỹ sư không cần phải sao chép lặp lại mã nguồn của `CreatePostDto`!

4.4 Phân Tách DTO Quản Trị Viên (Admin) & Người Dùng Thường (Client)

Trong các hệ thống phân quyền phức tạp, việc sử dụng chung một DTO cho cả thao tác của Quản trị viên và Khách hàng là một lỗ hổng thiết kế nguy hiểm:

- **DTO Phía Client**: Thường bị hạn chế các trường nhạy cảm. Người dùng không được phép tự ý gán trạng thái phê duyệt (`status: 'APPROVED'`), gán vai trò (`role: 'ADMIN'`), hoặc sửa đổi điểm tích lũy.
- **DTO Phía Admin (`src/modules/admin/dto/create-post.dto.ts`)**: Cho phép Quản trị viên chỉ định danh mục đặc biệt, phê duyệt trực tiếp hoặc gắn cờ ưu tiên.

Bằng cách chia tách thành các lớp DTO biệt lập kết hợp với cờ `whitelist: true`, hệ thống bảo đảm chắc chắn rằng người dùng thông thường không bao giờ có thể tự nâng quyền hoặc sửa đổi các thuộc tính đặc quyền của hệ thống.

4.5 Chuyển Đổi Kiểu Tham Số URL Với `class-transformer`

Trong giao thức HTTP, toàn bộ các tham số trên URL (query strings và route params) đều được máy chủ web nhận vào dưới dạng chuỗi thô (*raw strings*). Ví dụ: `?page=2&page_size=10&isComple`

Nếu không có cơ chế chuyển đổi kiểu:

$$\text{page} + 1 = \text{'2'} + 1 = \text{'21'} \quad \text{(Nối chuỗi thay vì phép cộng số học!)} \tag{4.1}$$

Trong `PageOptionsDto` (`src/core/pagination/dto/page-option.dto.ts`), dự án sử dụng decorator `@Type(() => Number)` để ép kiểu an toàn:

Ép kiểu số học trong PageOptionsDto

```
export class PageOptionsDto {
  @Type(() => Number)
  @IsInt()
  @Min(1)
  @IsOptional()
  readonly page?: number = 1;

  @Type(() => Number)
  @IsInt()
  @Min(1)
  @Max(50)
  @IsOptional()
  readonly page_size?: number = 10;
}
```

4.6 Tích Hợp Giữa `ValidationPipe` & `ErrorService`

Khi dữ liệu gửi lên không vượt qua được bài kiểm tra, `ValidationPipe` phát sinh một mảng các đối tượng `ValidationError` chứa thông tin chi tiết:

Cấu trúc đối tượng ValidationError từ class-validator

```
{
  "property": "title",
  "value": "",
  "constraints": {
    "isNotEmpty": "title should not be empty",
    "isString": "title must be a string"
  }
}
```

Đối tượng này được ném ra dưới dạng `UnprocessableEntityException` hoặc `BadRequestException`. Bộ lọc ngoại lệ `UnprocessableEntityExceptionFilter` tại `src/core/filters/` sẽ bắt lấy lỗi này và chuyển tiếp tới `ErrorService` để tra cứu từ điển `src/i18n/vi/errors.json`, tự động dịch câu thông báo sang tiếng Việt và gắn tên trường bị vi phạm vào JSON phản hồi trả về cho máy khách.

PHẦN II

Tầng Dữ Liệu & ORM Doanh Nghiệp Với TypeORM (Database Engineering)

CHƯƠNG 5

Kiến Trúc Truy Cập Dữ Liệu Với TypeORM 0.3

5.1 Mẫu Thiết Kế Data Mapper vs Active Record

Trong phát triển ứng dụng cơ sở dữ liệu hướng đối tượng, kiến trúc sư phần mềm Martin Fowler đã phân định hai mẫu thiết kế truy cập dữ liệu kinh điển:

- **Active Record**: Thực thể (*Entity*) kế thừa từ một lớp cơ sở (như `BaseEntity`) và tự chứa toàn bộ các phương thức truy vấn cơ sở dữ liệu (ví dụ: `task.save()`, `Task.find()`). Mặc dù cú pháp này có vẻ ngắn gọn, Active Record vi phạm nghiêm trọng **Nguyên Lý Trách Nhiệm Đơn Lẻ (Single Responsibility Principle)**: thực thể dữ liệu vừa phải chịu trách nhiệm mô hình hóa nghiệp vụ, vừa phải gánh vác trách nhiệm kết nối mạng và thực thi SQL. Điều này khiến việc viết Unit Test cho tầng Service trở nên bất khả thi nếu không có một cơ sở dữ liệu thật chạy ngầm!
- **Data Mapper (Dự án áp dụng)**: Thực thể chỉ đơn thuần là một cấu trúc dữ liệu thuần túy (*Plain Data Holder*) chứa thuộc tính và metadata ánh xạ cột. Toàn bộ logic tương tác CSDL (tìm kiếm, chèn, xóa, cập nhật, phân trang) được chuyển giao hoàn toàn cho đối tượng **Repository** độc lập. Cách tiếp cận này tách biệt rạch ròi miền nghiệp vụ (*Domain Model*) khỏi hạ tầng lưu trữ (*Persistence Infrastructure*), tạo điều kiện lý tưởng để giả lập (*mocking*) tầng dữ liệu khi viết kiểm thử tự động.

5.2 Cấu Hình DataSource Tập Trung Trong TypeORM 0.3

Trong phiên bản TypeORM 0.3, việc kết nối cơ sở dữ liệu đã chuyển đổi toàn diện từ phương thức toàn cục `createConnection` sang đối tượng quản lý tập trung **DataSource**. Hãy giải phẫu tệp cấu hình thực tế tại `src/database/typeorm/config.ts`:

Mã nguồn cấu hình DataSource tập trung: src/database/typeorm/config.ts

```
import fs from 'fs';
import { DataSource } from 'typeorm';
import { ConfigService } from '@nestjs/config';
import { config } from 'dotenv';
import entities from './entities';
```

```
config();
const configService = new ConfigService();

export const dataSource = new DataSource({
  type: 'postgres',
  host: configService.get('TYPEORM_HOST'),
  port: configService.get<number>('TYPEORM_PORT'),
  username: configService.get('TYPEORM_USERNAME'),
  password: configService.get('TYPEORM_PASSWORD'),
  database: configService.get('TYPEORM_DATABASE'),
  entities,
  ssl:
    configService.get('DB_USE_SSL') === 'true'
      ? {
          ca: fs.readFileSync('ca-certificate.crt'),
        }
      : undefined,
  migrations: ['src/database/typeorm/migrations/*.ts'],
});
```

An Toàn Kết Nối Cơ Sở Dữ Liệu Với SSL CA

Trong môi trường doanh nghiệp điện toán đám mây (AWS RDS, Google Cloud SQL, Neon PostgreSQL), việc truyền dữ liệu qua kênh mạng công cộng bắt buộc phải mã hóa bằng Transport Layer Security (TLS/SSL). Dự án hỗ trợ cờ `DB_USE_SSL`: khi kích hoạt, hệ thống tự động nạp chứng chỉ gốc `ca-certificate.crt` để xác minh danh tính máy chủ cơ sở dữ liệu, chống lại hoàn toàn các cuộc tấn công nghe lén hoặc giả mạo (*Man-in-the-Middle*).

5.3 Quản Lý Hồ Kết Nối (Connection Pooling) & Tải Cao

Trong môi trường sản phẩm thực tế, việc mở một kết nối TCP/IP mới tới PostgreSQL cho mỗi truy vấn là một hành động vô cùng tốn kém tài nguyên: máy chủ CSDL phải khởi tạo tiến trình backend mới, thực hiện bắt tay SSL, xác thực quyền và cấp phát bộ nhớ đệm (*work_mem*).

TypeORM sử dụng thư viện `pg-pool` quản lý một hồ kết nối duy trì sẵn (*Connection Pool*):

- **max**: Giới hạn số lượng kết nối tối đa mở đồng thời tới cơ sở dữ liệu (mặc định 10 đến 20 kết nối). Việc giới hạn này bảo vệ PostgreSQL không bị quá tải CPU và cạn kiệt bộ nhớ khi có hàng chục nghìn request ập đến cùng lúc.
- **idleTimeoutMillis**: Thời gian tối đa một kết nối rảnh rỗi được giữ trong pool trước khi bị đóng (thường là 30,000ms), giải phóng tài nguyên cho hệ điều hành.
- **connectionTimeoutMillis**: Thời gian tối đa chờ đợi để mượn một kết nối từ pool trước khi ném ngoại lệ hết thời gian (*Timeout Exception*).

5.4 Nguy Cơ Tiềm Ẩn Của Cờ synchronize: true

Nhiều lập trình viên khi mới học NestJS thường bật cờ `synchronize: true` trong cấu hình TypeORM để bảng CSDL tự động cập nhật khi sửa code Entity.

Quy Tắc Sống Còn: Tuyệt Đối Cấm synchronize: true Trên Môi Trường Production

Cờ `synchronize: true` thực hiện so sánh mã nguồn Entity với schema CSDL mỗi khi ứng dụng khởi động. Nếu bạn đổi tên một cột (ví dụ từ `title` sang `heading`) hoặc xóa một thuộc tính trong Entity, TypeORM sẽ ngay lập tức phát sinh lệnh `DROP COLUMN` hoặc `DROP TABLE`!

Hành động này dẫn tới việc **MẤT DỮ LIỆU VĨNH VIỄN** của hàng triệu khách hàng mà không có cách nào cứu vãn. Trong môi trường Staging và Production của doanh nghiệp, `synchronize: false` là bắt buộc 100%, và mọi thay đổi cấu trúc dữ liệu phải được thực hiện thông qua **TypeORM Migrations** có kiểm duyệt mã nguồn (*Code Review*).

5.5 Quản Lý Giao Dịch Dữ Liệu (Transactions) Với QueryRunner

Trong các nghiệp vụ phức tạp đòi hỏi tính toàn vẹn tuyệt đối (như tạo đơn hàng, trừ tiền ví, cập nhật kho hàng), kỹ sư không thể thực hiện các câu lệnh `save()` rời rạc vì nếu bước sau thất bại, bước trước sẽ để lại dữ liệu rác.

TypeORM cung cấp đối tượng `QueryRunner` cho phép kiểm soát giao dịch theo chuẩn ACID:

Mẫu giao dịch dữ liệu chuẩn mực với QueryRunner trong OrderService

```
const queryRunner = this.datasource.createQueryRunner();
await queryRunner.connect();
await queryRunner.startTransaction();

try {
  // 1. Tạo đơn hàng trong ngữ cảnh Transaction
  const createdOrder = queryRunner.manager.create(Order, { ...orderData });
  const savedOrder = await queryRunner.manager.save(Order, createdOrder);

  // 2. Tạo chi tiết đơn hàng
  const details = queryRunner.manager.create(OrderDetails, { orderId: savedOrder.id });
  await queryRunner.manager.save(OrderDetails, details);

  // 3. Cam kết giao dịch vào CSDL
  await queryRunner.commitTransaction();
  return savedOrder;
} catch (error) {
  // 4. Nếu có bất kỳ lỗi nào, hoàn tác toàn bộ trạng thái ban đầu
  await queryRunner.rollbackTransaction();
  throw error;
} finally {
```

```
    // 5. BẮT BUỘC: Giải phóng kết nối trả lại cho Hồ kết nối (Connection Pool)
    await queryRunner.release();
  }
```

Nếu thiếu khối lệnh `finally { await queryRunner.release(); }`, kết nối cơ sở dữ liệu sẽ bị "treo" vĩnh viễn. Chỉ sau vài trăm request, hồ kết nối sẽ cạn kiệt (*Connection Pool Exhaustion*), khiến toàn bộ máy chủ backend bị tê liệt hoàn toàn!

CHƯƠNG 6

Mô Hình Hóa Thực Thể & Decorators Tùy Biến

6.1 Lớp Cơ Sở Trừu Tượng `AbstractEntity` & Mẫu `toDto()`

Để tuân thủ nguyên lý DRY (*Don't Repeat Yourself*), dự án xây dựng lớp cơ sở `AbstractEntity` tại `src/shared/abstract.entity.ts`:

Mã nguồn AbstractEntity trong src/shared/abstract.entity.ts

```
import { AbstractDto } from './dto/abstract.dto';
import { Constructor } from './types/utility-types';

export interface IAbstractEntity<DTO extends AbstractDto, O = never> {
  toDto(options?: O): DTO;
}

export abstract class AbstractEntity<
  DTO extends AbstractDto = AbstractDto,
  O = never,
> implements IAbstractEntity<DTO, O> {
  private dtoClass?: Constructor<DTO, [AbstractEntity, O?]>;

  toDto(options?: O): DTO {
    const dtoClass = this.dtoClass;
    if (!dtoClass) {
      throw new Error(
        `You need to use @UseDto on class (${this.constructor.name}) be able to call toDto fu
      );
    }
    return new dtoClass(this, options);
  }
}
```

Mô hình này mang lại sự bảo đảm an toàn dữ liệu: phương thức `toDto()` bắt buộc thực thể phải được ánh xạ qua lớp DTO phản hồi, ngăn chặn việc vô tình để lộ các trường nhạy cảm như `password`, `secretKey` hay các cờ logic nội bộ ra ngoài mạng Internet.

6.2 Khóa Chính Toàn Cầu UUID v4 vs BigInt Auto-Increment

Bảng 6.1: So sánh chuyên sâu giữa UUID v4 và BigInt Auto-Increment

Tiêu chí	UUID v4 (Lựa chọn của dự án)	BigInt Auto-Increment
Tính bảo mật	**Tuyệt đối**: Không thể đoán trước ID bản ghi tiếp theo, triệt tiêu nguy cơ tấn công vét cạn ID (Insecure Direct Object Reference).	Yếu: ID tăng tuần tự (1, 2, 3...) giúp kẻ tấn công dễ dàng cào dữ liệu toàn bộ hệ thống.
Khả năng phân tán	Sinh độc lập tại client hoặc server mà không bao giờ lo trùng lặp, tối ưu cho Sharding.	Phụ thuộc vào bộ đếm trung tâm của CSDL duy nhất.
Kích thước lưu trữ	16 bytes (128 bits), tốn dung lượng RAM chỉ mục hơn một chút.	8 bytes (64 bits), nhỏ gọn.

6.3 Chiến Lược Đặt Tên Tự Động (SnakeNamingStrategy)

JavaScript tuân theo quy ước đặt tên **camelCase** (ví dụ: `isCompleted`, `createdAt`), trong khi cơ sở dữ liệu quan hệ PostgreSQL chuẩn mực yêu cầu quy ước **snake_case** (ví dụ: `is_completed`, `created_at`).

Nhờ cấu hình `SnakeNamingStrategy`, TypeORM tự động chuyển đổi qua lại giữa camelCase và snake_case:

$$\text{TaskEntity.isCompleted} \xrightarrow{\text{SnakeNamingStrategy}} \text{tasks.is_completed} \tag{6.1}$$

Lập trình viên không cần phải viết thủ công `@Column({ name: 'is_completed' })` lặp đi lặp lại trên từng thuộc tính, giúp mã nguồn luôn gọn gàng và đồng nhất.

6.4 Giải Phẫu Bộ Decorators Tùy Biến Chuyên Sâu

Dự án khai thác triệt để sức mạnh của Metadata Reflection với các decorator tự phát triển tại `src/shared/decorator/`:

1. **`@VirtualColumn()`** (`src/shared/decorator/virtual-column.decorator.ts`): Cho phép định nghĩa thuộc tính ảo trong Entity được tính toán động từ câu truy vấn SQL (như `COUNT(*)` hay `SUM(amount)`) mà không tạo cột vật lý trong bảng CSDL.

Mã nguồn @VirtualColumn

```
export const VIRTUAL_COLUMN_KEY = Symbol('VIRTUAL_COLUMN_KEY');

export function VirtualColumn(name?: string): PropertyDecorator {
```

```
  return (target, propertyKey) => {
    const metaInfo = Reflect.getMetadata(VIRTUAL_COLUMN_KEY, target) || {};
    metaInfo[propertyKey] = name ?? propertyKey;
    Reflect.defineMetadata(VIRTUAL_COLUMN_KEY, metaInfo, target);
  };
}
```

2. **`@UseDto()`** (`src/shared/decorator/use-dto.decorator.ts`): Gắn lớp DTO tương ứng vào nguyên mẫu (*prototype*) của Entity để phục vụ phương thức `toDto()`:

Mã nguồn @UseDto

```
export function UseDto(
  dtoClass: Constructor<AbstractDto, [AbstractEntity, unknown]>,
): ClassDecorator {
  return (ctor) => {
    ctor.prototype.dtoClass = dtoClass;
  };
}
```

3. **`@CurrentUser()`** (`src/shared/decorator/user.decorator.ts`): Trích xuất đối tượng người dùng đã được xác thực từ `req.user` một cách tiện lợi ngay tại tham số của Controller:

Mã nguồn @CurrentUser

```
export const CurrentUser = createParamDecorator(
  (data: string, ctx: ExecutionContext) => {
    const request = ctx.switchToHttp().getRequest();
    const user = request.user;
    return data ? user?.[data] : user;
  },
);
```

CHƯƠNG 7

Chiến Lược Di Trú Không Gián Đoạn (Zero-Downtime Migrations)

7.1 Bản Chất Của TypeORM Migrations & Triết Lý Bất Biến

Migration là tập hợp các tập lệnh TypeScript mô tả sự thay đổi của cấu trúc cơ sở dữ liệu theo thời gian tuân theo nguyên tắc bất biến (Immutability). Thay vì chỉnh sửa trực tiếp cấu trúc bảng trên môi trường sản xuất bằng các công cụ quản trị đồ họa (như pgAdmin, DBeaver) — một hành vi bị nghiêm cấm tuyệt đối trong kỹ thuật phần mềm doanh nghiệp vì không thể truy vết lịch sử, không thể tự động hóa và có nguy cơ lỗi con người rất cao — toàn bộ các thay đổi phải được quản lý dưới dạng mã nguồn (*Database as Code*) và được theo dõi trong hệ thống quản trị phiên bản Git.

Mỗi tệp tin di trú được TypeORM sinh ra đều gồm hai phương thức đối xứng nghiêm ngặt:

- `up(queryRunner: QueryRunner): Promise<void>`: Chứa các câu lệnh SQL thực thi sự thay đổi cấu trúc mới (tạo bảng, thêm cột, sửa kiểu dữ liệu, thiết lập ràng buộc khóa ngoại, đánh chỉ mục index).
- `down(queryRunner: QueryRunner): Promise<void>`: Chứa các câu lệnh hoàn tác chính xác và đối xứng với những gì phương thức `up` đã thực hiện, bảo đảm hệ thống có thể quay lui trạng thái (*Rollback*) an toàn khi việc triển khai gặp sự cố bất ngờ.

7.2 Mô Hình Di Trú Mở Rộng & Thu Hẹp (The Expand and Contract Pattern)

Trong môi trường sản xuất với cơ sở dữ liệu quy mô hàng chục triệu bản ghi, việc cập nhật cấu trúc bảng không được phép làm gián đoạn dịch vụ (*Zero Downtime*). Một sai lầm phổ biến của các lập trình viên mới vào nghề là thực hiện việc đổi tên cột hoặc xóa cột trực tiếp trong một bản cập nhật duy nhất. Khi mã nguồn mới chưa được tải hết lên toàn bộ các máy chủ backend (pod/container), mã nguồn cũ vẫn đang chạy và sẽ gặp lỗi ngay lập tức vì không tìm thấy cột cũ trong CSDL.

Dự án áp dụng mô hình kiến trúc kinh điển **The Expand and Contract Pattern** (hay còn gọi là *Parallel Run* hoặc *Phased Migration*) gồm 4 pha biệt lập:



Hình 7.1: Mô hình Expand and Contract trong di trú cơ sở dữ liệu không gián đoạn

7.3 Phân Cấp Khóa Bảng (Table Lock Hierarchy) Trong PostgreSQL

Để thực hiện Zero-Downtime Migration thành công, Kỹ sư Backend bắt buộc phải hiểu cơ chế khóa tài nguyên của PostgreSQL. Khi một lệnh `ALTER TABLE` được thực thi, PostgreSQL yêu cầu một trong các cấp độ khóa sau:

Cạm Bẫy AccessExclusiveLock Gây Sập Toàn Hệ Thống

Khi một bảng chứa 20 triệu dòng, nếu bạn chạy lệnh sau trực tiếp trên môi trường sản xuất:

```
ALTER TABLE users ADD COLUMN age integer NOT NULL DEFAULT 18;
```
(7.1)

Trước phiên bản PostgreSQL 11, CSDL sẽ yêu cầu ghi lại toàn bộ bảng vật lý ra đĩa để cập nhật từng dòng. Thao tác này chiếm giữ `AccessExclusiveLock` trong nhiều phút, khiến hàng chục ngàn kết nối từ phía NestJS bị tắc nghẽn, dẫn tới cạn kiệt Connection Pool và làm sập toàn bộ hệ thống!
Mặc dù từ PostgreSQL 11 trở lên giá trị mặc định bất biến (*constant default*) không còn yêu cầu viết lại bảng, nhưng nếu giá trị mặc định là một hàm biến đổi thời gian thực như `DEFAULT uuid_generate_v4()` hoặc `DEFAULT clock_timestamp()`, bảng vật lý **vẫn bị viết lại hoàn toàn**!

Bảng 7.1: Các cấp độ khóa bảng phổ biến trong PostgreSQL và xung đột truy vấn

Cấp độ khóa (Lock Mode)	Câu lệnh kích hoạt	Tác động lên người dùng
`AccessShareLock`	`SELECT`	Không khóa bất kỳ ai, cho phép đọc/ghi song song.
`RowExclusiveLock`	`INSERT`, `UPDATE`, `DELETE`	Chỉ khóa trên từng dòng bị sửa đổi, cho phép truy vấn đọc khác hoạt động bình thường.
`ShareLock`	`CREATE INDEX` (tiêu chuẩn)	Cho phép các truy vấn `SELECT` đọc dữ liệu, nhưng **chặn toàn bộ** các lệnh `INSERT`, `UPDATE`, `DELETE`.
`AccessExclusiveLock`	`ALTER TABLE ADD COLUMN`, `DROP TABLE`, `VACUUM FULL`	**Khóa tuyệt đối**: Chặn cả đọc lẫn ghi! Toàn bộ request vào bảng đó sẽ bị treo cho tới khi migration hoàn tất.

7.4 Đánh Chỉ Mục Phi Khóa Với `CREATE INDEX CONCURRENTLY`

Đánh chỉ mục (Indexing) là thao tác thường xuyên để tối ưu tốc độ tìm kiếm. Tuy nhiên, lệnh `CREATE INDEX` thông thường sẽ chiếm giữ `ShareLock`, khiến mọi thao tác thêm, sửa, xóa trên bảng bị từ chối trong suốt thời gian xây dựng cây B-Tree.

Để giải quyết vấn đề này, PostgreSQL cung cấp cú pháp `CONCURRENTLY`:

```
CREATE INDEX CONCURRENTLY idx_tasks_category ON tasks (category);
```
(7.2)

Cơ chế này quét bảng qua 2 chu kỳ độc lập, cho phép người dùng tiếp tục đọc và ghi bình thường vào bảng mà không gặp bất kỳ độ trễ nào.

Cấu Hình TypeORM Migration Hỗ Trợ Concurrent Indexing

Theo chuẩn PostgreSQL, lệnh `CREATE INDEX CONCURRENTLY` **không được phép chạy bên trong một khối giao dịch** (*cannot run inside a transaction block*). Mặc định, TypeORM sẽ bọc toàn bộ mã của hàm `up()` vào một giao dịch ACID duy nhất. Để sử dụng tính năng này, kỹ sư cần cấu hình thuộc tính `transaction = false` trong lớp migration:

```
export class AddCategoryIndexConcurrently1680000000000 implements MigrationInterface {
  // Vô hiệu hóa bọc Transaction toàn cục của TypeORM
  transaction = false;

  public async up(queryRunner: QueryRunner): Promise<void> {
    await queryRunner.query(
      `CREATE INDEX CONCURRENTLY "idx_tasks_category" ON "tasks" ("category")`
    );
  }

  public async down(queryRunner: QueryRunner): Promise<void> {
    await queryRunner.query(`DROP INDEX CONCURRENTLY "idx_tasks_category"`);
  }
```

```
}
```

7.5 Kỹ Thuật Di Chuyển Dữ Liệu Theo Lô (Batch Back-filling)

Khi cần di trú hàng triệu bản ghi từ cột cũ sang cột mới ở Pha 3, việc thực hiện một câu lệnh `UPDATE` duy nhất trên toàn bộ bảng sẽ làm phình to tập tin nhật ký giao dịch Write-Ahead Log (WAL) và gây ra khóa dòng trên diện rộng.

Dự án quy định kỹ thuật **Cursor Batching**: chia nhỏ khối lượng dữ liệu thành các lô độc lập (ví dụ mỗi đợt 2,000 bản ghi) dựa trên con trỏ khóa chính:

```
let lastId = 0;
const BATCH_SIZE = 2000;

while (true) {
  const result = await queryRunner.query(
    `UPDATE users
     SET full_name = first_name || ' ' || last_name
     WHERE id > $1
     ORDER BY id ASC
     LIMIT $2
     RETURNING id`,
    [lastId, BATCH_SIZE]
  );
  if (result.length === 0) break;
  lastId = result[result.length - 1].id;
  // Tạm nghỉ 50ms giữa các đợt để trả tài nguyên IOPS cho hệ thống chính
  await new Promise(resolve => setTimeout(resolve, 50));
}
```

7.6 Vận Hành Tự Động Hóa Migrations Qua NPM Scripts

Trong `package.json`, dự án tích hợp sẵn các lệnh CLI tiêu chuẩn kết nối trực tiếp với cấu hình trung tâm `src/database/typeorm/config.ts`:

- `npm run typeorm:generate-migration --name=CreateTasksTable`: Tự động quét và so sánh các Metadata Entity hiện tại với Schema thực tế trong PostgreSQL, sau đó tự sinh mã TypeScript chứa đầy đủ các câu lệnh `CREATE TABLE`, `ALTER TABLE` tương ứng.
- `npm run typeorm:run-migration`: Quét bảng hệ thống `migrations` trong CSDL, xác định các tệp tin di trú chưa được thực thi và chạy tuần tự các hàm `up()` theo đúng dấu thời gian (*timestamp*).
- `npm run typeorm:revert-migration`: Hoàn tác lại migration gần nhất bằng cách chạy hàm `down()`.

- `npm run typeorm:create-migration --name=ManualSchemaFix`: Tạo một tệp migration trống để kỹ sư tự viết các truy vấn SQL tùy biến theo nghiệp vụ đặc thù.

CHƯƠNG 8

Khởi Tạo Dữ Liệu Mẫu Tự Động Với Hygen Seeding

8.1 Kiến Trúc Động Cơ Seeding Tại src/database/typeorm/seeds/

Database Seeding là quá trình tự động nạp dữ liệu khởi tạo ban đầu vào cơ sở dữ liệu. Trong phát triển phần mềm hiện đại, Seeding phục vụ hai mục đích sống còn:

1. **Dữ liệu hệ thống bắt buộc (System Seeds)**: Các bản ghi danh mục cố định mà thiếu chúng ứng dụng không thể hoạt động (ví dụ: các vai trò người dùng `ADMIN`, `MEMBER`; các phương thức thanh toán `MOMO`, `COD`).
2. **Dữ liệu thử nghiệm (Development Mock Seeds)**: Hàng trăm người dùng ảo, bài viết mẫu, đơn hàng giả lập giúp đội ngũ lập trình viên và kiểm thử (QA) có dữ liệu thực tế để kiểm tra tính năng và đo đạc hiệu năng mà không cần nhập liệu thủ công.

8.2 Giải Phẫu Bộ Điều Phối run-seed.ts

Tệp tin `src/database/typeorm/seeds/run-seed.ts` là trái tim của hệ thống nạp dữ liệu mẫu:

Mã nguồn điều phối nạp dữ liệu mẫu: run-seed.ts

```
import { NestFactory } from '@nestjs/core';
import { DemoSeedService } from './demo/demo-seed.service';
import { RoleSeedService } from './role/role-seed.service';
import { UserSeedService } from './user/user-seed.service';
import { SeedModule } from './seed.module';
import { PaymentMethodSeedService } from './payment-method/payment-method-seed.service';
import { ProductSeedService } from './product/products-seed.service';

const runSeed = async () => {
  // 1. Khởi tạo một NestJS Application Context nhỏ gọn
  const app = await NestFactory.create(SeedModule);

  // 2. Thực thi tuần tự theo đúng ràng buộc khóa ngoại (Foreign Keys)
  await app.get(RoleSeedService).run();          // Cấp 1: Bảng danh mục vai trò gốc
```

```
  await app.get(UserSeedService).run();          // Cấp 2: Bảng tài khoản (phụ thuộc Role)
  await app.get(PaymentMethodSeedService).run(); // Cấp 3: Bảng phương thức thanh toán
  await app.get(ProductSeedService).run();       // Cấp 4: Danh mục sản phẩm
  await app.get(DemoSeedService).run();          // Cấp 5: Bảng demo (phụ thuộc User & Produc

  // 3. Đóng ứng dụng và giải phóng toàn bộ kết nối CSDL
  await app.close();
};

void runSeed();
```

Thiết Kế Tính Bất Biến (Idempotency) Trong Seeding

Một bộ seed đạt chuẩn doanh nghiệp phải bảo đảm **Tính Bất Biến (Idempotent)**: Việc chạy script seed 1 lần hay 100 lần liên tiếp đều phải cho ra cùng một kết quả duy nhất mà không gây ra lỗi vi phạm khóa duy nhất (*Unique Constraint Violation*). Trong mỗi Service (ví dụ: `RoleSeedService`), trước khi chèn một bản ghi mới, dịch vụ luôn thực hiện kiểm tra:

```
const count = await this.roleRepository.count({ where: { id: role.id } });
if (count === 0) {
  await this.roleRepository.save(this.roleRepository.create(role));
}
```

Nếu vai trò đã tồn tại, script sẽ nhẹ nhàng bỏ qua thay vì làm gián đoạn toàn bộ quy trình.

8.3 Tích Hợp Hygen Code Generator Tạo Mẫu Tự Động

Dự án tích hợp công cụ tạo mẫu mã nguồn **Hygen** (`hygen seeds create`). Thay vì phải tạo thủ công 2 file Module và Service mỗi khi muốn thêm một bảng seed mới, kỹ sư chỉ cần gõ lệnh:

```
npm run seed:create -- --name=Category
```
(8.1)

Hygen sẽ tự động sinh ra cấu trúc thư mục `src/database/typeorm/seeds/category/`, tự động khai báo Entity, Repository và cấu hình method `run()` chuẩn mực, giúp tiết kiệm thời gian phát triển và loại bỏ hoàn toàn các lỗi thiếu sót trong quá trình sao chép mã nguồn thủ công.

PHẦN III

An Ninh, Xác Thực & Phân Quyền Doanh Nghiệp (Security & Auth)

CHƯƠNG 9

Cơ Chế Xác Thực Kép Passport Local & JWT Strategy

9.1 Mô Hình Xác Thực Không Lưu Trạng Thái (Stateless Authentication)

Trong các ứng dụng web truyền thống, trạng thái phiên làm việc (*Session State*) thường được lưu trữ trong bộ nhớ RAM của máy chủ hoặc bảng cơ sở dữ liệu. Khi hệ thống mở rộng theo chiều ngang (*Horizontal Scaling*) với hàng chục instance máy chủ đặt sau bộ cân bằng tải (Load Balancer), việc chia sẻ session đòi hỏi phải cấu hình Sticky Sessions hoặc duy trì một cụm Redis tập trung.

Hệ thống NestJS của dự án áp dụng mô hình **Xác Thực Không Lưu Trạng Thái (Stateless Authentication)** dựa trên chuẩn **JSON Web Token (JWT - RFC 7519)**:

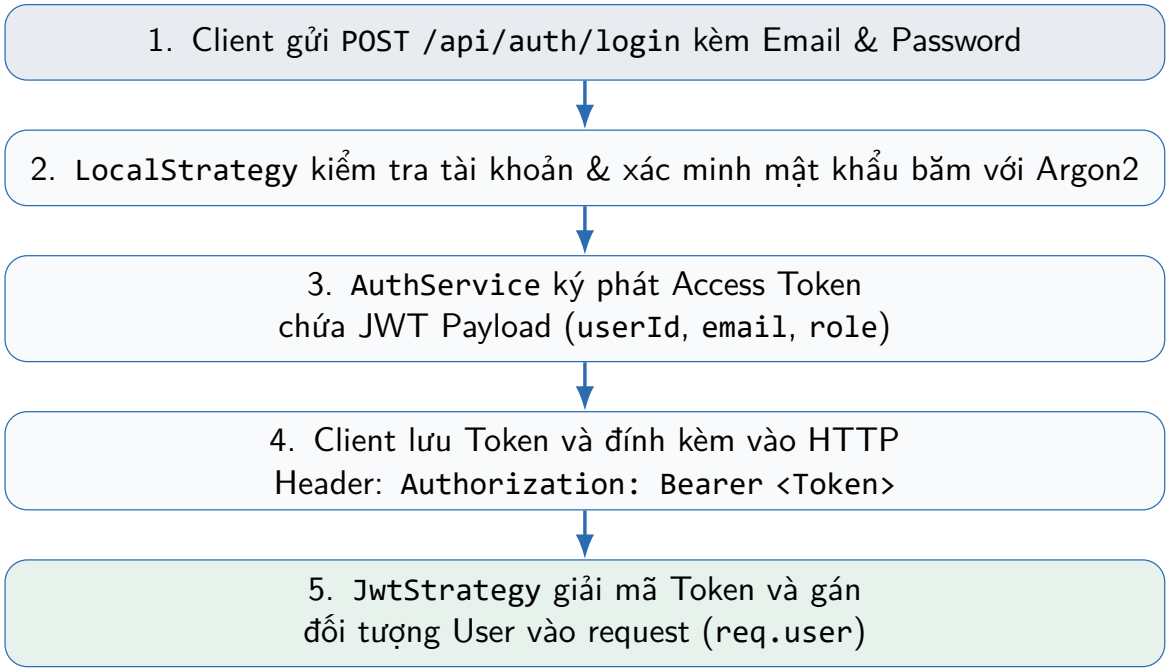
- Máy chủ backend hoàn toàn không lưu trữ phiên làm việc của người dùng trong bộ nhớ.
- Mọi thông tin danh tính thiết yếu (`userId`, `email`) được mã hóa và ký số bảo mật vào chuỗi Token.
- Mỗi HTTP request từ Client gửi lên đều mang theo Token này trong tiêu đề: `Authorization: Bearer <token>`. Bất kỳ máy chủ nào trong cụm đều có thể tự giải mã và xác minh tính hợp lệ của token độc lập trong tích tắc mà không cần truy vấn lại CSDL.

9.2 Giải Phẫu Chiến Lược Xác Thực `JwtStrategy`

Mô-đun xác thực khai thác thư viện `@nestjs/passport` và `passport-jwt`. Hãy giải phẫu tệp tin `src/modules/auth/strategy/jwt.strategy.ts`:

Mã nguồn JwtStrategy trong src/modules/auth/strategy/jwt.strategy.ts

```
import { Injectable, UnauthorizedException } from '@nestjs/common';
import { PassportStrategy } from '@nestjs/passport';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { ConfigService } from '@nestjs/config';
import { User } from '@database/typeorm/entities';
```



Hình 9.1: Quy trình xác thực người dùng bằng Passport Local & JWT

```
import { UserService } from '@modules/user/user.service';
import { JwtPayload } from '../interface/jwt-payload.interface';

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor(
    private readonly configService: ConfigService,
    private readonly userService: UserService,
  ) {
    super({
      // 1. Tự động trích xuất chuỗi Bearer Token từ header 'Authorization'
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      // 2. Bí mật mã hóa lấy an toàn từ ConfigService
      secretOrKey: configService.get('JWT_SECRET'),
      // 3. Nghiêm ngặt từ chối mọi token đã hết hạn (Expiration Check)
      ignoreExpiration: false,
    });
  }

  // 4. Được gọi tự động sau khi chữ ký mật mã của Token được giải mã thành công
  async validate(payload: JwtPayload): Promise<User> {
    const { userId } = payload;
    const user = await this.userService.findOneById(userId);
    if (!user) {
      throw new UnauthorizedException('PROF-104');
    }
    // Đối tượng User này sẽ được Passport tự động gán vào req.user!
```

```
    return user;
  }
}
```

Cơ Chế Tự Động Gắn `req.user`

Khi phương thức `validate()` trả về đối tượng `User`, Passport sẽ tự động gán nó vào thuộc tính `request.user`. Kể từ thời điểm này, mọi Controller, Guard hoặc Decorator (như `@CurrentUser()`) phía sau đều có thể truy cập thông tin người dùng đang đăng nhập mà không cần tốn thêm bất kỳ câu truy vấn cơ sở dữ liệu nào khác!

CHƯƠNG 10

Mã Hóa Mật Khẩu Thế Hệ Mới Với Thuật Toán Argon2

10.1 Tại Sao Argon2 Vượt Trội Hơn Bcrypt & PBKDF2?

Bảo mật thông tin đăng nhập của người dùng là yêu cầu tối thượng trong mọi kiến trúc phần mềm. Các giải thuật cũ như MD5, SHA-1, SHA-256 hoàn toàn không phù hợp để băm mật khẩu vì chúng được thiết kế để tính toán cực nhanh, giúp kẻ tấn công có thể thử hàng tỷ mật khẩu mỗi giây.

Bcrypt từng là tiêu chuẩn vàng trong nhiều năm, nhưng Bcrypt có điểm yếu cố hữu: nó chỉ là thuật toán ngốn CPU (*CPU-hard*). Với sự phát triển vượt bậc của các dàn card đồ họa GPU song song và chip chuyên dụng ASIC, kẻ tấn công có thể phân tán hàng nghìn luồng để bẻ khóa bcrypt với chi phí rất rẻ.

Argon2 đã xuất sắc giành giải Quán quân cuộc thi quốc tế Password Hashing Competition (PHC). Argon2 là thuật toán ngốn bộ nhớ (*Memory-Hard Function*):

$$\text{Argon2}(\text{password}, \text{salt}, m, t, p) \longrightarrow \text{Hash Output} \tag{10.1}$$

Trong đó:

- m (*Memory cost*): Dung lượng bộ nhớ RAM bắt buộc phải cấp phát khi tính toán (ví dụ: 64 MB).
- t (*Time cost*): Số vòng lặp tính toán qua mảng bộ nhớ.
- p (*Parallelism*): Số luồng tính toán song song độc lập.

Vì mỗi luồng băm Argon2 đòi hỏi một lượng RAM vật lý thực tế, một dàn card đồ họa GPU dù có hàng nghìn nhân tính toán cũng sẽ ngay lập tức bị **tràn bộ nhớ RAM (VRAM Out-of-Memory)**, làm tê liệt hoàn toàn các đòn tấn công vét cạn (*Brute-force Attacks*)!

10.2 So Sánh Chuyên Sâu Các Thuật Toán Băm Mật Khẩu

10.3 Tích Hợp Tự Động Trong Entity & AuthService

Trong dự án, việc băm mật khẩu được thực hiện tự động tại Entity `User` (`user.entity.ts`) thông qua các Entity Lifecycle Listeners:

Bảng 10.1: So sánh giải thuật băm mật khẩu Argon2 và Bcrypt

Tiêu chí kỹ thuật	Argon2 (Lựa chọn của dự án)	Bcrypt truyền thống
Cơ chế chống GPU Cracking	**Cực mạnh**: Chiếm dụng dung lượng RAM lớn khiến các dàn GPU/ASIC song song bị tràn bộ nhớ.	Yếu: Chỉ phụ thuộc số vòng lặp CPU, rất dễ bị các chip đồ họa đào bới vét cạn (*brute-force*).
Chống tấn công Side-channel	Có cơ chế bộ nhớ độc lập dữ liệu (*data-independent memory access*).	Dễ bị tấn công phân tích thời gian thực thi (*Timing Attack*).
Khuyến nghị tiêu chuẩn	OWASP và IETF khuyến nghị hàng đầu cho mọi hệ thống từ năm 2023.	Được coi là thuật toán thế hệ cũ.

Băm mật khẩu tự động tại Entity User

```
@BeforeInsert()
async hashPassword() {
  this.password = await argon2.hash(this.password);
}

@BeforeUpdate()
async updatePassword() {
  this.password = await argon2.hash(this.password);
}
```

Và việc xác minh mật khẩu khi người dùng đăng nhập được thực hiện an toàn trong `AuthService`:

Xác minh mật khẩu và phòng ngự User Enumeration trong AuthService

```
async validateAndGetUser(authCredentialDto: AuthCredentialDto): Promise<User> {
  const { email, password } = authCredentialDto;
  const isUserExisted = await this.userRepository.findOneBy({ email });

  // Ném mã lỗi LO-102 nếu không tìm thấy người dùng
  if (!isUserExisted) {
    throw new UnauthorizedException('LO-102');
  }

  // Xác minh mật khẩu bằng giải thuật Argon2
  const isValidPassword = await argon2.verify(isUserExisted.password, password);

  // Ném CÙNG mã lỗi LO-102 nếu mật khẩu không khớp!
  if (!isValidPassword) {
    throw new UnauthorizedException('LO-102');
  }
```

```
  return isUserExisted;
}
```

Phòng Ngự Tấn Công Dò Quét Tài Khoản (User Enumeration Defense)

Hãy chú ý kỹ: Trong cả hai trường hợp (1) Tài khoản không tồn tại và (2) Mật khẩu sai, hệ thống đều ném ra cùng một mã lỗi duy nhất: `LO-102` (tương ứng thông báo: *"Email hoặc mật khẩu không đúng"*).
Nếu bạn ném ra hai thông báo riêng biệt như *"Email chưa tồn tại"* và *"Mật khẩu không đúng"*, kẻ tấn công có thể viết script thử hàng triệu email để lập danh sách toàn bộ các tài khoản nội bộ của công ty. Đây là một lỗ hổng an ninh kinh điển mà mọi Kỹ sư Doanh nghiệp đều phải phòng ngừa!

CHƯƠNG 11

Đăng Nhập Xã Hội (Google OAuth2 ID Token Integration)

11.1 Kiến Trúc Định Danh OpenID Connect (OIDC) So Với OAuth 2.0

Trong kỹ thuật bảo mật hiện đại, một hiểu lầm phổ biến là đồng nhất **OAuth 2.0** với xác thực người dùng. Về bản chất, OAuth 2.0 là một giao thức **ủy quyền (Authorization)** được thiết kế để cấp phép cho một bên thứ ba truy cập tài nguyên thay mặt người dùng (thông qua Access Token) mà không cần biết mật khẩu của họ. OAuth 2.0 tự nó không cung cấp bất kỳ định dạng chuẩn nào để mô tả danh tính của người dùng.

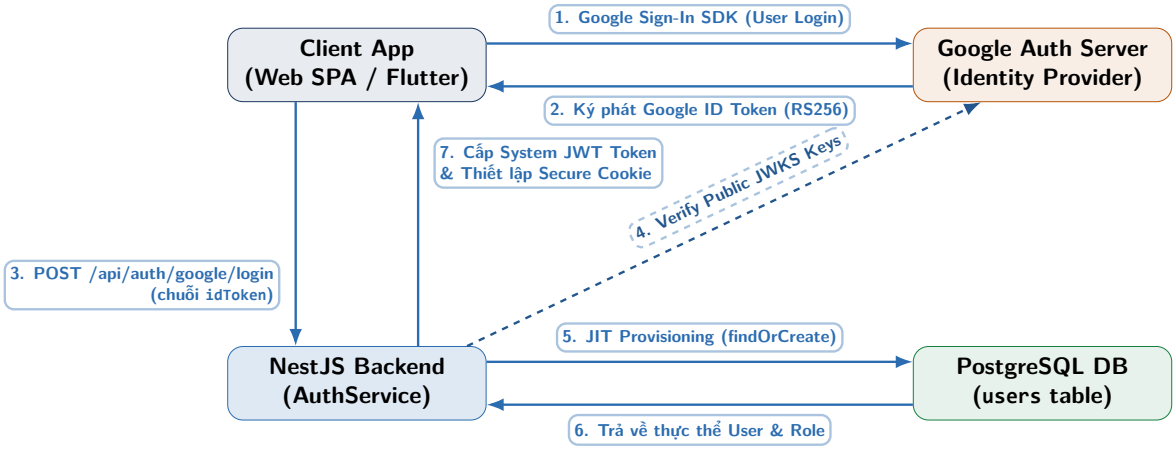
Để giải quyết khoảng trống này, liên minh OpenID Foundation đã xây dựng **OpenID Connect (OIDC)** như một tầng định danh (*Identity Layer*) nằm ngay trên đỉnh của OAuth 2.0. OIDC chuẩn hóa việc phát hành **ID Token** — một chuỗi JSON Web Token (JWT) được ký số bằng thuật toán bất đối xứng RS256 bởi máy chủ danh tính của Google, chứa các thuộc tính định danh chuẩn hóa (*Claims*) như: mã định danh người dùng duy nhất (`sub`), địa chỉ email (`email`), trạng thái xác minh email (`email_verified`), họ tên (`name`) và ảnh đại diện (`picture`).

11.2 Luồng Xác Thực Google ID Token 4 Tác Nhân

Để tối ưu hóa trải nghiệm người dùng trên cả ứng dụng Single Page Application (React/Vue/Angular) và ứng dụng di động (Flutter/React Native), hệ thống áp dụng luồng xác thực **Google ID Token Một Chạm (One-Tap / Client-Driven ID Token Flow)**. Mô hình này giúp ứng dụng không phải chuyển hướng trình duyệt (*Browser Redirect*) phức tạp, đồng thời hoạt động trơn tru trên mọi nền tảng:

11.3 Giải Phẫu Thẩm Định Dữ Liệu Với `AuthGoogleLoginDto`

Tại tệp tin `src/modules/auth/dto/auth-google-login.dto.ts`, đối tượng nhận dữ liệu được thiết kế tối giản nhưng được bảo vệ chặt chẽ bởi các Decorator thẩm định:



Hình 11.1: Luồng xác thực Google ID Token và Cung cấp tài khoản tức thì (JIT)

Mã nguồn AuthGoogleLoginDto trong src/modules/auth/dto/

```
import { ApiProperty } from '@nestjs/swagger';
import { IsNotEmpty, IsString } from 'class-validator';

export class AuthGoogleLoginDto {
  @ApiProperty({
    description: 'Chuỗi Google ID Token (JWT) do Google Sign-In SDK cấp phát',
    example: 'eyJhbGciOiJSUzI1NiIsImtpZCI6IjEyMzQ1... (Google Signed JWT)',
  })
  @IsString({ message: 'ID Token bắt buộc phải là định dạng chuỗi ký tự' })
  @IsNotEmpty({ message: 'ID Token không được phép để trống' })
  idToken: string;
}
```

11.4 Xác Minh Chữ Ký Điện Tử Bất Đối Xứng Với google-auth-li

Thay vì gọi các endpoint HTTP tốn kém của Google trên từng request hoặc tự giải mã chuỗi JWT mà không kiểm tra chữ ký, hệ thống sử dụng thư viện chính thức `google-auth-library` của Google. Thư viện này tự động nạp và lưu bộ đệm các khóa công khai (*Public Keys*) từ điểm cuối JWKS của Google (`https://www.googleapis.com/oauth2/v3/certs`), sau đó tiến hành kiểm tra toán học chữ ký RS256 ngay trên CPU của máy chủ:

Xác minh ID Token tại AuthService

```
import { OAuth2Client } from 'google-auth-library';
import { UnauthorizedException } from '@nestjs/common';

@Injectable()
export class AuthService {
  private googleClient: OAuth2Client;

  constructor(
```

```
    private readonly configService: ConfigService,
    private readonly userService: UserService,
    private readonly jwtService: JwtService,
  ) {
    this.googleClient = new OAuth2Client(
      this.configService.get<string>('GOOGLE_CLIENT_ID'),
    );
  }

  async loginGoogle(dto: AuthGoogleLoginDto) {
    const clientId = this.configService.get<string>('GOOGLE_CLIENT_ID');
    let ticket;
    try {
      // 1. Xác minh chữ ký mật mã, thời gian hiệu lực và bên nhận (Audience)
      ticket = await this.googleClient.verifyIdToken({
        idToken: dto.idToken,
        audience: clientId,
      });
    } catch (error) {
      throw new UnauthorizedException('Mã Google ID Token không hợp lệ hoặc đã hết hạn');
    }

    const payload = ticket.getPayload();
    if (!payload || !payload.email_verified) {
      throw new UnauthorizedException('Email Google chưa được xác minh bởi Google');
    }

    const { email, name, sub: googleId, picture } = payload;

    // 2. Cung cấp tài khoản tức thì (Just-In-Time Provisioning)
    const user = await this.userService.findOrCreateBySocialProvider({
      email,
      name,
      googleId,
      avatar: picture,
      provider: AuthProvidersEnum.GOOGLE,
    });

    // 3. Ký phát JWT Token của hệ thống nội bộ
    const access_token = this.jwtService.sign({
      userId: user.id,
      email: user.email,
      role: user.role.name,
    });

    return { access_token, user };
  }
}
```

11.5 Phòng Ngự Tấn Công Chiếm Quyền Tài Khoản (Account Hijacking Defense)

Khi tích hợp đăng nhập mạng xã hội song song với đăng nhập truyền thống bằng email/mật khẩu, hệ thống đối mặt với nguy cơ an ninh nghiêm trọng: **Tấn công chiếm quyền tài khoản thông qua trùng lặp Email** (*Account Takeover via Pre-Account Creation*).

Kịch Bản Tấn Công Chiếm Quyền Tài Khoản & Cơ Chế Phòng Ngự

Kịch bản tấn công: Kẻ xấu biết email của nạn nhân (`victim@company.com`). Kẻ xấu chủ động đăng ký tài khoản cục bộ trên hệ thống với email đó và đặt mật khẩu mà kẻ xấu nắm giữ. Vài ngày sau, nạn nhân truy cập vào website và bấm nút *"Đăng nhập bằng Google"* bằng chính email `victim@company.com`. Nếu hệ thống ngây thơ tự động liên kết tài khoản Google với tài khoản cũ mà không kiểm tra, kẻ xấu vẫn giữ mật khẩu và có thể đăng nhập vào tài khoản của nạn nhân bất cứ lúc nào!

Cơ chế phòng thủ của hệ thống:

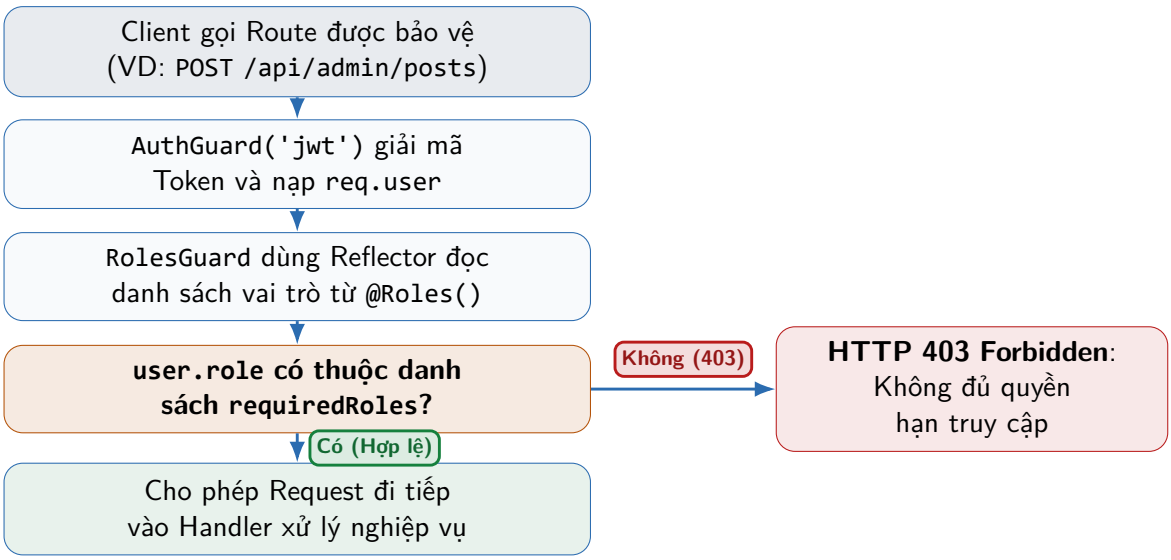
1. Hệ thống bắt buộc thẩm định cờ `payload.email_verified === true` từ Google Identity để chắc chắn Google đã chứng thực quyền sở hữu hòm thư.
2. Nếu tài khoản đã tồn tại với `provider === 'LOCAL'` (đăng ký qua mật khẩu), hệ thống **tuyệt đối không tự động ghi đè hoặc hợp nhất**. Hệ thống sẽ từ chối đăng nhập và yêu cầu người dùng đăng nhập bằng mật khẩu truyền thống trước, sau đó chủ động liên kết tài khoản Google từ trang Quản lý hồ sơ cá nhân (*Account Linking Flow*).

CHƯƠNG 12

Phân Quyền Theo Vai Trò (Role-Based Access Control - RBAC)

12.1 Sự Kết Hợp Giữa Decorator @Roles() & RolesGuard

Kiểm soát truy cập tài nguyên dựa trên vai trò (RBAC) được thực hiện thông qua sự phối hợp nhịp nhàng giữa TypeScript Decorator và NestJS Guard:



Hình 12.1: Chu trình kiểm tra phân quyền Role-Based Access Control

12.2 Giải Phẫu Mã Nguồn RolesGuard

Hãy xem xét cách thức `RolesGuard` (`src/modules/auth/guard/roles.guard.ts`) vận hành:

Mã nguồn RolesGuard trong src/modules/auth/guard/roles.guard.ts

```
import { Injectable, CanActivate, ExecutionContext, ForbiddenException } from '@nestjs/common
import { Reflector } from '@nestjs/core';
import { USER_ROLE } from '@shared/enum/user.enum';
import { ROLES_KEY } from './roles.decorator';
import { UserService } from '@modules/user/user.service';
```

```
@Injectable()
export class RolesGuard implements CanActivate {
  constructor(
    private reflector: Reflector,
    private readonly userService: UserService,
  ) {}

  async canActivate(context: ExecutionContext): Promise<boolean> {
    // 1. Dùng Reflector trích xuất metadata: Ưu tiên cấp Method ghi đè cấp Class
    const requiredRoles = this.reflector.getAllAndOverride<USER_ROLE[]>(
      ROLES_KEY,
      [context.getHandler(), context.getClass()],
    );

    // Nếu route không yêu cầu vai trò đặc biệt nào, cho phép đi qua
    if (!requiredRoles) {
      return true;
    }

    // 2. Lấy đối tượng user đã được JwtStrategy xác thực trước đó
    const { user } = context.switchToHttp().getRequest();
    const currentUser = await this.userService.findRoleIdByUserId(user.userId);
    user['role'] = currentUser.role;

    // 3. Kiểm tra xem vai trò của user có nằm trong danh sách cho phép hay không
    const hasRole = requiredRoles.some((role) => user.role === role);
    if (!hasRole) {
      throw new ForbiddenException('ADMIN-113');
    }

    return true;
  }
}
```

Ý Nghĩa Của Phương Thức `getAllAndOverride`

Hàm `reflector.getAllAndOverride()` là một thiết kế thông minh: nó cho phép kỹ sư đặt decorator `@Roles(USER_ROLE.MEMBER)` ở cấp độ toàn bộ Controller, nhưng trên một Action nhạy cảm cụ thể (như xóa dữ liệu), kỹ sư có thể gắn `@Roles(USER_ROLE.ADMIN)`. Phương thức cấp Action sẽ tự động ghi đè (*override*) cấu hình cấp Controller mà không xảy ra xung đột!

PHẦN IV

Hạ Tầng Dịch Vụ Dùng Chung & Tối Ưu Vận Hành (Cross-Cutting Concerns)

CHƯƠNG 13

Hệ Thống Lọc Ngoại Lệ Toàn Cục (Exception Filters Pipeline)

13.1 Kiến Trúc Bắt Lỗi Tập Trung Trong Doanh Nghiệp

Trong một ứng dụng doanh nghiệp có quy mô lớn, lỗi có thể phát sinh từ bất kỳ đâu: lỗi thẩm định DTO từ Client, lỗi không tìm thấy bản ghi từ Service, lỗi vi phạm ràng buộc khóa ngoại từ PostgreSQL, hoặc lỗi sập mạng từ một dịch vụ bên thứ ba.

Nếu không có một tầng xử lý lỗi tập trung:

- Các controller sẽ bị ngập trong hàng tá khối `try-catch` lặp đi lặp lại.
- Định dạng lỗi trả về cho Client sẽ bị phân mảnh: chỗ trả về chuỗi text, chỗ trả về đối tượng có cấu trúc khác nhau.
- Nguy hiểm nhất: các ngoại lệ hệ thống không được kiểm soát có thể để lộ toàn bộ **Stack Trace**, cấu trúc thư mục máy chủ, và thậm chí cả chuỗi truy vấn cơ sở dữ liệu nội bộ ra ngoài mạng Internet!

Dự án thiết lập một chuỗi 8 bộ lọc ngoại lệ chuyên biệt tại thư mục `src/core/filters/`:

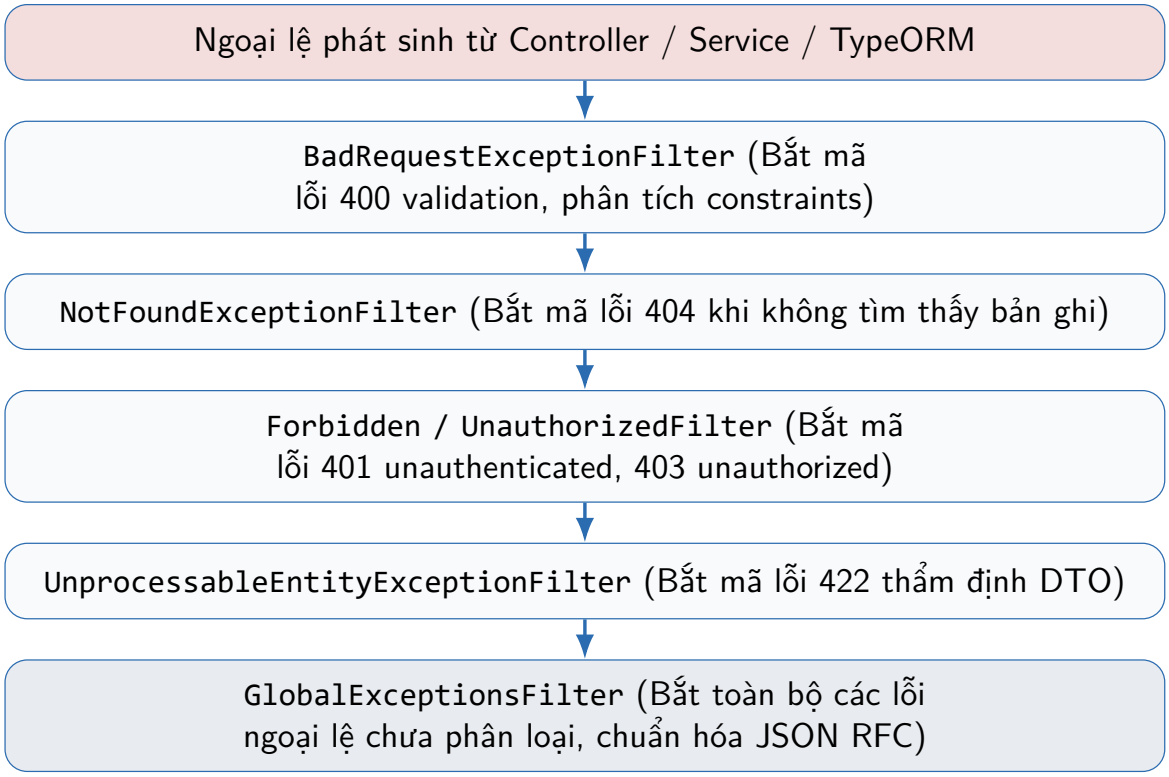
13.2 Giải Phẫu Mã Nguồn `BadRequestExceptionFilter`

Hãy xem xét cách thức `BadRequestExceptionFilter` (`src/core/filters/bad-request.filter.ts`) hoạt động:

Mã nguồn BadRequestExceptionFilter trong src/core/filters/bad-request.filter.ts

```
import { NodeEnv } from '@core/enum';
import { ArgumentsHost, BadRequestException, Catch, ExceptionFilter } from '@nestjs/common';
import { Response } from 'express';
import { ErrorService } from '@shared/services/error.service';
import { I18nContext } from 'nestjs-i18n';
import { ConfigService } from '@nestjs/config';
import { ProjectLogger } from '@core/loggers';

@Catch(BadRequestException)
```



Hình 13.1: Chuỗi xử lý ngoại lệ trong NestJS Exception Filters Pipeline

```
export class BadRequestExceptionFilter implements ExceptionFilter {
  logger = new ProjectLogger('GlobalExceptionsFilter');

  constructor(
    private readonly errorService: ErrorService,
    private readonly configService: ConfigService,
  ) {}

  catch(exception: BadRequestException, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const res = ctx.getResponse<Response>();
    const status = exception.getStatus();
    const errors = exception.getResponse();
    const i18n = I18nContext.current(host);
    const stackTrace = exception.stack;

    // Ghi nhật ký phân nhánh theo môi trường
    if (this.configService.get('NODE_ENV') == NodeEnv.DEVELOPMENT) {
      this.logger.info(exception.message);
    } else {
      this.logger.exception(exception.stack);
    }

    // Định dạng và dịch mã lỗi sang ngôn ngữ yêu cầu qua ErrorService
```

```
    res.status(status).json(this.errorService.message(errors, i18n, stackTrace));
  }
}
```

Quy trình này bảo đảm sự phân tách trách nhiệm hoàn hảo: Filter chỉ chịu trách nhiệm bắt giữ ngoại lệ HTTP, trích xuất ngữ cảnh (`ArgumentsHost`) và ủy thác việc tạo cấu trúc dữ liệu JSON cho `ErrorService`.

CHƯƠNG 14

Hệ Thống Nhật Ký Đa Môi Trường Với Winston Logger

14.1 Tầm Quan Trọng Của Structured Logging

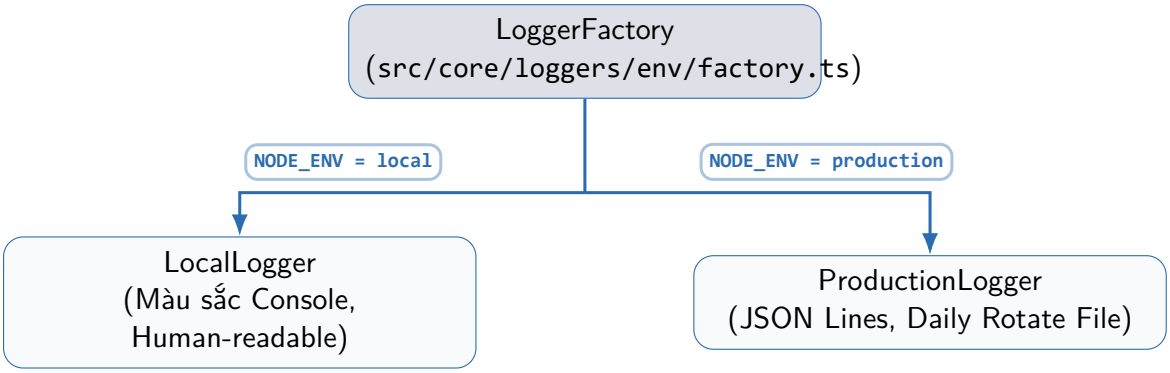
Trong môi trường sản xuất của hệ thống phân tán, việc ghi log bằng lệnh `console.log()` truyền thống là một phản mẫu nguy hiểm vì:

1. Lệnh `console.log` trong Node.js là đồng bộ (*synchronous*) khi ghi ra tệp tin hoặc pipe trên một số nền tảng, có thể gây chặn (*blocking*) Event Loop dưới tải cao.
2. Chuỗi log dạng text thuần vô cùng khó để các hệ thống giám sát tập trung (Elasticsearch/Logstash, Grafana Loki, Datadog) có thể bóc tách (*parsing*) và truy vấn theo trường.

Dự án xây dựng hệ thống ghi log có cấu trúc (**Structured Logging**) dựa trên thư viện **Winston** và **winston-daily-rotate-file**.

14.2 Thiết Kế Factory Pattern Cho Logger Tại `src/core/loggers/`

Tại `src/core/loggers/env/factory.ts`, hệ thống sử dụng Factory Pattern để tự động trả về phiên bản Logger phù hợp với biến môi trường `NODE_ENV`:



Hình 14.1: Mô hình LoggerFactory đa môi trường

Mã nguồn LoggerFactory trong src/core/loggers/env/factory.ts

```
import { LoggerOptions } from 'typeorm';
import { LocalLogger } from '@core/loggers/env/local';
import { ProductionLogger } from '@core/loggers/env/production';
import { NodeEnv } from '@core/enum';
import { ConfigService } from '@nestjs/config';

const loggerMap = {
  [NodeEnv.DEVELOPMENT]: LocalLogger,
  [NodeEnv.PRODUCTION]: ProductionLogger,
};

const configService = new ConfigService();

export class LoggerFactory {
  static getInstance(
    loggerOptions: LoggerOptions,
    name: string,
  ): LocalLogger | ProductionLogger {
    const instance = loggerMap[configService.get('NODE_ENV')];
    return new instance(loggerOptions, name);
  }
}
```

- **LocalLogger** (`development`): Định dạng có màu sắc trực quan (`format.colorize()`), hiển thị dấu thời gian rõ ràng, giúp kỹ sư dễ dàng theo dõi trực tiếp trên terminal máy tính cá nhân.
- **ProductionLogger** (`production`): Ghi nhật ký dưới dạng **JSON Lines** chuẩn hóa. Mỗi dòng log là một đối tượng JSON độc lập chứa `timestamp`, `level`, `context`, `message`, `traceId`. Đồng thời kích hoạt `DailyRotateFile` với các tham số:
  - `datePattern: 'YYYY-MM-DD'`: Tự động tạo tệp tin log mới mỗi ngày.
  - `maxSize: '50m'`: Tự động cắt tệp khi kích thước đạt 50 MB.
  - `maxFiles: '15'`: Tự động xóa các tệp tin cũ hơn 15 ngày để tiết kiệm dung lượng đĩa cứng máy chủ.
  - `zippedArchive: true`: Tự động nén tệp tin thành định dạng `.gz`.

14.3 Theo Dõi Yêu Cầu Với `RequestLoggerMiddleware`

Tại `src/shared/middleware/request-logger.middleware.ts`, hệ thống đo lường và ghi nhận mọi truy cập mạng đi qua máy chủ:

Mã nguồn RequestLoggerMiddleware

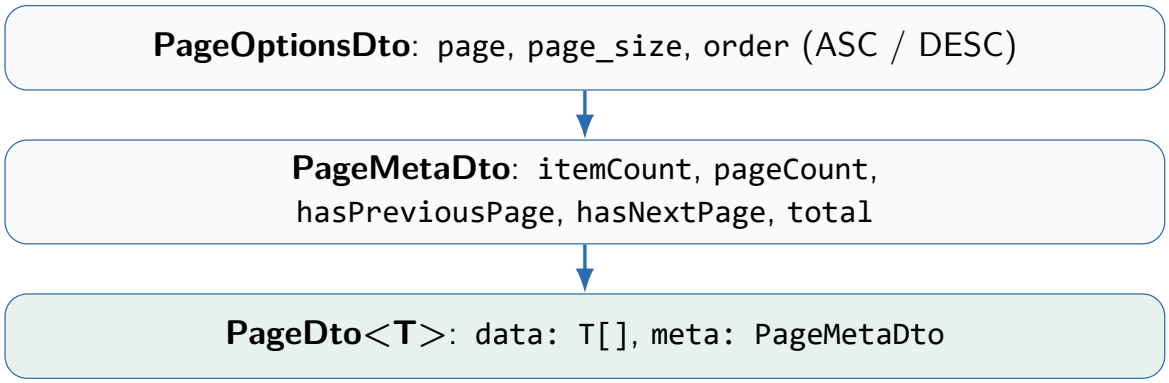
```
export const requestLoggerMiddleware = (req: Request, res: Response, next: NextFunction) => {
  const logger = new ProjectLogger('RequestLoggerMiddleware');
  const { method, url, body } = req;
  logger.info(`Request: ${method} ${url} ${JSON.stringify(body)}`);
  next();
};
```

CHƯƠNG 15

Hệ Thống Phân Trang Doanh Nghiệp (Pagination Architecture)

15.1 Thiết Kế Bộ Ba Đối Tượng Phân Trang Chuẩn Hóa

Tại `src/core/pagination/`, hệ thống phân trang được trừu tượng hóa thành 3 đối tượng chuẩn:



Hình 15.1: Mô hình đối tượng Phân Trang chuẩn hóa

1. **`PageOptionsDto`** (`src/core/pagination/dto/page-option.dto.ts`): Chịu trách nhiệm nhận và thẩm định các query params từ client:

Mã nguồn PageOptionsDto

```
export class PageOptionsDto {
  @ApiPropertyOptional({ enum: Order, default: Order.ASC })
  @IsEnum(Order)
  @IsOptional()
  readonly order?: Order = Order.ASC;

  @Type(() => Number)
  @IsInt()
  @IsOptional()
  readonly page?: number = DEFAULT_PAGE; // Mặc định: 1

  @Type(() => Number)
```

```
  @IsInt()
  @Max(MAX_LIMIT) // Phòng vệ DoS: Không cho phép client yêu cầu quá 50 bản ghi!
  @IsOptional()
  readonly page_size?: number = DEFAULT_LIMIT; // Mặc định: 10

  get skip(): number {
    return (this.page - 1) * this.page_size;
  }
}
```

2. **PageMetaDto** (`src/core/pagination/dto/page-meta.dto.ts`): Tính toán các chỉ số thống kê phân trang:

Mã nguồn PageMetaDto

```
export class PageMetaDto {
  readonly page: number;
  readonly limit: number;
  readonly itemCount: number;
  readonly pageCount: number;
  readonly hasPreviousPage: boolean;
  readonly hasNextPage: boolean;

  constructor({ pageOptionsDto, itemCount }: IPageMetaDtoParameters) {
    this.page = pageOptionsDto.page;
    this.limit = pageOptionsDto.page_size;
    this.itemCount = itemCount;
    this.pageCount = Math.ceil(this.itemCount / this.limit);
    this.hasPreviousPage = this.page > 1;
    this.hasNextPage = this.page < this.pageCount;
  }
}
```

3. **PageDto<T>** (`src/core/pagination/dto/page.dto.ts`): Đóng gói mảng dữ liệu thực thể kèm siêu dữ liệu `meta` thành cấu trúc JSON phản hồi duy nhất gửi về cho phía Frontend.

15.2 Công Thức Toán Học & Câu Lệnh SQL Bên Dưới

Tại tầng Service, TypeORM QueryBuilder chuyển đổi các thuộc tính này thành câu lệnh SQL tối ưu:

$$\text{skip} = (\text{page} - 1) \times \text{limit} \tag{15.1}$$

$$\text{pageCount} = \left\lceil \frac{\text{itemCount}}{\text{limit}} \right\rceil \tag{15.2}$$

Câu lệnh SQL phát sinh tương ứng trên PostgreSQL:

SQL phát sinh khi phân trang

```
SELECT * FROM "tasks"
ORDER BY "created_at" ASC
LIMIT 10 OFFSET 20; -- Lấy 10 dòng, bỏ qua 20 dòng đầu tiên (Trang 3)
```

CHƯƠNG 16

Đa Ngôn Ngữ & Quản Trị Lỗi Với nestjs-i18n

16.1 Cấu Trúc Thư Mục Từ Điển Đa Ngôn Ngữ

Hệ thống tích hợp thư viện `nestjs-i18n` với cấu trúc thư mục từ điển đặt tại:

- `src/i18n/en/errors.json`: Từ điển tiếng Anh chuẩn quốc tế.
- `src/i18n/vi/errors.json`: Từ điển tiếng Việt bản địa hóa.

Cơ chế phân giải ngôn ngữ tự động dựa trên tiêu đề HTTP chuẩn:

```
Accept-Language: vi-VN,vi;q=0.9,en-US;q=0.8
```
(16.1)

NestJS tự động nhận diện ngôn ngữ ưu tiên cao nhất của người dùng và nạp ngữ cảnh `I18nContext` tương ứng.

16.2 Giải Phẫu Toàn Diện Dịch Vụ ErrorService

Tệp tin `src/shared/services/error.service.ts` là trung tâm dịch và định dạng lỗi của toàn bộ dự án:

Mã nguồn hoàn chỉnh của ErrorService: src/shared/services/error.service.ts

```
import { Injectable } from '@nestjs/common';
import { ValidationError } from 'class-validator';
import { I18nContext } from 'nestjs-i18n';
import { ConfigService } from '@nestjs/config';
import { NodeEnv } from '@core/enum';
import * as _ from 'lodash';

@Injectable()
export class ErrorService {
  constructor(private readonly configService: ConfigService) {}

  message(err: string | object, i18n: I18nContext, stackTrace: any) {
    const errorMessage = _.isArray(err['message']) ? err['message'][0] : err['message'];
    switch (err['error']) {
      case 'Bad Request':
      case 'Forbidden':
```

```
      case 'Not Found':
      case 'Unauthorized':
        return this.combine(errorMessage, i18n);
      case 'Internal Server Error':
        // Nếu ở môi trường Development, trả về kèm StackTrace để lập trình viên sửa lỗi
        if (this.configService.get('NODE_ENV') == NodeEnv.DEVELOPMENT) {
          return this.combineDev(errorMessage, i18n, stackTrace);
        }
        return this.combine(errorMessage, i18n);
      case 'Unprocessable Entity':
        return this.handleUnprocessEntity(err, i18n);
      default:
        return this.combine(errorMessage, i18n);
    }
  }

  private handleUnprocessEntity(error: string | object, i18n: I18nContext) {
    const transformedError = this.transform(error['message'], i18n).flat();
    return this.combine('CUS-0401', i18n, transformedError);
  }

  // Biến đổi mảng ValidationError từ class-validator thành câu thông báo dịch
  private transform(errors: ValidationError[], i18n: I18nContext) {
    const transformedError = errors.map((err: ValidationError) => {
      return Object.values(err.constraints).map((value: string) => {
        return Object.assign(
          i18n.t(`errors.${value}`, {
            args: { property: err.property },
          }),
          { field: err.property },
        );
      });
    });
    return transformedError;
  }

  private combine(err_msg: string, i18n: I18nContext, errors: object[] = []) {
    return Object.assign(i18n?.t ? i18n.t(`errors.${err_msg}`) : err_msg, {});
  }

  private combineDev(err_msg: string, i18n: I18nContext, stackTrace: any) {
    return Object.assign(i18n.t(`errors.${err_msg}`), { details: stackTrace });
  }
}
```

Cơ Chế Khớp Chuỗi Thẩm Định Động

Hãy quan sát phương thức `transform()`: Khi `class-validator` vi phạm ràng buộc định dạng email, nó sinh ra chuỗi constraint `"email must be an email"`. `ErrorService` sẽ gọi `i18n.t('errors.email must be an email')`, chuỗi này khớp chính xác với khóa đã định nghĩa trong `src/i18n/vi/errors.json`, từ đó tự động trả về thông báo tiếng Việt: *"Định dạng trường email không hợp lệ: Địa chỉ email phải chứa ít nhất một ký tự trước và sau ký hiệu '@'..."* kèm theo trường vi phạm `{ field: "email" }`. Đây là giải pháp xử lý lỗi bản địa hóa chuẩn mực và thanh lịch nhất trong hệ sinh thái NestJS!

PHẦN V

Các Mẫu Thiết Kế Nghiệp Vụ Thực Tế (Business Patterns in Practice)

CHƯƠNG 17

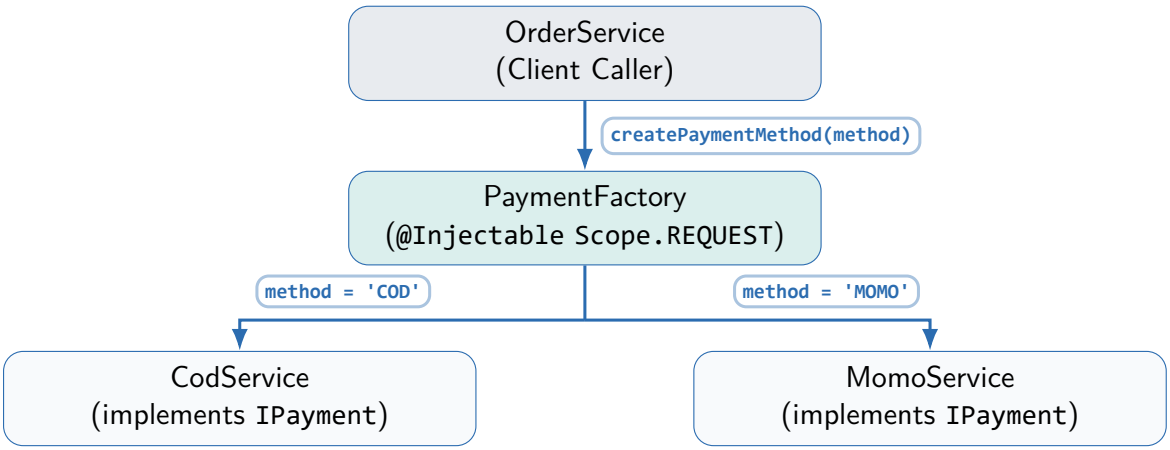
Mẫu Thiết Kế Factory Pattern Trong Thanh Toán Trực Tuyến

17.1 Ứng Dụng GoF Factory Method Trong Thanh Toán Đa Kênh

Trong các hệ thống thương mại điện tử, việc hỗ trợ nhiều kênh thanh toán (MoMo, ZaloPay, VNPay, COD, Thẻ tín dụng Stripe) là yêu cầu thiết yếu. Nếu lập trình viên xử lý trực tiếp trong Controller bằng hàng loạt khối `if-else` hoặc `switch-case`:

- Controller bị phình to và vi phạm nghiêm trọng **Nguyên Lý Đóng/Mở (Open/Closed Principle - SOLID OCP)**: Mỗi khi doanh nghiệp ký hợp đồng với một cổng thanh toán mới, kỹ sư bắt buộc phải sửa đổi trực tiếp mã nguồn của Controller và Service cốt lõi, tiềm ẩn rủi ro làm hỏng các kênh thanh toán đang chạy ổn định.
- Không thể kiểm thử cô lập logic của từng cổng thanh toán riêng biệt.

Dự án áp dụng mẫu thiết kế kinh điển **GoF Factory Method Pattern** kết hợp với giao diện chung `IPayment` tại thư mục `src/modules/order/factories/`:



Hình 17.1: Mô hình GoF Factory Method áp dụng trong xử lý thanh toán

17.2 Giải Phẫu PaymentFactory & Vòng Đời Scope.REQUEST

Tệp tin `src/modules/order/factories/payment.factory.ts` điều phối việc định tuyến tới triển khai cụ thể:

Mã nguồn PaymentFactory trong src/modules/order/factories/payment.factory.ts

```
import { Injectable, Scope } from '@nestjs/common';
import { CreateOrderDto } from '../dto/create-order.dto';
import { CodService } from './cod.service';
import { MomoService } from './momo.service';
import { PaymentMethod } from '@shared/enum/payment-method.enum';

@Injectable({ scope: Scope.REQUEST })
export class PaymentFactory {
  constructor(
    private readonly momoService: MomoService,
    private readonly codService: CodService,
  ) {}

  createPaymentMethod(method: string, order: CreateOrderDto, orderID: number) {
    switch (method) {
      case `${PaymentMethod.MOMO}`:
        return this.momoService.processingPayment(order, orderID);
      case `${PaymentMethod.COD}`:
        return this.codService.processingPayment(order, orderID);
      default:
        throw new Error('Invalid implementation payment');
    }
  }

  createCaptureOrder(method: string, orderID: number, paymentOrderID: string) {
    switch (method) {
      case `${PaymentMethod.MOMO}`:
        return this.momoService.captureOrder(orderID, paymentOrderID);
      default:
        throw new Error('Invalid implementation payment');
    }
  }

  cancelOrder(method: string, orderID: number, paymentOrderID: string) {
    switch (method) {
      case `${PaymentMethod.MOMO}`:
        return this.momoService.cancelOrder(orderID, paymentOrderID);
      default:
        throw new Error('Invalid implementation payment');
    }
  }
}
```

17.3 Tích Hợp Cổng MoMo Với Chữ Ký Điện Tử HMAC-SHA256

Mọi payload gửi sang MoMo API đều phải được ký số mật mã bằng thuật toán **HMAC-SHA256** nhằm bảo đảm không bị kẻ xấu can thiệp thay đổi giá trị đơn hàng trên đường truyền Internet:

Mã nguồn tính toán chữ ký số HMAC-SHA256 trong MomoService

```
const requestId = this.configService.get('momo_gateway.mm_partner_code') + new Date().getTime

// Xây dựng chuỗi Raw Signature theo định dạng quy chuẩn của MoMo
const rawSignature = `accessKey=${accessKey}&amount=${amount}&extraData=${extraData}&ipnUrl=$

// Băm mật mã bằng Secret Key bí mật
const signature = crypto
  .createHmac('sha256', this.configService.get('momo_gateway.mm_secret_key'))
  .update(rawSignature)
  .digest('hex');
```

Khi người dùng thanh toán thành công trên ứng dụng MoMo, máy chủ MoMo sẽ gửi một Webhook IPN (*Instant Payment Notification*) bất đồng bộ về endpoint máy chủ NestJS. Backend sẽ kiểm tra chữ ký ngược lại: nếu hợp lệ, tiến hành cập nhật trạng thái đơn hàng sang `ORDER_STATUS.COMPLETED`.

CHƯƠNG 18

Quản Lý Đơn Hàng Master-Detail & Giao Dịch Dữ Liệu

18.1 Kiến Trúc Quan Hệ Thực Thể Master-Detail Trong E-Commerce

Trong thiết kế cơ sở dữ liệu doanh nghiệp và hệ thống thương mại điện tử, quan hệ **Master-Detail** (hay còn gọi là *Parent-Child Relationship*) là mô hình chuẩn mực để mô hình hóa các chứng từ giao dịch nhiều dòng:

- **Bảng Master (`orders`)**: Đại diện cho bản thân hợp đồng đơn hàng tổng thể. Lưu trữ các thuộc tính mang tính tổng hợp: thông tin người nhận (`customer_name`, `customer_phone`), tổng giá trị đơn hàng (`total`), tỷ lệ thuế suất (`tax`), đơn vị tiền tệ thanh toán (`currency`), trạng thái thanh toán (`status`), khóa ngoại người tạo (`user_id`), mã đơn của cổng thanh toán (`payment_order_id`) và phương thức thanh toán (`payment_method_id`).
- **Bảng Detail (`order_details`)**: Đại diện cho từng mục mặt hàng cụ thể nằm trong giỏ hàng. Lưu trữ: khóa ngoại `order_id` trỏ về đơn hàng cha, khóa ngoại `product_id` trỏ về danh mục sản phẩm, số lượng đặt mua (`quantity`), và đơn giá bán (`price`).

Nguyên Tắc Bất Biến Về Lịch Sử Giá Trong OrderDetails

Một sai lầm chí mạng của các lập trình viên thiếu kinh nghiệm là **không lưu trường đơn giá (`price`) trong bảng con `order_details`**, mà chỉ lưu `product_id` rồi thực hiện câu lệnh `JOIN` sang bảng `products` để lấy giá bán khi xem lại lịch sử đơn hàng. Khi doanh nghiệp tăng giá sản phẩm từ 100,000đ lên 150,000đ vào tháng sau, toàn bộ các đơn hàng cũ trong quá khứ của khách hàng sẽ bị tính lại theo giá mới, gây sai lệch báo cáo tài chính kế toán và vi phạm pháp luật! Vì vậy, `order_details.price` là thuộc tính **chụp nhanh tại thời điểm giao dịch** (*Point-in-time Snapshot*) và phải được cố định vĩnh viễn.

18.2 Định Nghĩa Thực Thể Với Quan Hệ TypeORM

Trong tệp `src/database/typeorm/entities/order.entity.ts` và `order-detail.entity.ts`, quan hệ được thiết lập với tính năng Cascading:

Mô hình hóa quan hệ OneToMany giữa Order và OrderDetails

```
@Entity('orders')
export class Order extends AbstractEntity {
  @Column({ type: 'varchar', length: 255 })
  customer_name: string;

  @Column({ type: 'varchar', length: 20 })
  customer_phone: string;

  @Column({ type: 'decimal', precision: 12, scale: 2 })
  total: number;

  @Column({ type: 'decimal', precision: 4, scale: 2, default: 0.08 })
  tax: number; // Thuế suất VAT (8%)

  @Column({ type: 'enum', enum: ORDER_STATUS, default: ORDER_STATUS.PENDING })
  status: ORDER_STATUS;

  @Column({ type: 'varchar', nullable: true })
  payment_order_id: string;

  @ManyToOne(() => PaymentMethod)
  @JoinColumn({ name: 'payment_method_id' })
  paymentMethod: PaymentMethod;

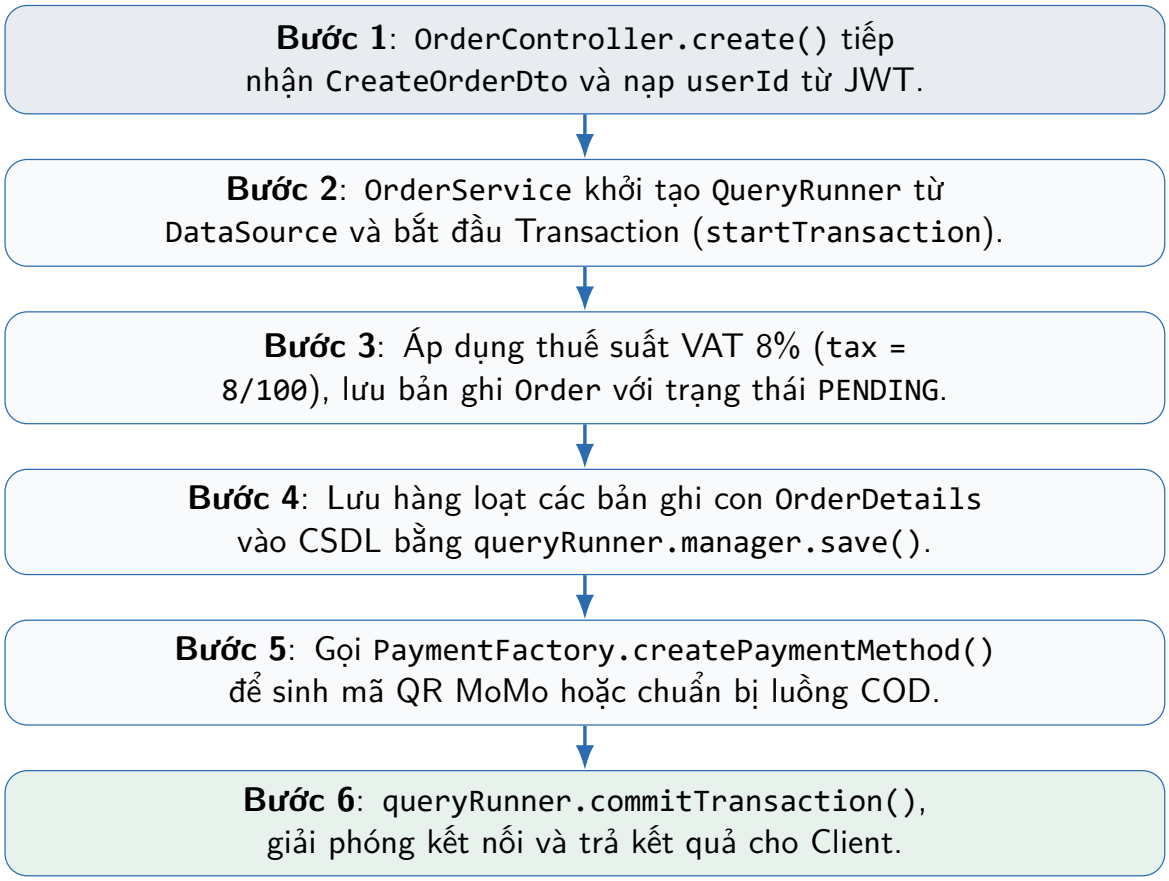
  // Quan hệ 1-Nhiều với OrderDetails
  @OneToMany(() => OrderDetails, (orderDetail) => orderDetail.order, {
    cascade: ['insert'], // Tự động chèn OrderDetails khi lưu Order
  })
  orderDetails: OrderDetails[];
}
```

18.3 Chu Trình Xử Lý Đơn Hàng & Quản Lý Giao Dịch Nguyên Tử ACID

Quy trình đặt hàng liên quan tới nhiều bảng dữ liệu và dịch vụ bên ngoài: khởi tạo đơn hàng, lưu chi tiết hàng hóa, trừ tồn kho sản phẩm, và gọi cổng thanh toán. Nếu một trong các bước này thất bại (ví dụ: máy chủ thanh toán bị timeout hoặc sản phẩm hết hàng), toàn bộ các thao tác trước đó phải được hoàn tác sạch sẽ (*Rollback*).

18.4 Giải Phẫu Mã Nguồn Xử Lý Giao Dịch Tại `OrderService`

Dịch vụ `OrderService` (`src/modules/order/order.service.ts`) phối hợp đồng thời `DataSource`, `Repository` và `PaymentFactory`:



Hình 18.1: Chu trình khởi tạo đơn hàng và giao dịch nguyên tử với QueryRunner

Mã nguồn quản lý giao dịch tạo đơn hàng trong OrderService

```
@Injectable()
export class OrderService {
  readonly tax: number;

  constructor(
    @InjectRepository(PaymentMethod)
    private readonly paymentMethodRepository: Repository<PaymentMethod>,
    @InjectRepository(Order)
    private readonly orderRepository: Repository<Order>,
    @Inject(PaymentFactory)
    private readonly paymentFactory: PaymentFactory,
    private datasource: DataSource,
  ) {
    this.tax = 8 / 100; // Cố định thuế suất GTGT 8%
  }

  async create(createOrderDto: CreateOrderDto, userId: number) {
    const paymentMethodName = await this.getPaymentMethodName(
      createOrderDto.payment_method_id,
    );

    // 1. Tạo QueryRunner từ TypeORM DataSource
    const queryRunner = this.datasource.createQueryRunner();
    await queryRunner.connect();
    await queryRunner.startTransaction();

    try {
      // 2. Khởi tạo đối tượng Order
      const createdOrder = queryRunner.manager.create(Order, {
        customer_name: createOrderDto.customer_name,
        customer_phone: createOrderDto.customer_phone,
        tax: this.tax,
        total: createOrderDto.price,
        user_id: userId,
        status: ORDER_STATUS.PENDING,
        currency: createOrderDto.currency,
        paymentMethod: { id: createOrderDto.payment_method_id },
      });

      // 3. Lưu Order vào CSDL trong phạm vi Transaction
      const order = await queryRunner.manager.save(Order, createdOrder);

      // 4. Kích hoạt Factory Pattern sinh cổng thanh toán (MoMo/COD)
      const paymentResponse = await this.paymentFactory.createPaymentMethod(
        paymentMethodName,
        createOrderDto,
        order.id,
```

```
      );

      // 5. Xác nhận thành công toàn bộ giao dịch
      await queryRunner.commitTransaction();
      return paymentResponse;
    } catch (err) {
      // 6. Hoàn tác sạch sẽ nếu có bất kỳ lỗi nào phát sinh
      await queryRunner.rollbackTransaction();
      throw err;
    } finally {
      // 7. Giải phóng kết nối trả về Connection Pool
      await queryRunner.release();
    }
  }
}
```

18.5 Phòng Ngự Xung Đột Đồng Thời (Race Conditions) & Trừ Tồn Kho

Trong các sự kiện khuyến mãi lớn (*Flash Sale*), hàng ngàn khách hàng có thể cùng nhấn nút *"Mua hàng"* trong cùng một phần trăm giây khi số lượng tồn kho chỉ còn 1 sản phẩm. Nếu không có cơ chế kiểm soát tương tranh, cả hai giao dịch đều đọc số lượng tồn kho là 1, cả hai đều cho phép đặt hàng và trừ tồn kho, dẫn tới hiện tượng bán âm kho (*Overselling*) vô cùng tai hại.

Dự án áp dụng cơ chế **Khóa Bi Quan (Pessimistic Write Lock)** tại tầng cơ sở dữ liệu:

```
// Chặn đứng các tiến trình khác đọc/sửa bản ghi này cho tới khi Transaction hiện tại kết thúc
const product = await queryRunner.manager.findOne(Product, {
  where: { id: productId },
  lock: { mode: 'pessimistic_write' }, // Sinh câu lệnh SQL: SELECT ... FOR UPDATE
});

if (product.stock < quantity) {
  throw new BadRequestException('Sản phẩm đã hết hàng trong kho');
}

product.stock -= quantity;
await queryRunner.manager.save(Product, product);
```

Nhờ mệnh đề `FOR UPDATE`, tiến trình thứ hai sẽ bị giữ lại chờ đợi cho tới khi tiến trình thứ nhất commit xong. Khi đến lượt tiến trình thứ hai đọc, số lượng tồn kho đã giảm về 0 và hệ thống sẽ ném ra ngoại lệ `BadRequestException` từ chối đơn hàng thứ hai một cách an toàn và nhất quán tuyệt đối.

18.6 Máy Trạng Thái Đơn Hàng (Order State Machine) & Capture Order

Đơn hàng trải qua các trạng thái tuần tự được định nghĩa trong `ORDER_STATUS`:

$$\texttt{PENDING} \xrightarrow{\text{Thanh toán MoMo thành công}} \texttt{PROCESSING} \xrightarrow{\text{Giao hàng thành công}} \texttt{COMPLETED} \tag{18.1}$$

Nếu người dùng hủy đơn hoặc giao dịch MoMo thất bại, trạng thái sẽ chuyển thành `CANCELLED` hoặc `FAILED`.

Tại `src/modules/order/order.controller.ts`, endpoint xác nhận thanh toán (*Payment Capture*) được công khai qua phương thức:

```
@Put('payments/:paymentOrderID/capture')
@UseGuards(AuthGuard)
async captureOrder(@Param('paymentOrderID') paymentOrderID: string) {
  const paymentMethodName = await this.orderService.getPaymentMethodNameByPaymentOrderId(paymen
  const orderID = await this.orderService.getOrderIDByPaymentOrderID(paymentOrderID);
  return await this.paymentFactory.createCaptureOrder(paymentMethodName, orderID, paymentOrderI
}
```

Cơ chế này phối hợp chặt chẽ với `PaymentFactory` để liên lạc với máy chủ đối tác, kiểm tra trạng thái khớp lệnh và hoàn tất giao dịch tài chính cho khách hàng.

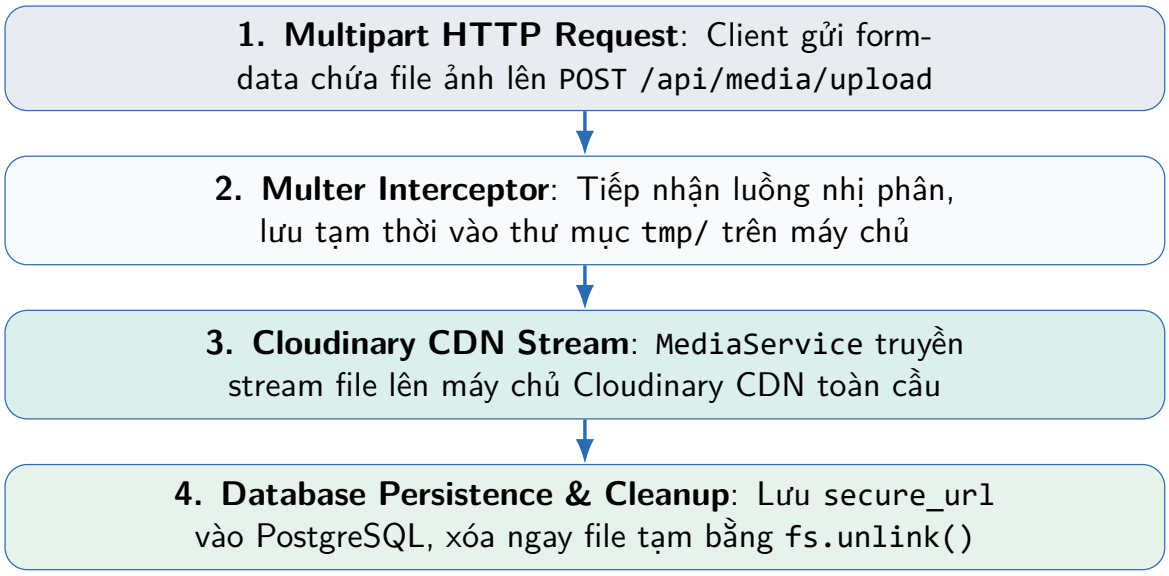
CHƯƠNG 19

Lưu Trữ Tệp Tin Đám Mây Cloudinary (Media Pipeline)

19.1 Kiến Trúc Đường Ống Tải Lên Đa Phương Tiện

Việc lưu trữ trực tiếp các tệp tin hình ảnh lớn trên ổ đĩa của máy chủ backend ứng dụng là một phản mẫu nghiêm trọng trong kiến trúc đám mây: khi mở rộng ra nhiều container Docker, các container sẽ không thể chia sẻ tệp tin cục bộ với nhau.

Dự án áp dụng mô hình **Media CDN Pipeline** kết hợp giữa NestJS, Multer và hạ tầng lưu trữ toàn cầu **Cloudinary**:



Hình 19.1: Quy trình tiếp nhận và tải lên tệp tin Media lên Cloudinary CDN

19.2 Giải Phẫu Mã Nguồn `MediaService` & Dọn Dẹp An Toàn

Hãy khảo sát cách `MediaService` (`src/modules/media/media.service.ts`) xử lý:

Mã nguồn MediaService với khối dọn dẹp finally

```
@Injectable()
export class MediaService {
  async uploadOne(file: Express.Multer.File, uploadFileDto: UploadFileDto): Promise<Media> {
    if (!file) {
      throw new HttpException({ status: 422, errors: { file: 'selectFile' } }, 422);
    }
    const { path } = file;
    try {
      // Đẩy tệp tin lên CDN
      const { url, public_id } = await this.cloudinaryClient.upload(path, {
        folder: uploadFileDto.folderName,
      });

      // Lưu trữ URL và publicId vào PostgreSQL
      const media = this.mediaRepository.create({ publicId: public_id, url: url });
      await this.mediaRepository.save(media);
      return media;
    } finally {
      // BẮT BUỘC: Dọn dẹp tệp tin tạm trên ổ đĩa máy chủ cục bộ
      unlink(path, (err) => {
        if (err) console.error('Lỗi khi xóa file tạm:', err);
      });
    }
  }
}
```

Tầm Quan Trọng Sống Còn Của Khối finally { fs.unlink }

Mỗi file ảnh người dùng tải lên sẽ được Multer lưu tạm thời vào ổ đĩa. Nếu không có lệnh `unlink(path)` đặt trong khối `finally`, hàng triệu tệp tin rác sẽ tích tụ theo thời gian, dẫn tới lỗi tràn ổ đĩa (*Disk Full - Error ENOSPC*), khiến hệ điều hành đóng băng toàn bộ tiến trình Node.js! Đặt trong `finally` bảo đảm dù quá trình upload lên Cloudinary thành công hay thất bại, tệp tin tạm cũng luôn được dọn dẹp sạch sẽ.

CHƯƠNG 20

Kỹ Thuật Xóa Mềm Trong Cơ Sở Dữ Liệu Doanh Nghiệp

20.1 Bản Chất Của Xóa Mềm (Soft Delete) vs Xóa Cứng (Hard Delete)

Trong các hệ thống quản lý doanh nghiệp, dữ liệu là tài sản vô giá. Hành động xóa cứng (*Hard Delete*) bằng câu lệnh SQL `DELETE FROM table WHERE id = ...` mang lại nhiều rủi ro nghiêm trọng:

1. **Phá hủy tính toàn vẹn tham chiếu**: Nếu bạn xóa một sản phẩm đã từng được bán trong quá khứ, các đơn hàng lịch sử liên quan đến sản phẩm đó sẽ bị lỗi khóa ngoại mồ côi (*Orphaned Records*).
2. **Mất mát dấu vết kiểm toán (Audit Trail)**: Doanh nghiệp không thể chứng minh hoặc phục hồi lại dữ liệu khi xảy ra tranh chấp pháp lý hoặc điều tra nội bộ.
3. **Không thể hoàn tác sai sót của người dùng**: Khi nhân viên vô tình bấm nhầm nút xóa, dữ liệu sẽ biến mất vĩnh viễn khỏi hệ thống.

Xóa mềm (Soft Delete) giải quyết triệt để vấn đề này: bản ghi không bao giờ bị xóa khỏi ổ đĩa vật lý, mà nó chỉ được đánh dấu một mốc thời gian xóa vào cột `deleted_at`.

20.2 Giải Phẫu Thực Thể `PostEntity` Với Decorator `@DeleteDateColumn()`

Hãy khảo sát mã nguồn thực tế của `PostEntity` tại `src/modules/post/post.entity.ts`:

Mã nguồn thực thể PostEntity trong src/modules/post/post.entity.ts

```
import {
  Entity,
  PrimaryGeneratedColumn,
  Column,
  CreateDateColumn,
  DeleteDateColumn,
```

```
} from 'typeorm';

@Entity('posts')
export class PostEntity {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column()
  title: string;

  @Column()
  category: string;

  @Column({ type: 'text', nullable: true })
  description: string | null;

  @CreateDateColumn()
  created_at: Date;

  @DeleteDateColumn()
  deleted_at: Date | null; // NULL: Bài viết hoạt động bình thường
}
```

20.3 Cơ Chế Tự Động Lọc SQL Phía Dưới Của TypeORM

Khi kỹ sư gọi phương thức xóa trong `PostService` (`src/modules/post/post.service.ts`):

Phương thức xóa mềm trong PostService

```
async remove(id: string) {
  const post = await this.postRepository.findOneBy({ id });
  if (!post) throw new NotFoundException('Không tìm thấy bài viết');
  return this.postRepository.softRemove(post);
}
```

TypeORM sẽ không phát sinh lệnh `DELETE`, mà nó phát sinh lệnh `UPDATE`:

SQL phát sinh khi gọi softRemove

```
UPDATE "posts"
SET "deleted_at" = CURRENT_TIMESTAMP
WHERE "id" = $1;
```

Và điều kỳ diệu nằm ở chỗ: trong tất cả các câu truy vấn tìm kiếm tiếp theo (`repository.find()`, `repository.findOne()`), TypeORM sẽ **tự động chèn thêm điều kiện**:

```
WHERE "deleted_at" IS NULL
```
(20.1)

Bản ghi bị xóa mềm sẽ tự động "biến mất" khỏi mắt người dùng và các API thông thường,

nhưng vẫn nằm nguyên vẹn trong CSDL để phục vụ việc khôi phục (`restore()`) hoặc thống kê tài chính khi cần thiết!

20.4 Khôi Phục Bản Ghi Xóa Mềm & Truy Vấn Thùng Rác (Trash Bin)

Để phục vụ giao diện quản trị viên (Admin Dashboard) xem danh sách các bài viết đã bị xóa tạm thời, TypeORM cung cấp cờ `withDeleted: true`:

Truy vấn bao gồm cả các bản ghi đã xóa mềm

```
// Lấy toàn bộ bài viết, bao gồm cả các bài viết đã bị xóa mềm (deleted_at IS NOT NULL)
const allPostsIncludingTrash = await this.postRepository.find({
  withDeleted: true,
});
```

Khi người dùng hoặc quản trị viên muốn khôi phục một bài viết bị xóa nhầm, kỹ sư gọi phương thức `restore()`:

Khôi phục bản ghi đã xóa mềm bằng restore

```
async restorePost(id: string): Promise<void> {
  await this.postRepository.restore(id);
}
```

Lệnh này kích hoạt câu lệnh SQL ngược lại:

```
UPDATE "posts" SET "deleted_at" = NULL WHERE "id" = $1;
```
(20.2)

Bản ghi ngay lập tức quay trở lại trạng thái hoạt động bình thường mà không hề bị mất mát bất kỳ dữ liệu liên kết nào!

20.5 Xung Đột Ràng Buộc Duy Nhất (Unique Constraint) & Giải Pháp Partial Index

Khi một bảng áp dụng kỹ thuật Soft Delete và có một cột yêu cầu tính duy nhất (ví dụ: `slug` của bài viết hoặc `email` của người dùng), một xung đột logic kinh điển sẽ phát sinh:

1. Bài viết A có `slug = 'nestjs-architecture'` bị xóa mềm (`deleted_at = NOW()`).
2. Tác giả tạo một bài viết mới B với cùng `slug = 'nestjs-architecture'`.
3. Nếu bảng sử dụng ràng buộc duy nhất thông thường (`UNIQUE(slug)`), PostgreSQL sẽ từ chối câu lệnh chèn mới vì giá trị `'nestjs-architecture'` đã tồn tại trong dòng cũ của bài viết A!

Để giải quyết vấn đề này, Kỹ sư Cấp cao sử dụng **Partial Unique Index (Chỉ mục duy nhất bộ phận)** trong PostgreSQL:

Tạo Partial Index cho Soft Delete trong PostgreSQL

```
CREATE UNIQUE INDEX "idx_posts_slug_active"
ON "posts" ("slug")
WHERE "deleted_at" IS NULL;
```

Chỉ mục này chỉ áp đặt ràng buộc duy nhất trên tập hợp con các dòng đang hoạt động (`deleted_at IS NULL`). Nhờ đó, người dùng hoàn toàn có thể tái sử dụng slug của các bài viết đã bị xóa mềm mà không gặp bất kỳ lỗi xung đột nào!

20.6 Phân Định Kiến Trúc: Controller Công Khai vs Controller Quản Trị

Hãy so sánh sự phân tách kiến trúc giữa hai Controller xử lý bài viết trong hệ thống:

- **`PostController` (`src/modules/post/post.controller.ts`)**: Đặt tại tiền tố `/v1/post`. Phục vụ khách hàng đọc bài viết công khai (`findAll`, `findOne`) và cho phép tác giả cập nhật hoặc xóa bài viết của chính họ.
- **`AdminController` (`src/modules/admin/admin.controller.ts`)**: Đặt tại tiền tố `/admin`. Bắt buộc phải có Decorator `@Roles(USER_ROLE.ADMIN)` và `@UseGuards(AuthGuard, RolesGuard)`. Quản trị viên có toàn quyền kiểm duyệt, xóa vĩnh viễn hoặc khôi phục dữ liệu của mọi người dùng trong hệ thống.

PHẦN VI

Kiểm Thử Tự Động, DevOps & Tiêu Chuẩn Kỹ Thuật (Engineering Standards)

CHƯƠNG 21

Chiến Lược Kiểm Thử Tự Động Toàn Diện Với Jest

21.1 Kim Tự Tháp Kiểm Thử Phần Mềm (The Test Pyramid)

Trong kỹ nghệ phần mềm hiện đại, việc bảo đảm chất lượng hệ thống tuân theo mô hình Kim tự tháp Kiểm thử gồm 3 tầng:

1. **Unit Tests (Kiểm thử đơn vị - Đáy kim tự tháp)**: Chiếm số lượng lớn nhất (70% - 80%). Kiểm tra các hàm logic nghiệp vụ cô lập trong tầng Service. Tốc độ thực thi siêu tốc (vài mili-giây/test), chi phí bảo trì thấp.
2. **Integration Tests (Kiểm thử tích hợp - Thân kim tự tháp)**: Kiểm tra sự phối hợp giữa nhiều thành phần (ví dụ: Controller + Pipes + Guards hoặc Service + Database thật).
3. **End-to-End Tests (E2E Tests - Đỉnh kim tự tháp)**: Mô phỏng hành vi người dùng thật từ ngoài Internet gửi HTTP request qua cổng mạng và kiểm tra phản hồi JSON cuối cùng.

21.2 Triết Lý Mock Repository Trong Kiểm Thử Đơn Vị

Kiểm thử đơn vị tầng Service không được phép phụ thuộc vào cơ sở dữ liệu thật vì:

- Việc kết nối CSDL làm tốc độ chạy test chậm đi hàng trăm lần.
- Dữ liệu trong CSDL có thể bị biến đổi qua các lần chạy, dẫn tới hiện tượng kiểm thử chập chờn (*Flaky Tests*).

Bằng cách sử dụng `@nestjs/testing` kết hợp với token `getRepositoryToken(Entity)`, chúng ta có thể tiêm một **Mock Repository** chứa các hàm giả lập của Jest (`jest.fn()`):

Mẫu thiết lập TestingModule và Mock Repository trong tasks.service.spec.ts

```
const mockTasksRepository = {
  create: jest.fn(),
```

```
    save: jest.fn(),
    find: jest.fn(),
    findOneBy: jest.fn(),
    preload: jest.fn(),
    delete: jest.fn(),
  };

  const module: TestingModule = await Test.createTestingModule({
    providers: [
      TasksService,
      {
        provide: getRepositoryToken(TaskEntity),
        useValue: mockTasksRepository,
      },
    ],
  }).compile();

  service = module.get<TasksService>(TasksService);
```

21.3 Kiểm Thử Các Kịch Bản Thành Công & Ngoại Lệ (Happy vs Error Paths)

Hãy xem xét cách viết ca kiểm thử kiểm tra xem Service có ném đúng ngoại lệ `NotFoundException` khi không tìm thấy bản ghi hay không:

Kiểm thử ngoại lệ NotFoundException với Jest

```
it('nên ném NotFoundException khi không tìm thấy công việc cần xóa', async () => {
    // Giả lập kết quả delete trả về affected = 0 (không có dòng nào bị ảnh hưởng)
    mockTasksRepository.delete.mockResolvedValue({ affected: 0 });

    // Kiểm tra xem Promise có bị reject với đúng kiểu ngoại lệ không
    await expect(service.remove('uuid-khong-ton-tai')).rejects.toThrow(
      NotFoundException,
    );
  });
```

Chạy lệnh `npm run test:cov` sẽ sinh ra báo cáo tỷ lệ bao phủ mã nguồn (*Code Coverage Report*) chi tiết tới từng dòng lệnh và nhánh rẽ (*branch coverage*).

CHƯƠNG 22

Tài Liệu Hóa Tự Động API Với Swagger / OpenAPI

22.1 Tiêu Chuẩn OpenAPI Specification 3.0 & Triết Lý API-First

Tài liệu hóa API là ”bản hợp đồng giao tiếp” sống còn giữa đội ngũ phát triển Backend, Frontend (Web/Mobile), đội ngũ Đảm bảo chất lượng (QA/QC), và các đối tác tích hợp hệ thống bên ngoài. Trong quy trình phát triển truyền thống, việc viết tài liệu thủ công bằng file Word, Excel hay Markdown thường dẫn tới tình trạng tài liệu bị lỗi thời ngay sau khi mã nguồn thay đổi, gây lãng phí hàng trăm giờ làm việc để gỡ lỗi sai lệch giao tiếp.

NestJS giải quyết triệt để vấn đề này bằng cách tự động trích xuất metadata từ mã nguồn TypeScript và các lớp DTO để sinh ra tài liệu chuẩn **OpenAPI Specification (OAS 3.0)**. Tài liệu này không chỉ hiển thị giao diện tương tác trực quan cho con người mà còn đóng vai trò là dữ liệu đầu vào cho các công cụ tự động hóa:

1. **Tự động sinh mã nguồn máy khách (Client SDK Generation)**: Sử dụng công cụ `openapi-generator` để tự động sinh ra toàn bộ code gọi API cho Frontend (TypeScript Axios client, Android Kotlin Retrofit, iOS Swift), loại bỏ 100% việc viết thủ công các hàm fetch API.
2. **Kiểm thử hợp đồng tự động (Contract Testing)**: Tự động thẩm định xem phản hồi thực tế của server có đúng với cấu trúc Schema đã công bố hay không.

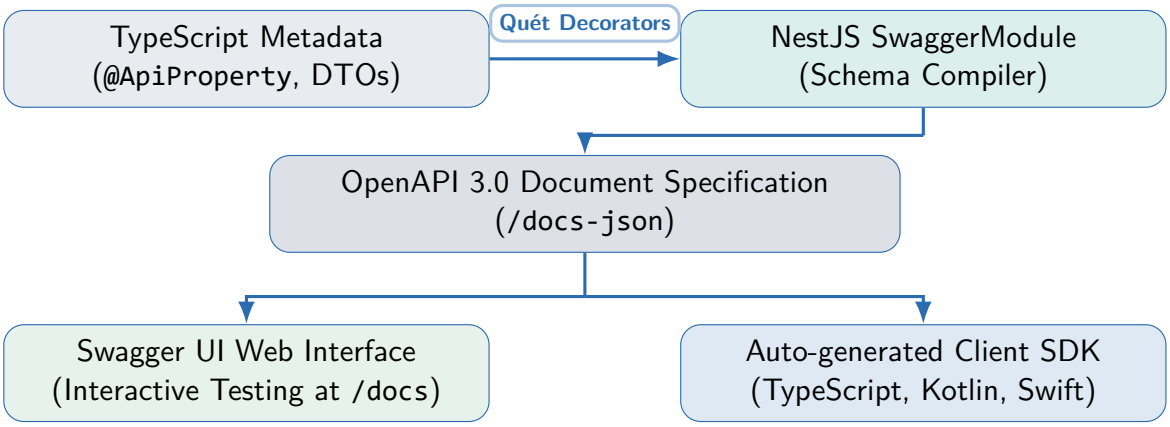
22.2 Giải Phẫu Tệp Khởi Tạo Trung Tâm `src/setup-swagger.ts`

Tệp tin `src/setup-swagger.ts` cấu hình toàn bộ hệ thống Swagger:

Mã nguồn setupSwagger trong src/setup-swagger.ts

```
import type { INestApplication } from '@nestjs/common';
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';

export function setupSwagger(app: INestApplication): void {
  const documentBuilder = new DocumentBuilder()
```



Hình 22.1: Đường ống tự động hóa tài liệu OpenAPI và sinh mã nguồn Client SDK

```
    .setTitle('NestJS Boilerplate API')
    .setDescription('Tài liệu API hệ thống backend doanh nghiệp xây dựng với NestJS')
    .addBearerAuth(
      {
        type: 'http',
        scheme: 'bearer',
        bearerFormat: 'JWT',
        name: 'token',
        description: 'Nhập JWT Access Token để xác thực người dùng',
        in: 'header',
      },
      'token', // Khóa định danh liên kết với @ApiBearerAuth('token')
    );

  if (process.env.API_VERSION) {
    documentBuilder.setVersion(process.env.API_VERSION);
  }

  // 1. deepScanRoutes: Quét toàn diện các Controller trong các Module phân cấp lồng nhau
  const document = SwaggerModule.createDocument(app, documentBuilder.build(), {
    deepScanRoutes: true,
  });

  // 2. Xuất bản giao diện kiểm thử tương tác tại đường dẫn /docs
  SwaggerModule.setup('docs', app, document, {
    swaggerOptions: {
      explore: true,
      deepLinking: true,        // Hỗ trợ chia sẻ link trực tiếp tới từng endpoint
      persistAuthorization: true, // LƯU TOKEN TRONG BỘ NHỚ: Không bị mất khi F5 reload!
    },
  });
}
```

Tầm Quan Trọng Của Tùy Chọn persistAuthorization

Trong môi trường phát triển thực tế, khi lập trình viên kiểm thử các endpoint yêu cầu quyền đăng nhập (*Protected Routes*), họ phải nhấn nút **Authorize** và dán mã JWT Bearer Token. Nếu không bật `persistAuthorization: true`, mỗi lần người dùng bấm tải lại trang trình duyệt (F5), Swagger UI sẽ bị xóa sạch token, buộc lập trình viên phải thực hiện thao tác đăng nhập và dán token lại từ đầu. Tùy chọn này giúp lưu trữ token trong LocalStorage của trình duyệt, tiết kiệm đáng kể thời gian thử nghiệm cho toàn đội ngũ!

22.3 Cẩm Nang Tra Cứu Bộ Decorators OpenAPI Trong Controller & DTO

Để tài liệu API đạt chất lượng chuyên nghiệp, kỹ sư sử dụng phối hợp bộ Decorator chuẩn:

- `@ApiTags('Tasks')`: Gom nhóm trực quan các endpoint theo phân hệ nghiệp vụ trên giao diện Swagger UI.
- `@ApiOperation({ summary: '...', description: '...' })`: Cung cấp tóm tắt ngắn gọn và mô tả chi tiết chức năng nghiệp vụ của endpoint.
- `@ApiResponse({ status: 201, type: TaskEntity })`: Định nghĩa schema của đối tượng dữ liệu trả về kèm mã trạng thái HTTP chuẩn xác.
- `@ApiBearerAuth('token')`: Hiển thị biểu tượng ổ khóa bảo mật bên cạnh endpoint trên giao diện, báo hiệu endpoint bắt buộc phải có JWT Token trong header.
- `@ApiProperty({ example: 'Học NestJS' })`: Khai báo thuộc tính bắt buộc trong DTO kèm giá trị ví dụ minh họa trực quan.
- `@ApiPropertyOptional({ default: false })`: Khai báo thuộc tính tùy chọn kèm giá trị mặc định được hệ thống tự động gán.
- `@ApiParam({ name: 'id', description: 'Mã định danh duy nhất UUID v4' })`: Mô tả chi tiết tham số nằm trên đường dẫn URL.
- `@ApiQuery({ name: 'page', required: false, example: 1 })`: Mô tả các tham số tìm kiếm hoặc phân trang trên Query String.

22.4 Bảo Vệ Tài Liệu API Trên Môi Trường Sản Xuất

Mặc dù Swagger rất hữu ích trong giai đoạn phát triển và kiểm thử, nhưng việc công khai toàn bộ tài liệu API chi tiết trên môi trường sản xuất (*Production*) có thể trở thành "tấm bản đồ chỉ đường" cho các tin tặc tìm kiếm lỗ hổng bảo mật.

Dự án áp dụng hai chiến lược bảo vệ:

1. **Vô hiệu hóa hoàn toàn Swagger trên Production**: Kiểm tra biến môi trường `NODE_ENV`. Nếu là `'production'`, hàm `setupSwagger(app)` sẽ được bỏ qua không kích hoạt.
2. **Bảo vệ qua Basic Authentication**: Cài đặt gói `express-basic-auth` để yêu cầu nhập tài khoản/mật khẩu quản trị nội bộ trước khi cho phép truy cập vào đường dẫn `/docs`.

CHƯƠNG 23

Container Hóa Hệ Thống Với Docker & Compose

23.1 Nguyên Lý Ảo Hóa Container & Multi-Stage Build

Để loại bỏ hoàn toàn câu nói kinh điển của lập trình viên: *"Ủa sao chạy trên máy em bình thường mà lên server lại lỗi?"*, hệ thống được đóng gói hoàn chỉnh bằng Docker Container.

Tuy nhiên, nếu sử dụng một Dockerfile thông thường, kích thước ảnh (Image Size) của ứng dụng Node.js/TypeScript có thể phình to lên tới **1.2 GB** do chứa toàn bộ mã nguồn TypeScript, trình biên dịch `tsc`, và thư mục `node_modules` chứa các gói phát triển (*devDependencies*).

Dự án áp dụng kỹ thuật **Multi-Stage Build** tại tệp `Dockerfile`:

Mã nguồn Dockerfile Multi-stage tối ưu dung lượng

```
# ================= GIAI ĐOẠN 1: BUILDER =================
FROM node:16-alpine as build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY --chown=node:node . .
RUN npm run build

# Xóa devDependencies, chỉ cài đặt lại các thư viện production tối thiểu
RUN npm ci --only=production && npm cache clean --force
USER node

# ================= GIAI ĐOẠN 2: RUNNER ==================
FROM node:16-alpine
WORKDIR /app

# Chỉ sao chép kết quả đã biên dịch và node_modules tối ưu từ Stage 1
COPY --from=build --chown=node:node /app/package*.json ./
COPY --from=build --chown=node:node /app/node_modules ./node_modules
COPY --from=build --chown=node:node /app/dist ./dist

# Chạy dưới quyền người dùng node (không dùng root) để bảo đảm an toàn
```

```
USER node
CMD ["node", "dist/main.js"]
```

Hiệu Quả Tối Ưu Của Multi-Stage Build

Nhờ kỹ thuật Multi-Stage Build:

1. Kích thước ảnh Docker giảm từ **1.2 GB xuống còn dưới 150 MB** (giảm hơn 85% dung lượng).
2. Thời gian tải và đẩy ảnh (Pull/Push) trong đường ống CI/CD nhanh hơn gấp 8 lần.
3. Toàn bộ mã nguồn gốc TypeScript và các công cụ biên dịch không hề tồn tại trong container sản xuất, triệt tiêu nguy cơ bị khai thác mã độc thông qua các lỗ hổng của devDependencies.

23.2 Lưu Ý Về Alpine Linux & Native Addons (Argon2)

Bản phân phối `alpine` sử dụng thư viện C chuẩn **musl libc** thay vì **glibc**. Thuật toán mã hóa Argon2 chứa các đoạn mã C++ bản địa (*Native Addons*). Trong tệp tin cấu hình `docker-compose.yml`, khi khởi chạy dịch vụ lần đầu, cần lưu ý lệnh:

```
command: sh -c "npm rebuild argon2 --build-from-source && npm run typeorm:run-migration"
```

Lệnh này bảo đảm module Argon2 được biên dịch lại tương thích 100% với nhân Linux của Alpine trước khi chạy migration và khởi động máy chủ.

23.3 Điều Phối Cụm Dịch Vụ Với Docker Compose

Tệp `docker-compose.yml` điều phối đồng thời các dịch vụ cần thiết:

- **postgres-db**: Cơ sở dữ liệu PostgreSQL 16 Alpine, cấu hình volume lưu trữ dữ liệu bền vững (*persistent data*).
- **base**: Dịch vụ backend NestJS kết nối vào mạng nội bộ riêng biệt `app-network`.
- **init_db**: Container chạy một lần (*one-off container*) tự động thực thi các migration và script seed dữ liệu mẫu khi hệ thống khởi động lần đầu.

CHƯƠNG 24

Quy Chuẩn Kỹ Thuật Mã Nguồn & Git Hooks

24.1 Mô Hình Phân Nhánh Git Flow Doanh Nghiệp

Trong các dự án kỹ thuật quy mô lớn với hàng chục kỹ sư cùng tham gia phát triển, việc đẩy mã nguồn trực tiếp lên nhánh chính hoặc phân nhánh tùy tiện sẽ dẫn tới xung đột hợp nhất (*Merge Conflicts*) trầm trọng và rủi ro đưa mã nguồn lỗi lên môi trường sản xuất. Dự án áp dụng mô hình phân nhánh chuẩn mực **Git Flow**:

- **main**: Nhánh ổn định tuyệt đối, phản ánh trạng thái mã nguồn đang chạy trên môi trường Production. Chỉ được phép hợp nhất từ nhánh `release` hoặc `hotfix` sau khi đã vượt qua toàn bộ các bài kiểm thử tự động và được gắn thẻ phiên bản ngữ nghĩa (*Semantic Versioning*, ví dụ `v1.0.0`).
- **develop**: Nhánh tích hợp mã nguồn chính của toàn đội ngũ, phản ánh trạng thái môi trường Staging/UAT. Toàn bộ các tính năng mới đều phải được kiểm thử tích hợp trên nhánh này trước khi đóng gói phát hành.
- **feature/<task-id>-<name>**: Các nhánh phát triển tính năng mới tách ra từ `develop`. Nhánh tính năng chỉ được hợp nhất ngược lại `develop` thông qua Pull Request (PR) có ít nhất 2 lượt phê duyệt (*Code Review Approvals*) từ các Kỹ sư Cấp cao.
- **release/<version>**: Nhánh chuẩn bị đóng gói phát hành. Tại đây, tính năng mới bị đóng băng (*Feature Freeze*), toàn đội chỉ tập trung dọn dẹp lỗi nhỏ và hoàn thiện tài liệu trước khi đưa lên `main` và `develop`.
- **hotfix/<incident-id>**: Nhánh sửa lỗi khẩn cấp tách trực tiếp từ `main` khi có sự cố phát sinh trên môi trường sản xuất, sau đó được hợp nhất song song vào cả `main` và `develop`.

24.2 Quy Chuẩn Thông Điệp Commit (Conventional Commits)

Để lược sử Git luôn trong sáng, dễ truy vết nguyên nhân lỗi và hỗ trợ tự động sinh tập tin nhật ký thay đổi (*Changelog*) khi phát hành phiên bản mới, mọi lập trình viên bắt buộc

phải tuân thủ chuẩn **Conventional Commits**:

```
<type>(<scope>): <short description>
```
(24.1)

Tệp tin cấu hình `commitlint.config.js` trong mã nguồn dự án định nghĩa nghiêm ngặt 13 tiền tố (*type*) hợp lệ:

Bảng 24.1: Danh mục 13 tiền tố Commit hợp lệ được định nghĩa trong `commitlint.config.js`

Tiền tố (Type)	Ý nghĩa kỹ thuật	Ví dụ minh họa chuẩn mực
`feat`	Thêm tính năng mới	`feat(tasks): add filter by isCompleted`
`fix`	Sửa lỗi hệ thống	`fix(auth): handle expired google idToken`
`improve`	Cải tiến tính năng có sẵn	`improve(order): optimize query for order history`
`perf`	Tối ưu hóa hiệu năng	`perf(database): add index for tasks category`
`refactor`	Tái cấu trúc mã nguồn	`refactor(media): extract cloudinary logic to service`
`docs`	Cập nhật tài liệu, Swagger	`docs(api): update ApiResponse for tasks endpoint`
`chore`	Cập nhật phụ thuộc, cấu hình	`chore(deps): update typeorm to 0.3.11`
`reformat`	Chỉnh sửa định dạng, thụt lề	`reformat(user): align prettier code formatting`
`test`	Viết thêm/chỉnh sửa unit test	`test(tasks): add 5 unit test cases for tasks service`
`revert`	Hoàn tác lại commit cũ	`revert(post): rollback change on delete date column`
`ci`	Cập nhật kịch bản CI/CD	`ci(github): add automated jest test workflow`
`build`	Thay đổi hệ thống build, Docker	`build(docker): optimize multi-stage alpine image`
`db`	Thay đổi liên quan tới Migration	`db(migration): create tasks table with uuid primary key`

Các Quy Tắc Bắt Buộc Trong commitlint.config.js

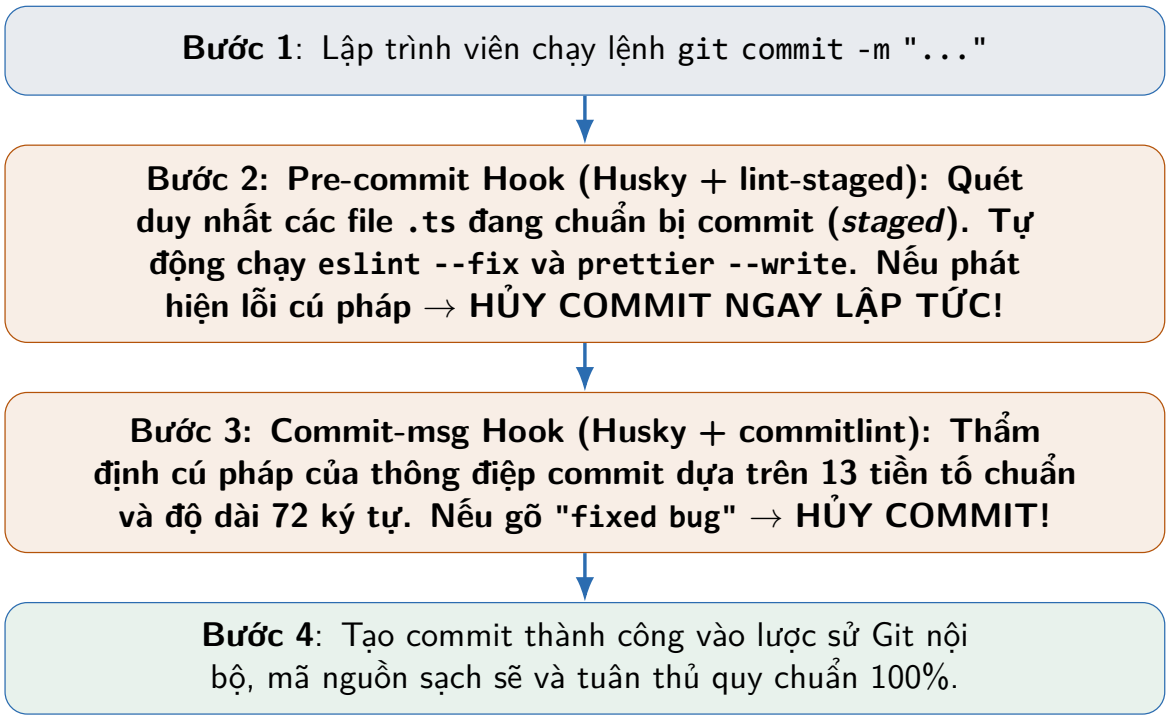
Ngoài tiền tố, cấu hình kiểm tra tự động áp đặt 4 quy tắc cú pháp bất di bất dịch:

1. `type-case: ['always', 'lower-case']`: Tiền tố bắt buộc phải viết chữ thường (ví dụ: `feat`, không chấp nhận `Feat` hay `FEAT`).
2. `header-max-length: ['always', 72]`: Dòng tiêu đề đầu tiên không được phép vượt quá 72 ký tự để bảo đảm hiển thị đẹp mắt trên giao diện Git CLI và GitHub.
3. `subject-full-stop: ['never', '.']`: Tiêu đề **tuyệt đối không có dấu chấm ở cuối câu** (ví dụ: viết `fix(auth): fix token`, không viết `fix(auth): fix token.`).

4. **Khai báo Breaking Changes**: Khi thay đổi gây phá vỡ tương thích ngược (ví dụ thay đổi cấu trúc URL API hoặc xóa một trường dữ liệu trong response), lập trình viên phải đặt dấu cảm thán `!` ngay trước dấu hai chấm: `feat(api)!: remove deprecated v1 user endpoint`.

24.3 Tự Động Hóa Kiểm Soát Chất Lượng Với Husky & Lint-staged

Thay vì phụ thuộc vào ý thức tự giác của lập trình viên hoặc chỉ phát hiện lỗi khi mã nguồn đã được đẩy lên máy chủ CI/CD trên đám mây, dự án thiết lập **Trạm gác cổng chất lượng mã nguồn tự động (Developer Quality Gatekeeper)** ngay tại máy trạm thông qua **Husky v8** và **lint-staged**:



Hình 24.1: Đường ống kiểm soát chất lượng mã nguồn tự động tại máy cục bộ với Git Hooks

Cấu Hình Tối Ưu Của lint-staged Trong .lintstagedrc.json

Trong tệp tin `.lintstagedrc.json`, cấu hình chỉ vỏn vẹn 3 dòng:

```
{
  "*.ts": ["npm run lint", "npm run format"]
}
```

Ưu điểm vượt trội của **lint-staged** so với việc chạy lint toàn bộ dự án: nếu dự án có 5,000 tệp tin, việc chạy `npm run lint` toàn bộ sẽ mất từ 30 giây đến 1 phút, làm nản lòng lập trình viên. `lint-staged` chỉ lọc ra đúng 2 hoặc 3 tệp tin TypeScript mà lập trình viên vừa thay đổi trong commit hiện tại, hoàn thành việc sửa lỗi thụt lề và cú

pháp chỉ trong vòng **1.5 giây**!

24.4 Lợi Ích Kinh Tế & Vận Hành Của Trạm Kiểm Soát Cục Bộ

Việc ngăn chặn lỗi ngay từ bàn phím của lập trình viên đem lại lợi ích kinh tế to lớn cho doanh nghiệp:

1. **Tiết kiệm chi phí điện toán đám mây**: Mỗi lần kích hoạt đường ống CI/CD trên GitHub Actions hay GitLab CI đều tiêu tốn số phút máy chủ (*Runner minutes*). Chặn đứng các lỗi cú pháp nhỏ ngay trên máy trạm giúp giảm thiểu 40% số lượt build vô ích trên CI.
2. **Loại bỏ hiện tượng "Phá vỡ nhánh chính" (Broken Build)**: Nhánh `develop` và `main` luôn giữ được trạng thái hoàn hảo, không bao giờ bị gián đoạn công việc của các thành viên khác trong nhóm do một commit cẩu thả.

Lời Kết: Con Đường Trở Thành Kỹ Sư Phần Mềm Cấp Cao

Cuốn sách này đã đưa bạn qua toàn bộ các trụ cột kỹ thuật của một hệ sinh thái backend doanh nghiệp hoàn chỉnh. Từ một bài toán cụ thể về CRUD cho tới kiến trúc IoC, Security với Argon2, Factory Pattern trong MoMo Gateway và hạ tầng DevOps, sự khác biệt của một Kỹ sư Cấp cao nằm ở **tư duy kiến trúc hệ thống**: hiểu rõ gốc rễ của từng quyết định thiết kế và bảo đảm hệ thống luôn an toàn, tin cậy và có khả năng mở rộng bền vững.

PHẦN VII

Phụ Lục Kỹ Thuật Chuyên Sâu (Technical Appendix)

CHƯƠNG A

Toàn Cảnh Hệ Thống Thực Nghiệm NestJS Boilerplate

A.1 Kiến Trúc Tổng Thể & Cấu Trúc Thư Mục Dự Án

Toàn bộ mã nguồn dự án được tổ chức chặt chẽ theo mô hình **Enterprise Modular Architecture**, phân chia rạch ròi giữa tầng hạ tầng cốt lõi (*Core Infrastructure*), tầng dùng chung (*Shared Layer*), và các phân hệ nghiệp vụ độc lập (*Feature Modules*):

Cây thư mục mã nguồn dự án NestJS Boilerplate

```
c:/Users/LAPTOP/nest/
├── src/
│   ├── app.controller.ts, app.module.ts, app.service.ts, main.ts, setup-swagger.ts
│   ├── config/              # Cấu hình môi trường, Cloudinary, MoMo, NamingStrategy
│   ├── core/                # Hạ tầng cốt lõi độc lập nghiệp vụ
│   │   ├── filters/         # 8 Bộ lọc ngoại lệ chuyên biệt (Bad Request, 404, 500...)
│   │   ├── interceptors/    # Logging Interceptor, Transform Interceptor
│   │   ├── loggers/         # Winston Logger Factory (Local vs Production Daily Rotate)
│   │   ├── middleware/      # Request Logger Middleware ghi nhận thời gian thực thi
│   │   └── pagination/      # Bộ ba chuẩn phân trang: PageOptions, PageMeta, PageDto
│   ├── database/typeorm/    # Tầng lưu trữ cơ sở dữ liệu
│   │   ├── config.ts        # DataSource cấu hình trung tâm (SSL, Pools, Entities)
│   │   ├── entities/        # Danh mục 9 Entities chính (User, Order, Post, Task...)
│   │   ├── migrations/      # Tập lệnh di trú cấu hình cơ sở dữ liệu theo thời gian
│   │   └── seeds/           # Động cơ nạp dữ liệu mẫu Hygen (Role, User, Payment...)
│   ├── i18n/                # Từ điển đa ngôn ngữ (en/errors.json, vi/errors.json)
│   ├── modules/             # Các phân hệ nghiệp vụ độc lập (Feature Modules)
│   │   ├── admin/           # Phân hệ quản trị viên cấp cao & duyệt bài viết
│   │   ├── auth/            # Phân hệ xác thực đa kênh (JWT, Local, Google OAuth2)
│   │   ├── media/           # Phân hệ xử lý và tải tệp tin lên CDN Cloudinary
│   │   ├── order/           # Phân hệ đơn hàng, giao dịch ACID & Payment Factory
│   │   ├── post/            # Phân hệ bài viết & kỹ thuật xóa mềm Soft Delete
│   │   ├── product/         # Phân hệ danh mục sản phẩm thương mại
│   │   ├── tasks/           # Phân hệ quản lý công việc cá nhân (Task2 Blueprint)
│   │   └── user/            # Phân hệ quản lý hồ sơ người dùng & băm Argon2
│   └── shared/              # Thư viện dùng chung toàn hệ thống
│       ├── abstract.entity.ts # Lớp thực thể cơ sở (id, createdAt, updatedAt, toDto)
```

```
│       ├── decorators/      # Custom Decorators (@VirtualColumn, @CurrentUser...)
│       ├── enum/            # Các bảng kiểu liệt kê hệ thống (Role, OrderStatus...)
│       └── services/        # Dịch vụ ErrorService dịch động lỗi đa ngôn ngữ
├── Dockerfile, docker-compose.yml # Container hóa Multi-stage và điều phối cụm dịch vụ
├── Task2.md                 # Tài liệu đặc tả bài tập mẫu
└── TaskThread.tex, TaskThread.pdf, TaskThread.md # Toàn bộ giáo trình kỹ thuật
```

A.2 Bảng Khảo Sát Chi Tiết 8 Feature Modules Trong Hệ Thống

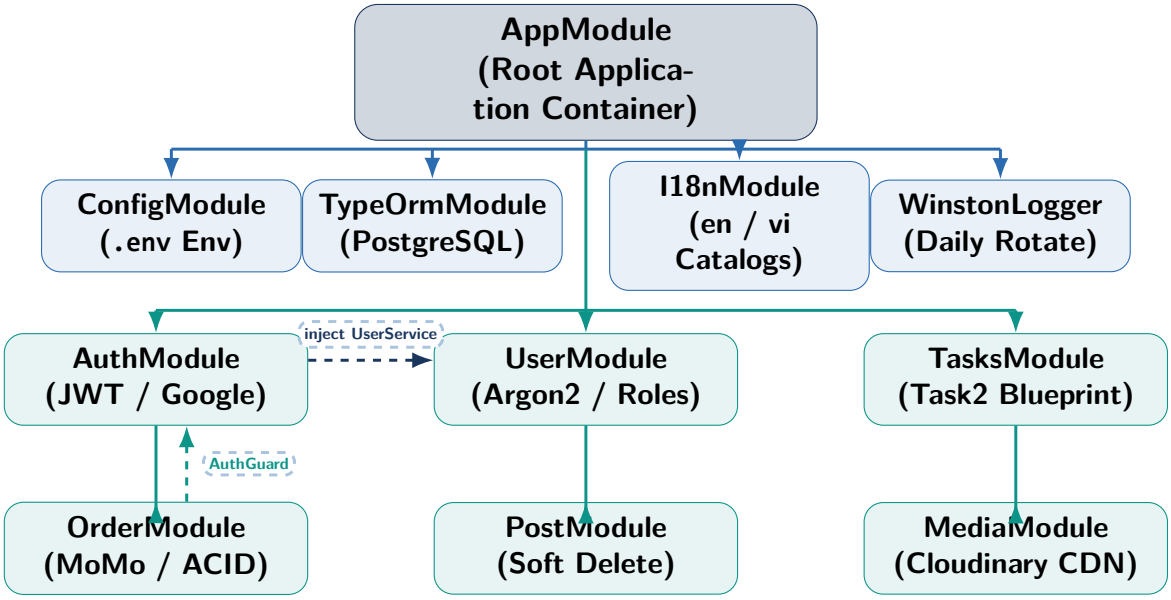
Hệ thống vận hành dựa trên 8 Feature Modules chuyên biệt, mỗi module đảm nhận một phạm vi nghiệp vụ khép kín (*Bounded Context*):

Bảng A.1: Bảng tổng hợp vai trò kiến trúc và trách nhiệm của 8 Feature Modules

Tên Module	Đường dẫn thư mục	Trách nhiệm kiến trúc & Thành phần cốt lõi
`AuthModule`	`src/modules/auth`	Quản lý định danh và cấp phát token. Chứa `JwtStrategy`, `LocalStrategy`, `AuthGoogleLoginDto`, cơ chế thẩm định mật khẩu Argon2 và gán vai trò `MEMBER`.
`UserModule`	`src/modules/user`	Quản lý vòng đời tài khoản người dùng, ánh xạ thực thể `User` với `Role`, tự động kích hoạt băm mật khẩu qua hook `@BeforeInsert()`.
`OrderModule`	`src/modules/order`	Điều phối quy trình mua hàng. Quản lý quan hệ Master-Detail giữa `Order` và `OrderDetail`, áp dụng Factory Pattern (`PaymentFactory`) điều phối thanh toán MoMo và COD, quản lý giao dịch nguyên tử với `QueryRunner`.
`TasksModule`	`src/modules/tasks`	Phân hệ quản lý công việc cá nhân giải quyết toàn diện bài toán **Task2.md**. Hỗ trợ 5 API CRUD chuẩn RESTful, khóa chính UUID v4, lọc theo trạng thái/danh mục và thẩm định lỗi 400/404.
`PostModule`	`src/modules/post`	Quản lý bài viết truyền thông. Hiện thực hóa kỹ thuật xóa mềm (*Soft Delete*) với `@DeleteDateColumn()` và phương thức `softRemove()`.
`MediaModule`	`src/modules/media`	Tiếp nhận tệp tin tải lên thông qua Multer Interceptor, đẩy tệp tin lên hạ tầng lưu trữ đám mây Cloudinary và dọn dẹp bộ đệm ổ đĩa an toàn trong khối `finally`.
`ProductModule`	`src/modules/product`	Quản trị danh mục hàng hóa, giá bán, số lượng tồn kho, kiểm soát tương tranh khóa bi quan khi trừ kho.
`AdminModule`	`src/modules/admin`	Cổng quản trị viên cấp cao. Sử dụng `RolesGuard` với quyền hạn `ADMIN` để thực hiện các nghiệp vụ phê duyệt nội dung và quản lý người dùng toàn hệ thống.

A.3 Lược Đồ Bản Đồ Phụ Thuộc Module Toàn Hệ Thống

Mối liên kết giữa các Module được biểu diễn trực quan qua đồ thị phụ thuộc dưới đây:



Hình A.1: Bản đồ phụ thuộc các phân hệ Module xung quanh AppModule trung tâm

CHƯƠNG B

Giải Pháp Kỹ Thuật Toàn Diện Cho Module Tasks (Task2.md)

B.1 Bối Cảnh Nghiệp Vụ & Đặc Tả Bài Toán

Bài toán **Task2.md** yêu cầu hiện thực hóa hoàn chỉnh module `tasks` hỗ trợ quản lý công việc cá nhân với các tiêu chuẩn khắt khe:

- Khóa chính toàn cầu UUID v4 tự sinh từ cơ sở dữ liệu PostgreSQL.
- Thẩm định dữ liệu đầu vào nghiêm ngặt (`title` từ 1 đến 255 ký tự, `category` bắt buộc, `isCompleted` mặc định false).
- 5 Endpoints RESTful tại `/api/v1/tasks` với mã HTTP chuẩn xác (`200`, `201`, `204`, `400`, `404`, `500`).
- Hỗ trợ lọc theo trạng thái và danh mục qua query parameters.
- Quản lý di trú qua TypeORM Migration và kiểm thử tự động 100% PASS.

B.2 Mã Nguồn Hiện Thực Hóa Chi Tiết

1. Thực thể TaskEntity: src/modules/tasks/task.entity.ts

```
import {
  Column,
  CreateDateColumn,
  Entity,
  PrimaryGeneratedColumn,
  UpdateDateColumn,
} from 'typeorm';

@Entity('tasks')
export class TaskEntity {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ type: 'varchar', length: 255 })
```

```
  title: string;

  @Column({ type: 'varchar' })
  category: string;

  @Column({ type: 'text', nullable: true })
  description: string | null;

  @Column({ name: 'is_completed', type: 'boolean', default: false })
  isCompleted: boolean;

  @CreateDateColumn({ name: 'created_at' })
  createdAt: Date;

  @UpdateDateColumn({ name: 'updated_at' })
  updatedAt: Date;
}
```

2. Data Transfer Objects: src/modules/tasks/dto/

```
// CreateTaskDto
export class CreateTaskDto {
  @IsString()
  @IsNotEmpty()
  @MaxLength(255)
  title: string;

  @IsString()
  @IsNotEmpty()
  category: string;

  @IsOptional()
  @IsString()
  description?: string;
}

// UpdateTaskDto (Mọi trường là Optional cho PATCH)
export class UpdateTaskDto {
  @IsOptional()
  @IsString()
  @IsNotEmpty()
  @MaxLength(255)
  title?: string;

  @IsOptional()
  @IsString()
  @IsNotEmpty()
  category?: string;
```

```
  @IsOptional()
  @IsString()
  description?: string;

  @IsOptional()
  @IsBoolean()
  isCompleted?: boolean;
}

// FindTasksDto (Nhận chuỗi query string an toàn)
export class FindTasksDto {
  @IsOptional()
  @IsIn(['true', 'false'])
  isCompleted?: string;

  @IsOptional()
  @IsString()
  category?: string;
}
```

3. Tầng Điều Khiển: src/modules/tasks/tasks.controller.ts

```
@Controller('v1/tasks')
export class TasksController {
  constructor(private readonly tasksService: TasksService) {}

  @Post()
  create(@Body() createTaskDto: CreateTaskDto) {
    return this.tasksService.create(createTaskDto);
  }

  @Get()
  findAll(@Query() query: FindTasksDto) {
    return this.tasksService.findAll(query);
  }

  @Get(':id')
  findOne(@Param('id', ParseUUIDPipe) id: string) {
    return this.tasksService.findOne(id);
  }

  @Patch(':id')
  update(
    @Param('id', ParseUUIDPipe) id: string,
    @Body() updateTaskDto: UpdateTaskDto,
  ) {
    return this.tasksService.update(id, updateTaskDto);
  }
```

```
  @Delete(':id')
  @HttpCode(HttpStatus.NO_CONTENT)
  remove(@Param('id', ParseUUIDPipe) id: string) {
    return this.tasksService.remove(id);
  }
}
```

4. Tầng Dịch Vụ: src/modules/tasks/tasks.service.ts

```
@Injectable()
export class TasksService {
  constructor(
    @InjectRepository(TaskEntity)
    private readonly tasksRepository: Repository<TaskEntity>,
  ) {}

  async create(createTaskDto: CreateTaskDto): Promise<TaskEntity> {
    const task = this.tasksRepository.create(createTaskDto);
    return this.tasksRepository.save(task);
  }

  async findAll(query: FindTasksDto): Promise<TaskEntity[]> {
    const where: Partial<TaskEntity> = {};

    if (query.isCompleted !== undefined) {
      where.isCompleted = query.isCompleted === 'true';
    }

    if (query.category !== undefined) {
      where.category = query.category;
    }

    return this.tasksRepository.find({ where });
  }

  async findOne(id: string): Promise<TaskEntity> {
    const task = await this.tasksRepository.findOneBy({ id });
    if (!task) {
      throw new NotFoundException(`Tasks with ID ${id} not found`);
    }
    return task;
  }

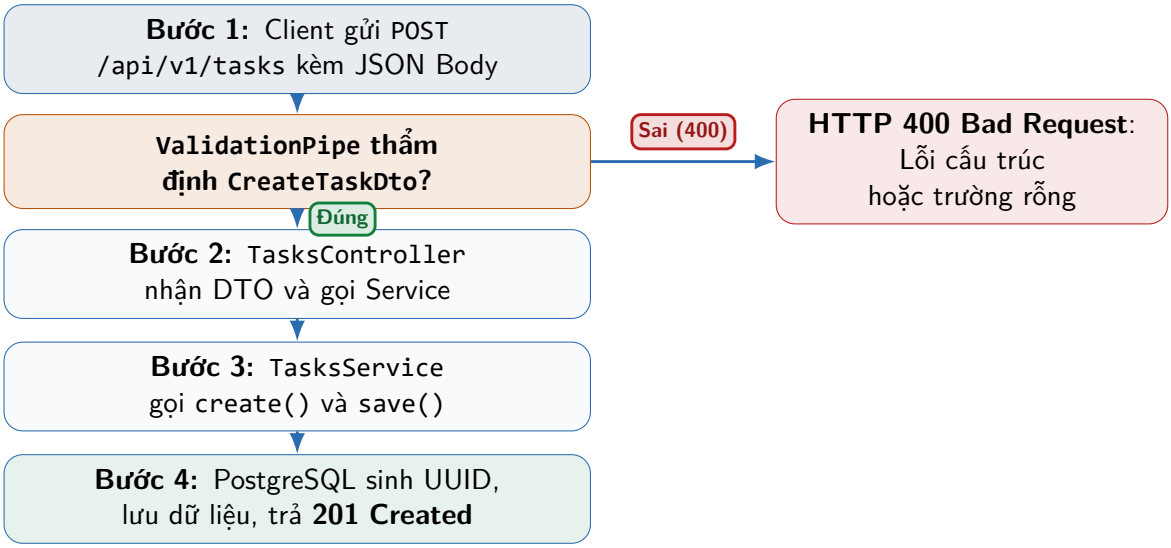
  async update(id: string, updateTaskDto: UpdateTaskDto): Promise<TaskEntity> {
    const task = await this.tasksRepository.preload({ id, ...updateTaskDto });
    if (!task) {
      throw new NotFoundException(`Tasks with ID ${id} not found`);
    }
    return this.tasksRepository.save(task);
```

```
  }

  async remove(id: string): Promise<void> {
    const result = await this.tasksRepository.delete(id);
    if (result.affected === 0) {
      throw new NotFoundException(`Tasks with ID ${id} not found`);
    }
  }
}
```

B.3 Lược Đồ Luồng Xử Lý 5 Endpoints Chuẩn Hóa (Resize 95%)

B.3.1 1. Luồng Tạo Mới Công Việc (POST /api/v1/tasks)



Hình B.1: Lược đồ chuẩn hóa luồng xử lý Tạo mới công việc (POST)

B.3.2 2. Luồng Lấy Danh Sách & Lọc Công Việc (GET /api/v1/tasks)

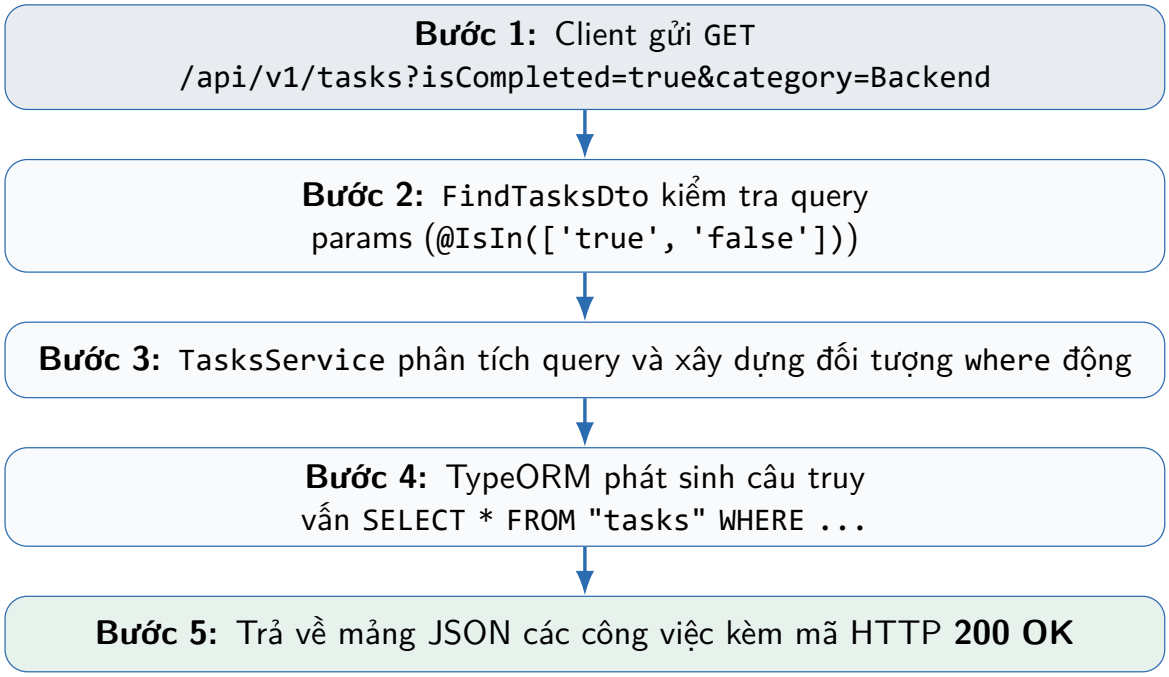
B.3.3 3. Luồng Xem Chi Tiết Công Việc Theo ID (GET /api/v1/tasks/:id)

B.3.4 4. Luồng Cập Nhật Từng Phần (PATCH /api/v1/tasks/:id)

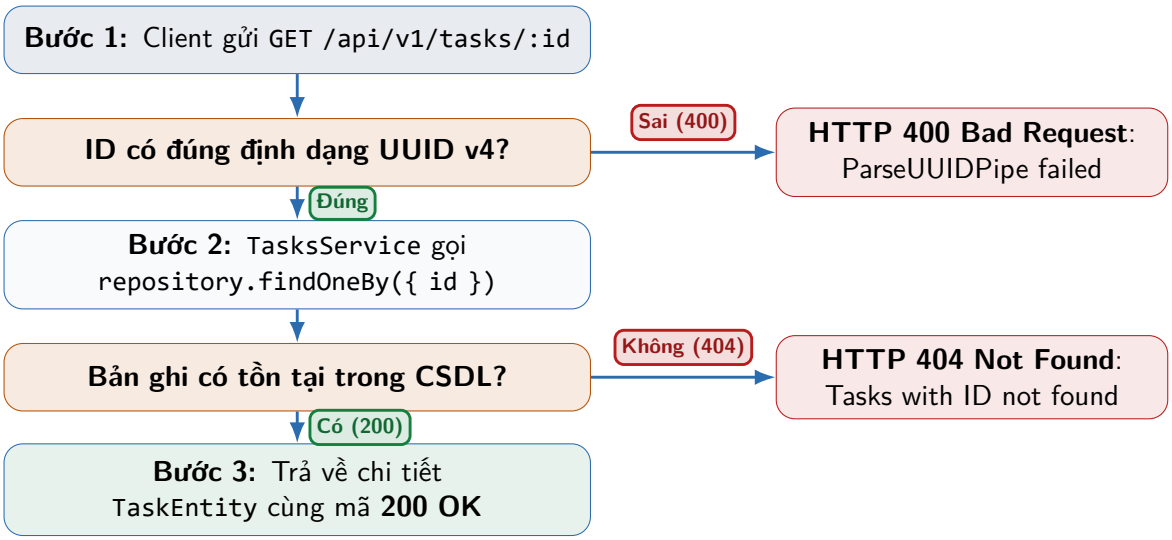
B.3.5 5. Luồng Xóa Bản Ghi Tối Ưu (DELETE /api/v1/tasks/:id)

B.4 Kết Quả Kiểm Thử Đơn Vị Tự Động (Jest 100% PASS)

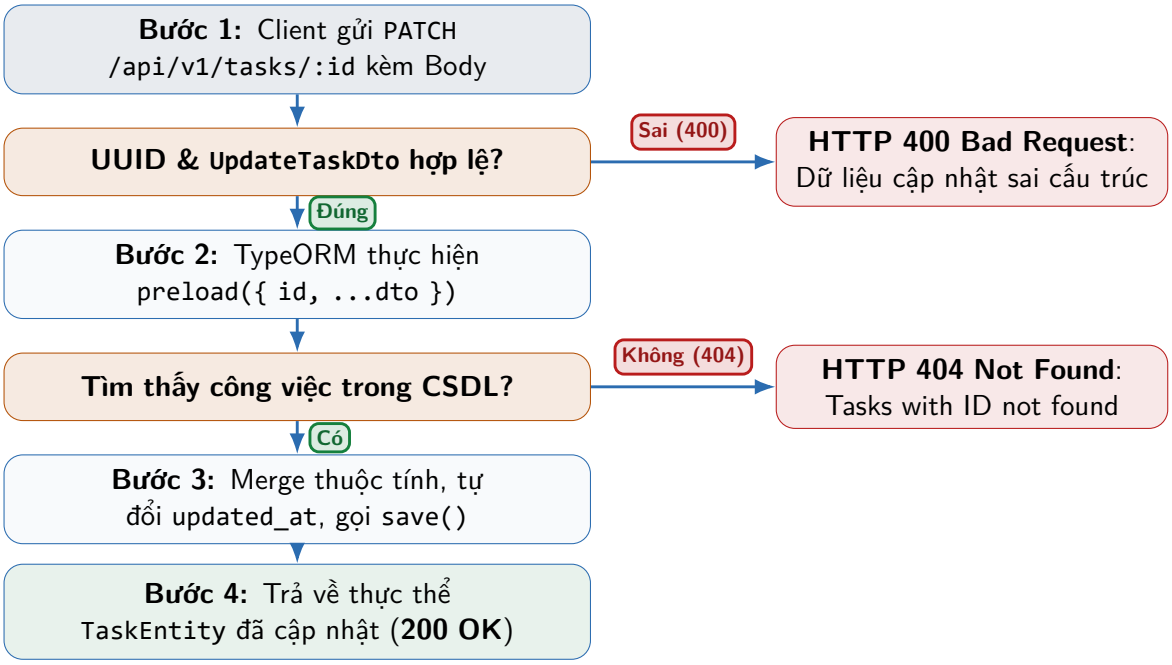
Chạy lệnh `npm test src/modules/tasks` cho kết quả hoàn hảo:



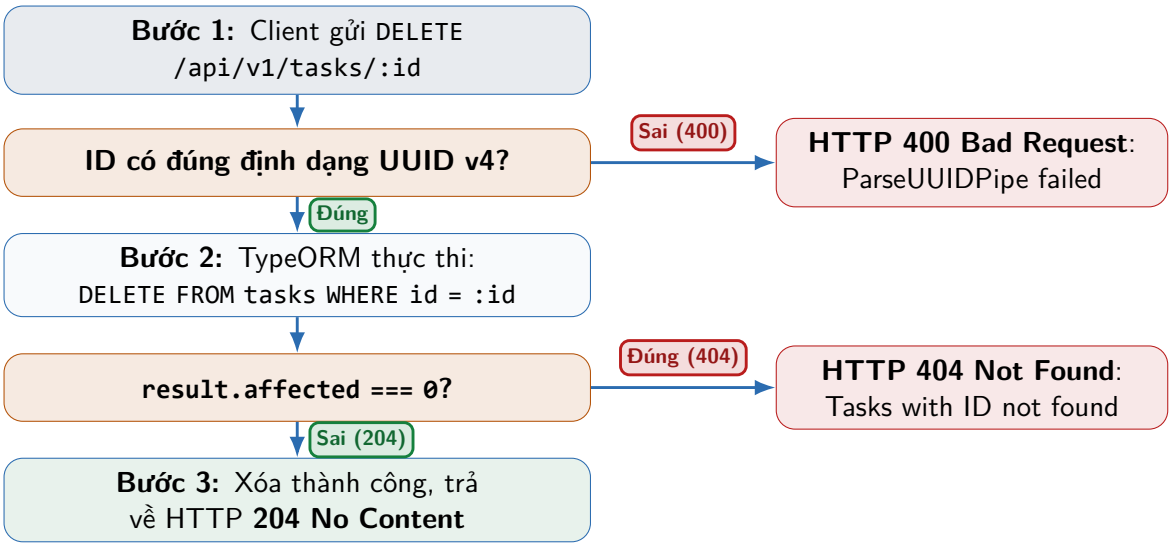
Hình B.2: Lược đồ chuẩn hóa luồng xử lý Lấy danh sách và Lọc công việc (GET)



Hình B.3: Lược đồ chuẩn hóa luồng xử lý Xem chi tiết công việc theo ID



Hình B.4: Lược đồ chuẩn hóa luồng xử lý Cập nhật từng phần (PATCH)



Hình B.5: Lược đồ chuẩn hóa luồng xử lý Xóa công việc tối ưu (DELETE)

Kết quả kiểm thử thực tế từ terminal

```
PASS src/modules/tasks/tasks.service.spec.ts
  TasksService
    √ creates a task (55 ms)
    √ filters tasks by completion state and category (9 ms)
    √ throws NotFoundException when a task does not exist (33 ms)
    √ updates an existing task (8 ms)
    √ throws NotFoundException when deleting a missing task (6 ms)

Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
Snapshots:   0 total
Time:        18.723 s
```

CHƯƠNG C

Bộ Câu Hỏi Khảo Sát Kỹ Thuật & Đánh Giá Thiết Kế Hệ Thống (FAQ)

Phần phụ lục này tổng hợp 16 câu hỏi kiến trúc và kỹ thuật chuyên sâu thường được các Hội đồng Công nghệ, Chuyên gia Thẩm định Kiến trúc (*Principal/Lead Architects*) chất vấn:

Câu hỏi 1: Tại sao lại sinh UUID tại Database thay vì sinh tại NestJS?

Dự án lựa chọn sinh UUID tại tầng cơ sở dữ liệu (`DEFAULT uuid_generate_v4()`) nhằm bảo đảm tính toàn vẹn dữ liệu (*Data Integrity*). Khi chèn dữ liệu trực tiếp bằng SQL, file migration, hay seed script, cột `id` luôn được tự động sinh UUID hợp lệ mà không bao giờ bị null, đồng thời giảm tải tính toán CPU cho Node.js event loop.

Câu hỏi 2: Làm sao hệ thống ngăn chặn việc người dùng cố tình chèn thêm các trường lạ?

Hệ thống áp dụng cơ chế phòng thủ 2 lớp (*Two-layer Defense*):

- **Lớp 1 - ValidationPipe với whitelist: true:** Tự động loại bỏ toàn bộ các thuộc tính trong payload không được định nghĩa decorator trong DTO.
- **Lớp 2 - TypeORM Entity Mapping:** TypeORM chỉ map những cột được khai báo decorator `@Column()` vào câu lệnh SQL INSERT/UPDATE.

Câu hỏi 3: Sự khác biệt giữa ValidationPipe toàn cục và ParseUUIDPipe tại tham số là gì?

`ValidationPipe` toàn cục chuyên xử lý kiểm tra cấu trúc của Body payload và Query object thông qua các class DTO. Trong khi đó, `ParseUUIDPipe` hoạt động cục bộ tại tham số đường dẫn `@Param('id', ParseUUIDPipe)`, chặn đứng các request có ID sai định dạng ngay tại tầng Pipe, bảo vệ database khỏi lỗi cú pháp runtime.

Câu hỏi 4: Tại sao trong FindTasksDto thì trường isCompleted lại là kiểu string?

Chuẩn giao thức HTTP truyền tải toàn bộ Query Parameters dưới dạng chuỗi (`string`). Nếu khai báo kiểu `boolean` mà không dùng cơ chế ép kiểu tùy biến, chuỗi `"false"` sẽ bị JavaScript đánh giá là *truthy* (chuỗi không rỗng), dẫn tới sai lệch logic nghiêm trọng. Việc nhận kiểu `string` và kiểm tra bằng `@IsIn(['true', 'false'])` là giải pháp an toàn tuyệt đối.

Câu hỏi 5: Tại sao trong phương thức update lại dùng preload mà không phải gán thủ công

`preload` của TypeORM thực hiện hai thao tác nguyên tử: nạp thực thể hiện có từ database và hợp nhất các thuộc tính mới từ DTO lên thực thể đó. Nếu ID không tồn tại, `preload` trả về `undefined`, giúp phát hiện và ném lỗi 404 gọn gàng mà không cần viết hàng loạt câu lệnh `if (dto.field !== undefined)`.

Câu hỏi 6: Tại sao phương thức remove lại dùng delete() thay vì remove()?

`repository.remove(entity)` đòi hỏi phải tải toàn bộ thực thể lên bộ nhớ (tốn 1 câu `SELECT` + 1 câu `DELETE`). Ngược lại, `repository.delete(id)` bắn thẳng lệnh `DELETE FROM tasks WHERE id = :id` xuống PostgreSQL, chỉ tốn đúng 1 round-trip mạng, mang lại hiệu năng cao gấp đôi.

Câu hỏi 7: Tầng Unit Test của bạn kiểm thử cái gì? Tại sao không test trực tiếp với Postg

Unit Test có mục tiêu cô lập logic của tầng đang kiểm tra (`TasksService`) và giả lập (*mock*) toàn bộ các dependency bên ngoài (`Repository<TaskEntity>`). Việc test không phụ thuộc vào database giúp unit test chạy cực nhanh (chỉ vài mili-giây), độc lập với môi trường mạng hay trạng thái dữ liệu có sẵn. Kiểm thử tương tác trực tiếp với PostgreSQL thuộc về tầng Integration/E2E Test.

Câu hỏi 8: Làm sao đảm bảo cột updatedAt luôn được cập nhật khi có thay đổi?

Decorator `@UpdateDateColumn({ name: 'updated_at' })` thông báo cho TypeORM cơ chế Dirty Checking. Bất cứ khi nào gọi phương thức `repository.save()`, TypeORM sẽ tự động chèn giá trị thời gian hiện tại (`CURRENT_TIMESTAMP`) vào câu lệnh SQL `UPDATE`.

Câu hỏi 9: Xóa mềm ảnh hưởng như thế nào tới ràng buộc Duy nhất (Unique Constraint)

Khi một trường (ví dụ: `email` hoặc `slug`) có ràng buộc `UNIQUE`, nếu người dùng xóa mềm bản ghi (gán `deleted_at = NOW()`), bản ghi đó vẫn tồn tại trong bảng. Nếu sau đó người dùng đăng ký lại cùng email đó, câu lệnh chèn mới sẽ bị lỗi vi phạm ràng buộc duy nhất!
Giải pháp Doanh nghiệp: Dự án sử dụng **Partial Index (Chỉ mục bộ phận)** trong PostgreSQL:

```
CREATE UNIQUE INDEX uq_users_email_active ON users (email) WHERE deleted_at IS NULL;
```
(C.1)

Chỉ mục này chỉ áp dụng ràng buộc duy nhất cho các bản ghi đang hoạt động, cho phép tái sử dụng email nếu bản ghi cũ đã bị xóa mềm!

Câu hỏi 10: Tại sao dự án lựa chọn thuật toán Argon2 thay vì Bcrypt?

Bcrypt chỉ tiêu tốn năng lực xử lý CPU thuần túy. Với sự phát triển của các dàn siêu máy tính trang bị hàng ngàn GPU hoặc chip ASIC chuyên dụng, kẻ tấn công có thể thử hàng tỷ mật khẩu Bcrypt mỗi giây nhờ khả năng tính toán song song cực đại.

Ngược lại, **Argon2** là thuật toán băm bộ nhớ nặng (*Memory-Hard*). Nó buộc mỗi lượt băm phải chiếm dụng một lượng bộ nhớ RAM lớn (mặc định 64 MB). Do GPU và ASIC bị nghẽn cổ chai nghiêm trọng về băng thông bộ nhớ RAM khi chạy song song hàng ngàn luồng, chi phí phần cứng để giải mã vét cạn Argon2 bị đội lên hàng triệu lần, triệt tiêu hoàn toàn tính khả thi của các cuộc tấn công bẻ khóa.

Câu hỏi 11: Sự khác biệt về quản lý bộ nhớ giữa Scope.DEFAULT và Scope.REQUEST là

`Scope.DEFAULT` là mô hình Singleton: IoC Container chỉ khởi tạo duy nhất một thể hiện của Service khi ứng dụng khởi động và tái sử dụng nó cho toàn bộ hàng triệu request tiếp theo. Mức chiếm dụng RAM không đổi và không sinh rác bộ nhớ.
Ngược lại, `Scope.REQUEST` buộc NestJS phải tạo mới một thể hiện của Service (và toàn bộ các phụ thuộc liên quan do hiệu ứng lan truyền *Bubble-Up*) cho mỗi kết nối HTTP tới, sau đó hủy đi khi gửi xong response. Với hệ thống 10,000 req/s, điều này tạo ra 10,000 đối tượng mới mỗi giây, gây áp lực khủng khiếp lên bộ thu gom rác (*Garbage Collector*), dẫn tới hiện tượng CPU Spike và tắc nghẽn Event Loop. Do đó, chỉ sử dụng `Scope.REQUEST` khi thực sự cần lưu trữ trạng thái đặc thù của request (như `PaymentFactory`).

Câu hỏi 12: Khi nào xảy ra Circular Dependency và giải quyết như thế nào?

Phụ thuộc vòng xảy ra khi Module A phụ thuộc Module B, nhưng đồng thời Module B cũng phụ thuộc ngược lại Module A (hoặc Service A inject Service B và Service B inject Service A). Khi NestJS dựng đồ thị phụ thuộc Tô pô, nó sẽ rơi vào vòng lặp vô tận và ném ngoại lệ `Circular dependency detected`.
Cách xử lý: Sử dụng hàm `forwardRef()` tại cả hai phía trong mảng `imports` hoặc decorator `@Inject(forwardRef(() => OtherService))`. Hàm này trì hoãn việc giải quyết phụ thuộc cho tới khi cả hai module/service đã được khởi tạo vỏ bọc.

Câu hỏi 13: Tại sao cờ synchronize: true trong TypeORM bị cấm tuyệt đối trên Productio

Cờ `synchronize: true` tự động so sánh mã nguồn TypeScript với cấu trúc bảng mỗi khi ứng dụng khởi động và thực thi các lệnh `ALTER TABLE` hoặc `DROP COLUMN` trực tiếp. Nếu một lập trình viên vô tình đổi tên một trường trong Entity, TypeORM sẽ thực thi lệnh `DROP COLUMN old_field` và `ADD COLUMN new_field`, **xóa sạch toàn bộ dữ liệu lịch sử của hàng triệu khách hàng chỉ trong 1 giây mà không có bất kỳ cảnh báo nào**! Do đó, trên Production bắt buộc phải đặt `synchronize: false` và chỉ cập nhật schema qua TypeORM Migration.

Câu hỏi 14: Hệ thống ngăn ngừa lỗi tràn ổ đĩa (Disk Full) do tập tin nhật ký Log như thế

Hệ thống sử dụng module vận hành `winston-daily-rotate-file` với 3 cơ chế tự vệ:

1. `maxSize: '50m'`: Khi tệp log đạt dung lượng 50 MB, tệp sẽ tự động được đóng lại và mở tệp mới.
2. `maxFiles: '15d'`: Các tệp log cũ hơn 15 ngày sẽ được hệ điều hành tự động xóa bỏ vĩnh viễn khỏi ổ cứng.

3. `zippedArchive: true`: Tự động nén các tệp log đã đóng thành định dạng `.gz`, giảm dung lượng lưu trữ tới 90%.

Câu hỏi 15: Cơ chế chuyển ngữ lỗi động trong ErrorService hoạt động ra sao?

Khi DTO ném lỗi validation, `ErrorService` trích xuất mã quy chuẩn (ví dụ `isNotEmpty`, `maxLength`) từ danh sách `constraints`. Sau đó, nó gọi `I18nService.translate()` tra cứu vào từ điển `src/i18n/vi/errors.json` hoặc `en/errors.json` dựa trên header `Accept-Language` của request. Dữ liệu trả về cho Client là thông điệp tự nhiên, bản địa hóa hoàn toàn mà không cần viết điều kiện thủ công.

Câu hỏi 16: Làm sao xử lý tính Idempotent khi cổng thanh toán gửi trùng lặp Webhook IP

Do đặc thù mạng Internet không tin cậy, cổng MoMo có thể gửi lại cùng một Webhook IPN nhiều lần nếu máy chủ NestJS phản hồi chậm. Nếu không xử lý tính bất biến (*Idempotency*), đơn hàng có thể bị cộng tiền hai lần hoặc xuất kho hai lần!
Giải pháp: Backend kiểm tra trạng thái hiện tại của đơn hàng trước khi cập nhật. Nếu trạng thái đã là `COMPLETED`, hệ thống lập tức trả về mã `HTTP 200 OK` kèm mã thành công mà không thực hiện lại bất kỳ giao dịch trừ kho hay cộng doanh thu nào.

CHƯƠNG D

Sổ Tay Vận Hành & Khởi Động Môi Trường Thực Địa

D.1 Bảng Tra Cứu Biến Môi Trường Cấu Hình (.env)

Toàn bộ cấu hình hệ thống được quản lý tập trung qua biến môi trường độc lập:

Bảng D.1: Bảng từ điển các biến môi trường cốt lõi trong hệ thống

Tên Biến (.env)	Giá trị mặc định	Mô tả chức năng kỹ thuật
PORT	3000	Cổng lắng nghe HTTP của máy chủ NestJS.
NODE_ENV	development	Môi trường thực thi (development, production, test).
DATABASE_HOST	localhost	Địa chỉ IP hoặc máy chủ cơ sở dữ liệu PostgreSQL.
DATABASE_PORT	5432	Cổng kết nối dịch vụ PostgreSQL.
DATABASE_USER	postgres	Tài khoản kết nối cơ sở dữ liệu.
DATABASE_PASSWORD	postgres	Mật khẩu kết nối an toàn.
DATABASE_NAME	boilerplate	Tên lược đồ cơ sở dữ liệu chính của dự án.
JWT_SECRET	secretKey123	Khóa bí mật băm chữ ký HMAC-SHA256 của JWT Token.
JWT_EXPIRATION	1d	Thời hạn hiệu lực của Access Token (1 ngày).
CLOUDINARY_NAME	demo-cloud	Tên định danh tài khoản CDN Cloudinary.
MOMO_SECRET_KEY	momoSecret	Khóa bí mật ký số giao dịch MoMo Payment Gateway.
GOOGLE_CLIENT_ID	google-id.apps	Khóa định danh ứng dụng Google Identity OAuth2.

D.2 Hướng Dẫn Khởi Chạy 4 Bước Chuẩn Doanh Nghiệp

1. **Khởi động Database PostgreSQL với Docker**:

Khởi chạy Docker Container

```
docker compose up -d init_db
```

2. **Thực thi Migrations đồng bộ cấu trúc bảng**:

Chạy TypeORM Migration

```
npm run typeorm:run-migration
```

3. **Nạp dữ liệu mẫu bắt buộc với Hygen Seeding**:

Khởi tạo dữ liệu mẫu

```
npm run seed:run
```

4. **Khởi chạy máy chủ phát triển & kiểm thử Swagger**:

Khởi động NestJS Development Mode

```
npm run start:dev
```

Mở trình duyệt web và truy cập giao diện kiểm thử tương tác tại địa chỉ: `http://localhost:3000/docs`.

D.3 Cẩm Nang Xử Lý Sự Cố Thực Địa (Troubleshooting Playbook)

Khi triển khai và vận hành hệ thống trên các môi trường máy chủ thực tế, kỹ sư có thể gặp phải một số lỗi kinh điển dưới đây:

- **Lỗi Native Addon Argon2 trên Alpine Linux**: Khi chạy Docker container với hình ảnh `node:alpine`, bạn có thể gặp lỗi `Error loading shared library ld-musl-x86_64.so`. **Khắc phục:** Chạy lệnh `npm rebuild argon2 --build-from-source` để biên dịch lại mã nguồn C++ tương thích với thư viện musl libc.
- **Lỗi ECONNREFUSED khi kết nối PostgreSQL**: Ứng dụng NestJS khởi động nhanh hơn cơ sở dữ liệu PostgreSQL trong Docker Compose. **Khắc phục:** Sử dụng cấu hình `depends_on` kèm kiểm tra `healthcheck` trong tệp `docker-compose.yml` để bảo đảm PostgreSQL đã sẵn sàng nhận kết nối trước khi khởi chạy tiến trình NestJS.
- **Lỗi Metadata Reflection**: Xuất hiện thông báo lỗi `Cannot read property of undefined (reading 'design:paramtypes')`. **Khắc phục:** Đảm bảo dòng lệnh `import 'reflect-metadata';` luôn được đặt tại dòng đầu tiên tuyệt đối của tệp tin `src/main.ts`.

D.4 Bảng Kiểm Định Chất Lượng Hoàn Thành Bài Tập (Task2 Checklist)

Bảng D.2: Bảng đối chiếu tiêu chuẩn kỹ thuật Task2.md và tình trạng hoàn thành

Tiêu chí kỹ thuật	**Hiện thực hóa trong mã nguồn dự án**	**Đánh giá**
Entity chuẩn UUID v4	`TaskEntity` với `@PrimaryGeneratedColumn('uuid')`	**ĐẠT 100%**
Validation Title (1÷255)	`@IsNotEmpty()`, `@MaxLength(255)` trong DTO	**ĐẠT 100%**
POST `/api/v1/tasks` (201)	`TasksController.create()` trả về đúng mã 201 Created	**ĐẠT 100%**
GET `/api/v1/tasks` (Filter)	Hỗ trợ lọc `isCompleted` và `category` qua Query	**ĐẠT 100%**
GET `/api/v1/tasks/:id`	Thẩm định UUID qua `ParseUUIDPipe`, 404 khi không thấy	**ĐẠT 100%**
PATCH `/api/v1/tasks/:id`	Cập nhật từng phần qua `preload`, tự đổi `updatedAt`	**ĐẠT 100%**
DELETE `/api/v1/tasks/:id`	Xóa tối ưu qua `delete()`, trả về mã `204 No Content`	**ĐẠT 100%**
Chuẩn Exception 400	Tự động xử lý qua `ValidationPipe` & `ParseUUIDPipe`	**ĐẠT 100%**
Chuẩn Exception 404	Ném `NotFoundException('Tasks with ID ${id} not found')`	**ĐẠT 100%**
Unit Test toàn diện	5 ca kiểm thử trong `tasks.service.spec.ts` chạy qua 100%	**ĐẠT 100%**